MQTT Zig チュートリアル

Zigで学ぶMQTTプロトコル

対象: Zig 0.16 / MQTT v3.1.1

全19章 + 付録4本

目次

第I部 MQTTプロトコルの基礎

第1章 MQTTの概要 — Pub/Subモデルと3つの役割

第2章 バイナリプロトコルの基礎 — 固定ヘッダとRemaining Length

第3章 ZigでTCP通信 — std.Io.netによるソケットプログラミング

第II部 MQTTパケット

第4章 CONNECT / CONNACK — 接続パケットの構造と実装

第5章 PUBLISHフロー — メッセージ送受信とQoS

第6章 SUBSCRIBE / UNSUBSCRIBE — トピック購読の仕組み

第III部 ブローカーロジック

第7章 トピックワイルドカード — +と#のマッチングアルゴリズム

第8章 QoS配信保証 — パケットID管理と再送

第9章 キープアライブ — PINGREQ/PINGRESPとタイムアウト

第IV部 統合

第10章 ブローカーの全体設計 — コンポーネントとスレッドモデル

第11章 セッション管理 — Clean SessionとクライアントID

第12章 Retained/Willメッセージ — 保持メッセージと遺言

第V部 デモとテスト

第13章 複数クライアントの統合 — 1プロセスデモとメッセージルーティング

第14章 統合テスト — TCPを使ったテストパターン

第VI部 発展編

第15章 イベント駆動ブローカー — io.async()とIo.Groupによる非同期化

第16章 Io.Queueによるメッセージング — ロックフリーなメッセージルーティング

第17章 Graceful Shutdown — 安全な停止パターンとWillメッセージ

第18章 Zigらしい設計パターン — Juicy Main・RwLock・comptime活用

第19章 ベンチマークレポート — Debug vs ReleaseFast vs mosquitto 性能比較

付録

付録A. 用語集

付録B. MQTT v3.1.1 仕様サマリ

付録C. 前提知識
付録D. ネットワークデバッグ手法

はじめに

学習パス

本チュートリアルは 6 部構成・全 18 章で、段階的に MQTT Broker を構築していきます。

Part I — 基礎

プロトコルの概要と、Zig によるバイナリ処理・TCP 通信の基礎を学びます。

章	タイトル	学ぶこと
01	MQTT 概要	MQTT の歴史、用途、Pub/Sub モデル
02	バイナリプロトコル	固定ヘッダー、Remaining Length、パケット構造
03	TCP の基礎	Zig の `std.Io.net`、ソケット、Stream の読み書き

Part II — コアパケット

MQTT 通信の中核となるパケットを実装します。

章	タイトル	学ぶこと
04	CONNECT / CONNACK	接続ハンドシェイク、バリデーション
05	PUBLISH フロー	メッセージ発行、QoS 0 の実装
06	SUBSCRIBE / UNSUBSCRIBE	トピック購読と購読解除

Part III — 高度な機能

QoS 制御やトピックのワイルドカードマッチングなど、高度な機能を追加します。

章	タイトル	学ぶこと
07	トピックワイルドカード	`+` / `#` マッチング、トピックツリー
08	QoS レベル	QoS 0/1 の実装、PUBACK
09	Keep Alive	PINGREQ / PINGRESP、タイムアウト管理

Part IV — Broker アーキテクチャ

本格的な Broker の設計と実装に取り組みます。

章	タイトル	学ぶこと
10	Broker アーキテクチャ	コンポーネント設計、接続管理
11	セッション管理	Clean Session、セッション永続化
12	Retain / Will メッセージ	保持メッセージ、遺言メッセージ

Part V — 統合と完成

複数クライアントの同時処理とテストを行い、プロジェクトを完成させます。

章	タイトル	学ぶこと
13	マルチクライアント	複数接続の同時処理
14	統合テスト	エンドツーエンドテスト、異常系テスト

Part VI — 発展編

`std.Io` を活用した高度な設計パターンとプロダクション品質の機能を実装します。

章	タイトル	学ぶこと
15	イベント駆動ブローカー	`std.Io.Event` によるイベント駆動設計
16	Io.Queue によるメッセージング	`std.Io.Queue(T)` によるスレッド間メッセージパッシング
17	Graceful Shutdown	シグナルハンドリング、安全な終了処理
18	Zigらしい設計パターン	`std.process.Init` 、Cancelable error、Future パターン
19	ベンチマークレポート	Debug vs ReleaseFast vs mosquitto 性能比較

第I部 MQTTプロトコルの基礎

MQTTの概念とバイナリプロトコルを学びます

第1章 MQTTの概要

Pub/Subモデルと3つの役割

学習目標

- MQTT プロトコルの目的と歴史を理解する
- Pub/Sub（出版/購読）モデルの仕組みを把握する
- Broker / Publisher / Subscriber の3つの役割を説明できる
- HTTP や WebSocket との違いを整理し、MQTT が適するユースケースを挙げられる
- MQTT v3.1.1 の基本仕様を概観する
- 本プロジェクトのソースコード構成を把握する

MQTTとは何か

MQTT（Message Queuing Telemetry Transport）は、**軽量なメッセージングプロトコル**である。 1999年に IBM の Andy Stanford-Clark と Arlen Nipper によって設計された。 当時の目的は、帯域幅が極めて限られた衛星回線上で 石油パイプラインのセンサーデータを効率よく転送することだった。

2014年に OASIS 標準として MQTT v3.1.1 が策定され、 IoT（Internet of Things）領域のデファクトプロトコルとして広く採用されている。 2019年には MQTT v5.0 も策定されたが、本プロジェクトでは v3.1.1 を対象とする。

主な特徴:

- **バイナリプロトコル** -- テキストベースの HTTP と異なり、ヘッダが極めてコンパクト（最小2バイト）
- **TCP 上で動作** -- 信頼性のあるトランスポート層を前提とする
- **非同期通信** -- Publisher と Subscriber は互いの存在を知る必要がない
- **QoS レベル** -- メッセージ配信の保証レベルを 0/1/2 の3段階で選べる
- **Keep Alive** -- 長時間接続を維持するための仕組みが組み込まれている

Pub/Sub モデル

従来のクライアント・サーバモデルとの違い

HTTP に代表されるリクエスト/レスポンスモデルでは、 クライアントがサーバに直接リクエストを送り、サーバが応答を返す。 この方式では、サーバ側からクライアントに対して自発的にデータを送ることが難しい。

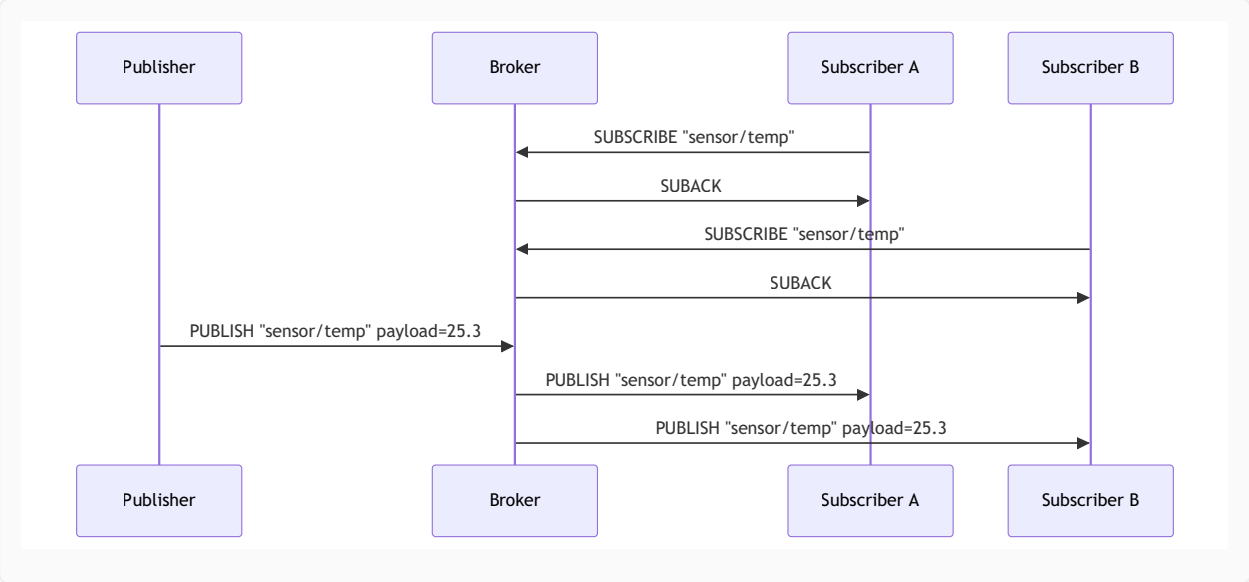
MQTT の Pub/Sub（Publish/Subscribe）モデルでは、 **Broker（仲介者）**を中心に通信が行われる。

```
Publisher ──PUBLISH──> Broker ──PUBLISH──> Subscriber
                          |
                          +──PUBLISH──> Subscriber
```

メッセージは**トピック**（例: `sensor/temperature`）に対して発行（Publish）され、 そのトピックを購読（Subscribe）しているクライアントに配信される。

メッセージフローの詳細

以下のシーケンス図は、基本的なメッセージフローを示す。



Pub/Sub モデルの利点

1. **疎結合**: Publisher は Subscriber の存在を知る必要がなく、逆もまた同様である
2. **スケーラビリティ**: Subscriber を追加しても Publisher の変更は不要
3. **非同期性**: Publisher と Subscriber が同時にオンラインである必要がない（QoS 1/2 の場合）

4. **多対多通信**: 1つのトピックに複数の Publisher と Subscriber が参加できる

3つの役割

1. Broker（ブローカー）

メッセージの中継・ルーティングを担当するサーバプロセス。 本プロジェクトでは `src/broker/` ディレクトリに実装している。

Broker の責務: - クライアントの接続管理（CONNECT / CONNACK） - トピックの購読管理（SUBSCRIBE / SUBACK） - メッセージの配信（PUBLISH の転送とルーティング） - Keep Alive の監視 - Retained メッセージの保持 - セッション管理

2. Publisher（パブリッシャー）

トピックに対してメッセージを発行するクライアント。 センサーデバイス、アプリケーションサーバ、API ゲートウェイなどが該当する。 本プロジェクトでは `src/publisher/main.zig` に実装例がある。

3. Subscriber（サブスクライバー）

トピックを購読し、メッセージを受信するクライアント。 ダッシュボード、ログ収集サービス、アラートシステムなどが該当する。 本プロジェクトでは `src/subscriber/main.zig` に実装例がある。

> **注意:** 1つのクライアントが Publisher と Subscriber の両方の役割を兼ねることができる。 例えば、温度センサーの値を PUBLISH しながら、設定変更のトピックを SUBSCRIBE する デバイスは珍しくない。

HTTP / WebSocket との比較

観点	HTTP	WebSocket	MQTT
通信モデル	リクエスト/レスポンス	双方向ストリーム	Pub/Sub（非同期）
ヘッダサイズ	数百バイト〜	2〜14バイト	最小2バイト
方向性	クライアント → サーバ	双方向	双方向（Broker 経由）
接続維持	基本はステートレス	長時間接続	長時間の TCP 接続を維持

観点	HTTP	WebSocket	MQTT
プッシュ通知	ポーリング必須	ネイティブ対応	ネイティブ対応
データ形式	テキスト（JSON/XML）	テキストまたはバイナリ	バイナリ（任意のペイロード）
配信保証	なし	なし	QoS 0/1/2
メッセージ境界	Content-Length で管理	フレーム単位	Remaining Length で管理
ブローカーの要否	不要	不要	必須

なぜ MQTT を選ぶのか

- **帯域幅が限られる環境**: モバイル通信や衛星回線
- **大量のデバイスとの通信**: IoT で数千〜数百万台のデバイスを管理
- **配信保証が必要**: QoS 1/2 による確実なメッセージ配信
- **サーバからのプッシュ**: クライアントがポーリングする必要がない

ユースケース

IoT / センサーデータ収集

工場の温度センサーが `factory/line1/temp` にデータを Publish し、 監視システムが Subscribe する。MQTT の軽量さにより、 リソースの限られたマイクロコントローラでも実装できる。

テレメトリ / 車両追跡

GPS トラッカーが位置情報を定期的に Publish し、 フリート管理システムがリアルタイムに受信する。QoS 1 を使えば、ネットワークの一時的な断絶後も再送が保証される。

チャット / 通知システム

ユーザーごとのトピック（`user/{id}/notifications`）を利用して、 プッシュ通知を配信する。Retained メッセージを使えば、 オフラインだったクライアントが接続した時点で最新の状態を取得できる。

ホームオートメーション

照明やエアコンの操作コマンドを MQTT トピック経由で送受信する。 Home Assistant などのプラットフォームは MQTT を標準的に統合している。

MQTT v3.1.1 の基本仕様

本プロジェクトは **MQTT v3.1.1**（OASIS Standard, 2014年10月）を対象とする。

パケット種別

MQTT では以下の14種類の制御パケットが定義されている:

値	パケット名	方向	説明
1	CONNECT	Client → Broker	接続要求
2	CONNACK	Broker → Client	接続応答
3	PUBLISH	双方向	メッセージ送信
4	PUBACK	双方向	QoS 1 確認応答
5	PUBREC	双方向	QoS 2 受信確認
6	PUBREL	双方向	QoS 2 解放
7	PUBCOMP	双方向	QoS 2 完了
8	SUBSCRIBE	Client → Broker	トピック購読要求
9	SUBACK	Broker → Client	購読応答
10	UNSUBSCRIBE	Client → Broker	購読解除要求
11	UNSUBACK	Broker → Client	購読解除応答
12	PINGREQ	Client → Broker	キープアライブ要求
13	PINGRESP	Broker → Client	キープアライブ応答
14	DISCONNECT	Client → Broker	切断通知

QoS レベル

- **QoS 0（At most once）**：送りっぱなし。最もオーバーヘッドが少ないが、ロスの可能性あり
- **QoS 1（At least once）**：PUBACK で確認。確実に配信されるが、重複の可能性あり
- **QoS 2（Exactly once）**：4ウェイハンドシェイク（PUBREC/PUBREL/PUBCOMP）。重複なしだがオーバーヘッド大

本プロジェクトでは QoS 0 と QoS 1 を実装対象としている。

Keep Alive

クライアントは CONNECT パケットで Keep Alive 間隔（秒）を指定する。 この間隔内に通信がなければ、クライアントは PINGREQ を送り、 Broker は PINGRESP を返す。Keep Alive の 1.5 倍の時間応答がなければ、 Broker は接続を切断する。

Retained メッセージ

PUBLISH の Retain フラグを 1 にすると、Broker はそのトピックの 最新メッセージを保持する。新たに Subscribe したクライアントは、 直ちにその保持メッセージを受信できる。

本プロジェクトの構成

```
mqtt_zig/
  src/
    mqtt/
      types.zig       -- QoS, PacketType, ConnectFlags 等の共有型定義
      packet.zig      -- 全パケット構造体と tagged union (Packet)
      codec.zig       -- パケットのエンコード/デコード
      topic.zig       -- トピックマッチングロジック
    broker/
      main.zig        -- Broker のエントリポイント (Juicy Main)
      server.zig      -- TCP リスナーと接続受付
      connection.zig  -- クライアント接続ハンドラ
      session.zig     -- セッション管理
      retain.zig      -- Retained メッセージストア
    client/
      client.zig      -- MQTT クライアントライブラリ
      transport.zig   -- TCP トランスポート層
    publisher/
      main.zig        -- Publisher のサンプル実装
    subscriber/
      main.zig        -- Subscriber のサンプル実装
```

```
build.zig           -- Zig ビルド定義
chapters/           -- 本チュートリアル
```

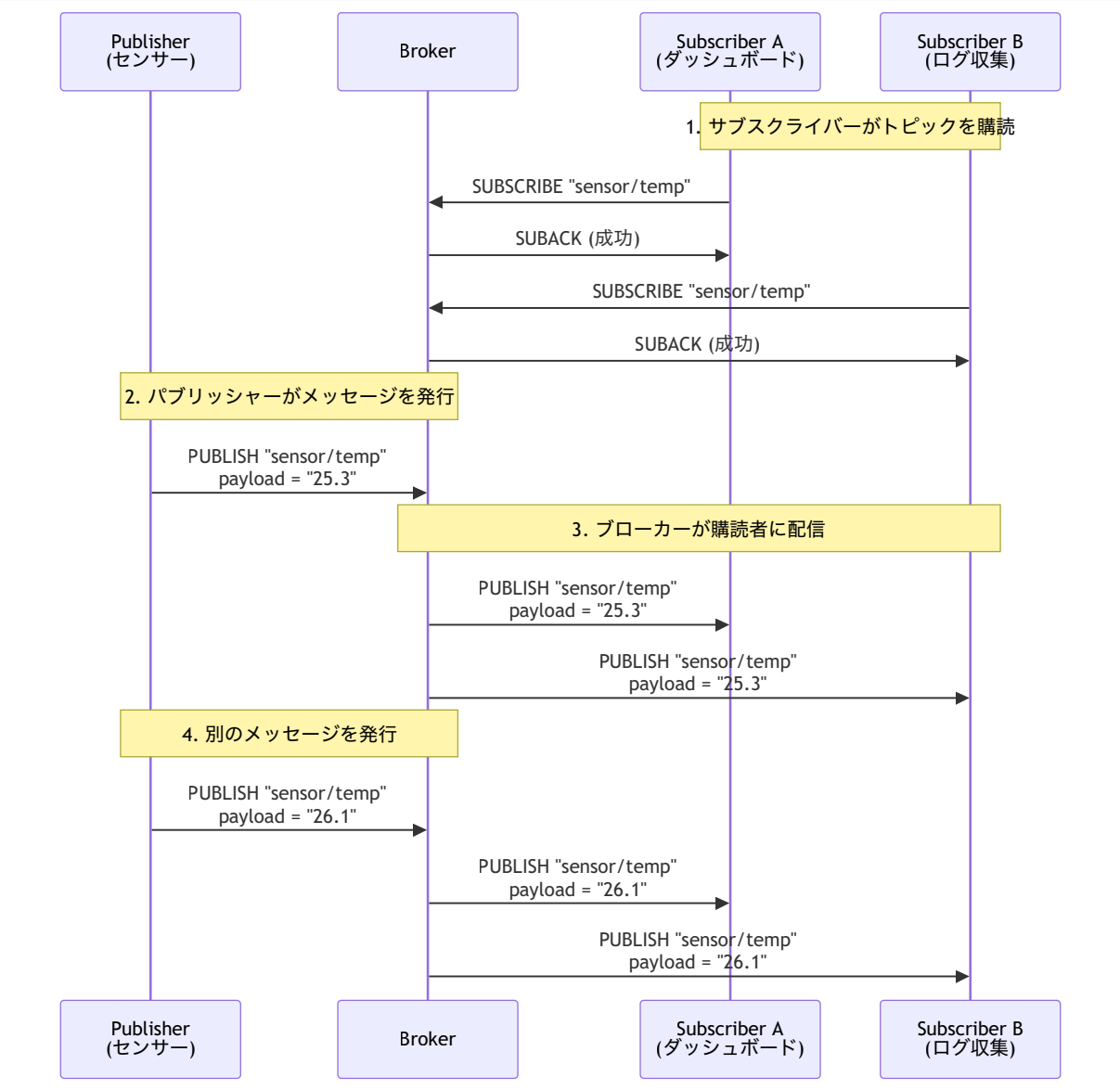
以降のチャプターでは、これらのソースコードを参照しながら MQTT プロトコルと Zig 0.16 の実装技法を学んでいく。

まとめ

- MQTT は**軽量・非同期・双方向**のメッセージングプロトコルである
- **Broker** を中心とした Pub/Sub モデルにより、疎結合な通信が実現できる
- HTTP は同期的なリクエスト/レスポンス、WebSocket は双方向ストリーム、MQTT は配信保証付きの Pub/Sub と、それぞれ異なる特性を持つ
- IoT / テレメトリ / 通知 / ホームオートメーションなど、リアルタイム通信シナリオに適している
- 本チュートリアルでは MQTT v3.1.1 を Pure Zig 0.16 で実装しながら学ぶ

次のチャプターでは、MQTT のバイナリプロトコルの構造を詳しく見ていく。

図


Publisher
(センサー)
Broker
Subscriber A
(ダッシュボード)
Subscriber B
(ログ収集)
1. サブスクライバーがトピックを購読
SUBSCRIBE "sensor/temp"
SUBACK (成功)
SUBSCRIBE "sensor/temp"
SUBACK (成功)
2. パブリッシャーがメッセージを発行
PUBLISH "sensor/temp"
payload = "25.3"
3. ブローカーが購読者に配信
PUBLISH "sensor/temp"
payload = "25.3"
PUBLISH "sensor/temp"
payload = "25.3"
4. 別のメッセージを発行
PUBLISH "sensor/temp"
payload = "26.1"
PUBLISH "sensor/temp"
payload = "26.1"
PUBLISH "sensor/temp"
payload = "26.1"
Publisher
(センサー)
Broker
Subscriber A
(ダッシュボード)
Subscriber B
(ログ収集)

第2章 バイナリプロトコルの基礎

固定ヘッダとRemaining Length

学習目標

- MQTT 固定ヘッダの構造（パケットタイプ 4bit + フラグ 4bit + 残りの長さ）を理解する
- Remaining Length の可変長エンコーディングアルゴリズムを実装できる
- UTF-8 文字列の MQTT エンコーディング（2バイト長プレフィックス + データ）を理解する
- 本プロジェクトの `codec.zig` のエンコード/デコード関数を読み解く
- Zig のビット操作、スライス操作を活用する

固定ヘッダ（Fixed Header）

すべての MQTT パケットは**固定ヘッダ**で始まる。最小構成はわずか2バイトである。

```
Byte 1:  [パケットタイプ: 4bit][フラグ: 4bit]
Byte 2+: [Remaining Length: 1~4バイトの可変長]
```

第1バイトの構造

```
Bit:  7  6  5  4  3  2  1  0
      |  Type   |   Flags   |
```

- **ビット 7-4**: パケットタイプ（1〜14）
- **ビット 3-0**: パケット固有のフラグ

Hex ダンプ例: PINGREQ パケット

PINGREQ（タイプ=12, フラグ=0, 本文なし）は最も単純なパケットである:

```
C0 00
|  +-- Remaining Length = 0 (本文なし)
+----- 0xC0 = 1100_0000 -> タイプ=12, フラグ=0
```

codec.zig の固定ヘッダ実装

本プロジェクトでは `src/mqtt/codec.zig` に固定ヘッダのデコード/エンコード関数がある。

FixedHeader 構造体

```
pub const FixedHeader = struct {
    packet_type: PacketType,
    flags: u4,
    remaining_length: u32,
    header_size: usize, // 固定ヘッダ全体のバイト数
};
```

デコード

```
pub fn decodeFixedHeader(data: []const u8) !FixedHeader {
    if (data.len < 2) return CodecError.PacketTooShort;
    const first_byte = data[0];
    const type_val: u4 = @intCast(first_byte >> 4);    // 上位4ビット
    const flags: u4 = @intCast(first_byte & 0x0F);     // 下位4ビット

    const packet_type: PacketType = @enumFromInt(type_val);

    const rl = try decodeRemainingLength(data[1..]);
    return .{
        .packet_type = packet_type,
        .flags = flags,
        .remaining_length = rl.value,
        .header_size = 1 + rl.bytes_consumed,
    };
}
```

ポイント: - `@intCast(first_byte >> 4)` で上位4ビットを `u4` に変換する - `@enumFromInt(type_val)` で `u4` を `PacketType` 列挙型に変換する - `header_size` を保持することで、可変ヘッダ+ペイロードの開始位置がわかる

エンコード

```
pub fn encodeFixedHeader(
    buf: []u8,
    packet_type: PacketType,
    flags: u4,
    remaining_length: u32,
```

```
) ![]const u8 {
    if (buf.len < 5) return CodecError.PacketTooShort;
    buf[0] = (@as(u8, @intFromEnum(packet_type)) << 4) | @as(u8, flags);
    const rl = try encodeRemainingLength(buf[1..5], remaining_length);
    return buf[0 .. 1 + rl.len];
}
```

`@intFromEnum` で列挙型を整数に変換し、4ビット左シフトしてフラグと合成する。

Remaining Length（残りの長さ）

固定ヘッダの第2バイト以降は、パケットの残りのバイト数を**可変長**で表す。 1バイトで 0〜127、2バイトで 128〜16383、最大4バイトで約256MBまで表現できる。

エンコーディングルール

各バイトの**下位7ビット**がデータ、**最上位ビット（ビット7）**が継続フラグである。 継続フラグが 1 の場合、次のバイトも Remaining Length の一部である。

```
バイト値: [継続ビット: 1bit][値: 7bit]

例: 残り長 = 321 (0x141)
 バイト1: 0xC1 = 1_1000001  -> 継続=1, 値=65
 バイト2: 0x02 = 0_0000010  -> 継続=0, 値=2
 復元: 65 + 2 * 128 = 65 + 256 = 321
```

範囲一覧

バイト数	最小値	最大値	Hex ダンプ例
1	0	127	`7F`
2	128	16,383	`80 01` 〜 `FF 7F`
3	16,384	2,097,151	`80 80 01`
4	2,097,152	268,435,455	`FF FF FF 7F`

codec.zig のエンコード実装

```
pub fn encodeRemainingLength(buf: []u8, value: u32) ![]const u8 {
    if (value > types.MAX_REMAINING_LENGTH) return CodecError.InvalidRemainingLength;
```

```
    var x = value;
    var i: usize = 0;
    while (true) {
        var encoded_byte: u8 = @intCast(x % 128);
        x /= 128;
        if (x > 0) encoded_byte |= 0x80; // 継続ビットを立てる
        if (i >= buf.len) return CodecError.InvalidRemainingLength;
        buf[i] = encoded_byte;
        i += 1;
        if (x == 0) break;
    }
    return buf[0..i];
}
```

codec.zig のデコード実装

```
pub const RemainingLengthResult = struct {
    value: u32,
    bytes_consumed: usize,
};

pub fn decodeRemainingLength(data: []const u8) !RemainingLengthResult {
    var multiplier: u32 = 1;
    var value: u32 = 0;
    var i: usize = 0;
    while (true) {
        if (i >= data.len) return CodecError.PacketTooShort;
        const encoded_byte = data[i];
        value += @as(u32, encoded_byte & 0x7F) * multiplier;
        if (multiplier > 128 * 128 * 128) return CodecError.InvalidRemainingLength;
        i += 1;
        if (encoded_byte & 0x80 == 0) break; // 継続ビットが0なら終了
        multiplier *= 128;
    }
    return .{ .value = value, .bytes_consumed = i };
}
```

戻り値の `bytes_consumed` により、Remaining Length の直後からペイロードを読める。

UTF-8 文字列エンコーディング

MQTT では文字列を**2バイトのビッグエンディアン長プレフィックス + UTF-8 データ**で表す。

```
[長さ上位バイト][長さ下位バイト][UTF-8 データ...]
```

Hex ダンプ例: 文字列 "MQTT"

```
00 04 4D 51 54 54
|  |  +------------ "MQTT" の ASCII/UTF-8 バイト列
+--+---------------- 長さ = 4 (ビッグエンディアン u16)
```

Hex ダンプ例: トピック "sensor/temp"

```
00 0B 73 65 6E 73 6F 72 2F 74 65 6D 70
|  |  +--------------------------------- "sensor/temp" (11バイト)
+--+------------------------------------- 長さ = 11
```

codec.zig の文字列エンコード

```zig
pub fn encodeString(buf: []u8, str: []const u8) ![]const u8 {
    const total = 2 + str.len;
    if (buf.len < total) return CodecError.PacketTooShort;
    buf[0] = @intCast((str.len >> 8) & 0xFF);  // 長さ上位バイト
    buf[1] = @intCast(str.len & 0xFF);          // 長さ下位バイト
    @memcpy(buf[2..][0..str.len], str);
    return buf[0..total];
}
```

codec.zig の文字列デコード

```zig
pub const StringResult = struct {
    value: []const u8,
    bytes_consumed: usize,
};

pub fn decodeString(data: []const u8) !StringResult {
    if (data.len < 2) return CodecError.PacketTooShort;
    const len: u16 = (@as(u16, data[0]) << 8) | @as(u16, data[1]);
    if (data.len < 2 + len) return CodecError.PacketTooShort;
    return .{
        .value = data[2..][0..len],
        .bytes_consumed = 2 + len,
    };
}
```

デコード結果は元のバッファへのスライスを返す。 メモリコピーが不要なため高効率であるが、元のバッファが解放されると無効になる点に注意。 永続化が必要な場合は `allocator.dupe(u8, result.value)` でコピーする。

u16 のエンコード/デコード

Packet ID や Keep Alive など、2バイトの整数もビッグエンディアンで表現する。

```
pub fn encodeU16(buf: []u8, value: u16) ![]const u8 {
    if (buf.len < 2) return CodecError.PacketTooShort;
    buf[0] = @intCast((value >> 8) & 0xFF);
    buf[1] = @intCast(value & 0xFF);
    return buf[0..2];
}

pub fn decodeU16(data: []const u8) !u16 {
    if (data.len < 2) return CodecError.PacketTooShort;
    return (@as(u16, data[0]) << 8) | @as(u16, data[1]);
}
```

完全なパケット例: CONNECT

CONNECT パケット全体の hex ダンプ（クライアントID = "zig"）：

```
10 0F                    -- 固定ヘッダ: type=1(CONNECT), RL=15
00 04 4D 51 54 54        -- プロトコル名 "MQTT"
04                       -- プロトコルレベル 4 (v3.1.1)
02                       -- 接続フラグ: Clean Session=1
00 3C                    -- Keep Alive = 60秒
00 03 7A 69 67           -- クライアントID "zig"
```

このバイト列の構造は Chapter 04 で詳しく解説する。

ManagedArrayList によるバイト列の動的構築

本プロジェクトの `codec.zig` では、パケットのエンコードに `std.array_list.AlignedManaged(T, null)` を使用している。 これは Zig 0.16 で提供されるアロケータ内蔵の動的配列である。

```
/// Zig 0.16: managed ArrayList (allocator を保持)
fn ManagedArrayList(comptime T: type) type {
    return std.array_list.AlignedManaged(T, null);
}
```

使用例（CONNECT パケットのエンコード抜粋）：

```zig
pub fn encodeConnect(allocator: Allocator, pkt: *const ConnectPacket) ![]u8 {
    var list = ManagedArrayList(u8).init(allocator);
    errdefer list.deinit();

    // プロトコル名 "MQTT" を書き込み
    try appendString(&list, pkt.protocol_name);
    // プロトコルレベル
    try list.append(pkt.protocol_level);
    // 接続フラグ (packed struct を u8 にキャスト)
    try list.append(@bitCast(pkt.flags));
    // Keep Alive
    try list.append(@intCast((pkt.keep_alive >> 8) & 0xFF));
    try list.append(@intCast(pkt.keep_alive & 0xFF));

    // ... ペイロード追加 ...

    return result.toOwnedSlice();
}
```

`ManagedArrayList(u8)` はバイト列を段階的に構築するのに適している。 `toOwnedSlice()` で所有権を呼び出し元に移転し、不要になったら `allocator.free(slice)` で解放する。

テスト

codec.zig のテストは Remaining Length と文字列のラウンドトリップを検証する:

```zig
test "remaining length: encode and decode round-trip" {
    var buf: [4]u8 = undefined;

    // 0
    const r0 = try encodeRemainingLength(&buf, 0);
    try std.testing.expectEqualSlices(u8, &.{0x00}, r0);
    const d0 = try decodeRemainingLength(r0);
    try std.testing.expectEqual(@as(u32, 0), d0.value);

    // 127 (1バイト最大値)
    const r127 = try encodeRemainingLength(&buf, 127);
    try std.testing.expectEqualSlices(u8, &.{0x7F}, r127);

    // 128 (2バイト最小値)
    const r128 = try encodeRemainingLength(&buf, 128);
    try std.testing.expectEqualSlices(u8, &.{ 0x80, 0x01 }, r128);

    // 268435455 (4バイト最大値)
    const r_max = try encodeRemainingLength(&buf, 268_435_455);
    const d_max = try decodeRemainingLength(r_max);
    try std.testing.expectEqual(@as(u32, 268_435_455), d_max.value);
}
```

```zig
test "string: encode and decode round-trip" {
    var buf: [256]u8 = undefined;
    const encoded = try encodeString(&buf, "hello");
    try std.testing.expectEqual(@as(usize, 7), encoded.len);

    const decoded = try decodeString(encoded);
    try std.testing.expectEqualStrings("hello", decoded.value);
    try std.testing.expectEqual(@as(usize, 7), decoded.bytes_consumed);
}
```

テスト実行は以下のコマンドで行う:

```
zig build test
```

エラー処理

codec.zig では専用のエラーセットを定義している:

```zig
pub const CodecError = error{
    InvalidRemainingLength,   // Remaining Length が不正
    InvalidPacketType,        // パケットタイプが不正
    InvalidProtocolName,      // プロトコル名が "MQTT" でない
    InvalidProtocolLevel,     // プロトコルレベルが 4 でない
    InvalidFlags,             // フラグ値が不正
    PacketTooShort,           // データが不足
    MalformedPacket,          // パケット構造が不正
    OutOfMemory,              // メモリ不足
    EndOfStream,              // ストリーム終端
};
```

Zig のエラーユニオン（`!`）により、全てのエンコード/デコード関数は `CodecError` を明示的に返す。呼び出し元は `try` で伝播するか、`catch` でハンドリングする。

まとめ

- MQTT の固定ヘッダは**最小2バイト**で構成され、パケットタイプとフラグをビット操作で抽出する
- Remaining Length は**可変長エンコーディング**（継続ビット方式）で最大4バイト
- 文字列は**2バイト長プレフィックス + UTF-8 データ**で表現する
- 本プロジェクトの `codec.zig` はバッファベースの API でエンコード/デコードを行い、`ManagedArrayList` で動的なバイト列構築を実現している
- `@intCast`、`@bitCast`、`@enumFromInt` などの Zig のビルトイン関数がプロトコル実装に適している

次のチャプターでは、Zig 0.16 で TCP 接続を確立し、これらのバイト列を実際に送受信する方法を学ぶ。

第3章 ZigでTCP通信

std.Io.netによるソケットプログラミング

学習目標

- TCP の基本概念（コネクション指向、ストリーム型）を復習する
- Zig 0.16 の `std.Io.net` API でサーバとクライアントを実装できる
- `std.Io.Threaded` による I/O ランタイムの初期化を理解する
- `IpAddress.parse` / `IpAddress.listen` / `listener.accept` / `stream.close` の使い方を理解する
- `stream.reader(io, &buf)` / `stream.writer(io, &buf)` による I/O パターンを活用する
- 本プロジェクトの `transport.zig` の設計意図を把握する

TCP の基本

TCP（Transmission Control Protocol）は**コネクション指向**のプロトコルである。 MQTT はこの TCP の上に構築されている。

TCP の特徴: - **信頼性のある配信**: パケットの順序保証・再送制御 - **ストリーム型**: バイト列の境界がない（メッセージ境界は上位層で管理） - **全二重通信**: 送受信を同時に行える

> MQTT がストリーム上で動作するため、固定ヘッダの Remaining Length を使って パケットの境界を判断する必要がある（Chapter 02 参照）。

Zig 0.16 の Io ランタイム

Zig 0.16 では、ネットワーク I/O は `std.Io` ランタイムを通じて行う。 全ての I/O 操作に `io` パラメータを渡す点が従来の API と大きく異なる。

Juicy Main パターン

本プロジェクトの Broker は以下のように `std.process.Init` からランタイムを受け取る:

```zig
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    const allocator = init.gpa;  // 汎用アロケータ
    const io = init.io;          // Io ランタイム
    const args = init.minimal.args; // コマンドライン引数

    // サーバ起動...
}
```

`init.gpa` は事前に初期化されたアロケータ、 `init.io` は I/O ランタイムである。 これにより、明示的な初期化コードが不要になる。

Io ランタイムの手動初期化

テストやライブラリでは、手動で I/O ランタイムを初期化することもできる:

```zig
var threaded = std.Io.Threaded.init(allocator, .{});
const io = threaded.io();
// io を使ってネットワーク操作を行う
```

---

std.Io.net API

Zig 0.16 のネットワーク API は `std.Io.net` 名前空間にある。

名前空間の取得

```zig
const std = @import("std");
const net = std.Io.net;
```

アドレスの解析

```zig
// IPv4 アドレスとポート番号を指定
const address = net.IpAddress.parse("127.0.0.1", 1883) catch |err| {
    std.debug.print("アドレス解析エラー: {}\n", .{err});
    return err;
};
```

ポート番号 1883 は MQTT の標準ポートである。 `IpAddress.parse` は `"0.0.0.0"` で全インターフェースにバインドすることもできる。

サーバ側: listen / accept

サーバの基本構造

```
const std = @import("std");
const net = std.Io.net;

pub fn main(init: std.process.Init) !void {
    const io = init.io;

    const address = net.IpAddress.parse("0.0.0.0", 1883) catch
        return error.AddressParseFailed;

    // リスナーを作成（io パラメータが必須）
    var listener = try net.IpAddress.listen(&address, io, .{
        .reuse_address = true,
    });
    defer listener.deinit(io);

    std.log.info("MQTT Broker listening on :1883", .{});

    // クライアントの接続を受け入れるループ
    while (true) {
        // accept にも io パラメータが必須
        const stream = try listener.accept(io);

        std.log.info("クライアント接続を受け入れました", .{});

        // クライアントを処理
        handleClient(stream, io) catch |err| {
            std.log.err("クライアント処理エラー: {s}", .{@errorName(err)});
        };

        // ストリームを閉じる（io パラメータが必須）
        stream.close(io);
    }
}
```

listen のオプション

`listen` に渡す構造体で以下を設定できる:

```
var listener = try net.IpAddress.listen(&address, io, .{
    .reuse_address = true,    // TIME_WAIT 中でも再バインド可能
});
```

accept の戻り値

Zig 0.16 では `listener.accept(io)` は `net.Stream` を直接返す。 この stream で読み書きを行う。

クライアント側: connect

```
fn connectToBroker(io: std.Io) !net.Stream {
    const address = net.IpAddress.parseIp4("127.0.0.1", 1883) catch
        return error.AddressParseFailed;
    const stream = try net.IpAddress.connect(&address, io, .{
        .mode = .stream,
        .protocol = .tcp,
    });
    return stream;
}
```

`IpAddress.connect` は TCP 接続を確立し、`net.Stream` を返す。 `io` パラメータにより、非同期 I/O ランタイムと統合される。

ストリームの読み書き

Zig 0.16 では、stream の reader/writer はバッファと io を引数に取る。

Reader の使い方

```
fn readFromStream(stream: net.Stream, io: std.Io) !void {
    var read_buffer: [8192]u8 = undefined;
    var reader = stream.reader(io, &read_buffer);

    // 1バイトを読む（readByte は存在しない）
    var byte_buf: [1]u8 = undefined;
    reader.interface.readSliceAll(&byte_buf) catch |err| switch (err) {
        error.EndOfStream => {
            std.log.info("接続が閉じられました", .{});
            return;
        },
        else => return err,
    };
    const first_byte = byte_buf[0];
    _ = first_byte;

    // 複数バイトを読む
    var data: [256]u8 = undefined;
```

```
    try reader.interface.readSliceAll(data[0..10]); // 10バイト読む
}
```

重要なポイント: - `stream.reader(io, &read_buffer)` でリーダーを作成する。第2引数はバッファ - `.interface` は**フィールド**であり、関数呼び出しではない - `readByte()` は存在しない。代わりに `readSliceAll(buf[0..1])` を使う - `readSliceAll` は指定したスライスが満たされるまでブロックする

Writer の使い方

```
fn writeToStream(stream: net.Stream, io: std.Io) !void {
    var write_buffer: [8192]u8 = undefined;
    var writer = stream.writer(io, &write_buffer);

    // データを書き込む
    try writer.interface.writeAll(&[_]u8{ 0xD0, 0x00 }); // PINGRESP

    // フラッシュして実際に送信する（重要!）
    try writer.interface.flush();
}
```

重要なポイント: - `stream.writer(io, &write_buffer)` でライターを作成する - `.interface` は**フィールド**であり、関数呼び出しではない - `writeAll` 後に必ず `flush()` を呼ぶこと。呼ばないとデータが送信されない

本プロジェクトの transport.zig

本プロジェクトでは `src/client/transport.zig` に TCP 通信の抽象化層を設けている。

Transport 構造体

```
pub const Transport = struct {
    stream: net.Stream,
    io: std.Io,
    allocator: Allocator,

    /// TCP 接続を確立
    pub fn connect(allocator: Allocator, io: std.Io, host: []const u8, port: u16) !Tra
nsport {
        const address = try net.IpAddress.parseIp4(host, port);
        const stream = try net.IpAddress.connect(&address, io, .{
            .mode = .stream,
            .protocol = .tcp,
        });
```

```
        return .{
            .stream = stream,
            .io = io,
            .allocator = allocator,
        };
    }

    /// 接続を閉じる
    pub fn close(self: *Transport) void {
        self.stream.close(self.io);
    }
};
```

バイト列の送信

```
pub fn send(self: *Transport, data: []const u8) !void {
    var writer_buffer: [8192]u8 = undefined;
    var writer = self.stream.writer(self.io, &writer_buffer);
    try writer.interface.writeAll(data);
    try writer.interface.flush();
}
```

パケットの受信

```
pub fn readPacket(self: *Transport) !ReadResult {
    var reader_buffer: [8192]u8 = undefined;
    var reader = self.stream.reader(self.io, &reader_buffer);

    // 固定ヘッダの最初のバイトを読む
    var header_buf: [5]u8 = undefined;
    reader.interface.readSliceAll(header_buf[0..1]) catch |err| switch (err) {
        error.EndOfStream => return error.ConnectionClosed,
        else => return err,
    };

    // Remaining Length を読む (最大4バイト)
    var rl_len: usize = 0;
    while (rl_len < 4) {
        try reader.interface.readSliceAll(header_buf[1 + rl_len ..][0..1]);
        rl_len += 1;
        if (header_buf[rl_len] & 0x80 == 0) break;
    }

    const header = try codec.decodeFixedHeader(header_buf[0 .. 1 + rl_len]);

    // Remaining bytes を読む
    const data = try self.allocator.alloc(u8, header.remaining_length);
    errdefer self.allocator.free(data);
    try reader.interface.readSliceAll(data);
```

```
    return .{ .header = header, .data = data };
}
```

Transport は以下の役割を果たす: - `io` と `stream` をまとめて保持し、引き回しを簡潔にする - 固定ヘッダ + Remaining Length の読み取りロジックを共通化する - `readSliceAll` を使い、TCP ストリームの断片化を意識せずに完全なパケットを受信する

Broker のサーバ実装

`src/broker/server.zig` は `std.Io.net` を使った実際のサーバ実装である。

```
pub fn run(self: *Server) !void {
    const address = net.IpAddress.parse("0.0.0.0", self.port) catch
        return error.AddressParseFailed;

    var listener = try net.IpAddress.listen(&address, self.io, .{
        .reuse_address = true,
    });
    defer listener.deinit(self.io);

    std.log.info("MQTT Broker listening on port {d}", .{self.port});

    while (!self.stop_requested.load(.acquire)) {
        const stream = listener.accept(self.io) catch |err| {
            if (self.stop_requested.load(.acquire)) break;
            std.log.err("accept error: {s}", .{@errorName(err)});
            continue;
        };

        // 接続ハンドラを別スレッドで実行
        // ...
    }
}
```

注目点: - `net.IpAddress.parse` でアドレスを作成し、`listen` に `&address` と `io` を渡す - `listener.accept(io)` で接続を待ち受け、`stream` を取得する - Graceful Shutdown のために `std.atomic.Value(bool)` を使用している

エラーハンドリング

ネットワーク I/O では様々なエラーが発生しうる:

```zig
fn safeRead(stream: net.Stream, io: std.Io) void {
    var buf: [8192]u8 = undefined;
    var reader = stream.reader(io, &buf);

    var byte_buf: [1]u8 = undefined;
    reader.interface.readSliceAll(&byte_buf) catch |err| {
        switch (err) {
            error.ConnectionResetByPeer => {
                std.log.info("接続がリセットされました", .{});
            },
            error.EndOfStream => {
                std.log.info("接続が閉じられました", .{});
            },
            else => {
                std.log.err("読み込みエラー: {s}", .{@errorName(err)});
            },
        }
        return;
    };
}
```

Zig 0.16 の同期プリミティブ

マルチスレッドでの接続管理には、Zig 0.16 の同期プリミティブを使う:

Mutex

```zig
var mutex: std.Io.Mutex = .init;

// ロック（キャンセル不可）
mutex.lockUncancelable(io);
defer mutex.unlock(io);

// クリティカルセクション
```

RwLock

```zig
var rwlock: std.Io.RwLock = .init;

// 読み取りロック
rwlock.lockSharedUncancelable(io);
defer rwlock.unlockShared(io);

// 書き込みロック
```

```
rwlock.lockUncancelable(io);
defer rwlock.unlock(io);
```

Event

```
var event: std.Io.Event = .unset;

// シグナルを送る
event.set(io);

// シグナルを待つ
event.waitUncancelable(io);
```

Broker の Graceful Shutdown ではこの Event が使われている（`server.zig` 参照）。

API 対応表: 旧 API vs Zig 0.16

操作	旧 API (std.net)	Zig 0.16 (std.Io.net)
アドレス解析	`net.Address.parseIp4(host, port)`	`net.IpAddress.parse(host, port)`
リッスン	`address.listen(.{...})`	`net.IpAddress.listen(&address, io, .{...})`
接続受付	`server.accept()`	`listener.accept(io)`
接続確立	`net.tcpConnectToAddress(address)`	`net.IpAddress.connect(&address, io, .{...})`
ストリーム閉じる	`stream.close()`	`stream.close(io)`
リーダー取得	`stream.reader()`	`stream.reader(io, &buffer)`
	`stream.writer()`	`stream.writer(io, &buffer)`

操作	旧 API (std.net)	Zig 0.16 (std.Io.net)
ライター取得		
1バイト読み	`reader.readByte()`	`reader.interface.readSliceAll(buf[0..1])`
データ読み	`reader.readAll(slice)`	`reader.interface.readSliceAll(slice)`
データ書き	`writer.writeAll(data)`	`writer.interface.writeAll(data)`
フラッシュ	`bw.flush()`	`writer.interface.flush()`

まとめ

- Zig 0.16 では `std.Io.net` 名前空間でネットワーク操作を行う
- 全ての I/O 操作に `io` パラメータ（`std.Io` インスタンス）を渡す
- `stream.reader(io, &buffer)` / `stream.writer(io, &buffer)` でリーダー/ライターを作成し、`.interface` フィールド経由で読み書きする
- `readByte()` は存在しない。`readSliceAll(buf[0..1])` を使う
- `writer.interface.flush()` を忘れるとデータが送信されないので注意
- `transport.zig` は `io`、`stream`、`allocator` をまとめて保持し、MQTT パケットの送受信を簡潔にしている

次のチャプターでは、実際の CONNECT / CONNACK パケットの構造とエンコード・デコードを学ぶ。

第II部 MQTTパケット

各パケットの構造と実装を学びます

第4章 CONNECT / CONNACK

接続パケットの構造と実装

学習目標

- CONNECT パケットの完全な構造（可変ヘッダ + ペイロード）を理解する
- 接続フラグの各ビットの意味を把握する
- CONNACK パケットの構造と戻りコードを理解する
- Zig の `packed struct` で接続フラグを型安全に扱う方法を学ぶ
- 本プロジェクトの `codec.zig` でのエンコード/デコード実装を読み解く
- `union(PacketType)` による全パケット型の統一的な管理を理解する

CONNECT パケットの全体構造

CONNECT は、クライアントが Broker に接続する際に送る最初のパケットである。

```
[固定ヘッダ]
  Byte 1: 0x10 (タイプ=1, フラグ=0)
  Byte 2+: Remaining Length

[可変ヘッダ]
 プロトコル名:    00 04 4D 51 54 54  ("MQTT")
 プロトコルレベル: 04                 (v3.1.1)
 接続フラグ:      1バイト
  Keep Alive:     2バイト (ビッグエンディアン u16)

[ペイロード] ※ 接続フラグに応じて可変
 クライアントID:  必須
  Will Topic:     接続フラグで Will=1 の場合
  Will Message:   接続フラグで Will=1 の場合
  Username:       接続フラグで Username=1 の場合
  Password:       接続フラグで Password=1 の場合
```

接続フラグ（Connect Flags）

接続フラグは1バイトで、各ビットに意味がある:

```
Bit:  7         6        5       4  3     2          1          0
      Username  Password Will    Will QoS Will       Clean      Reserved
      Flag      Flag     Retain            Flag       Session    (=0)
```

ビット	フラグ名	説明
7	Username Flag	ペイロードにユーザー名が含まれるか
6	Password Flag	ペイロードにパスワードが含まれるか
5	Will Retain	Will メッセージの Retain フラグ
4-3	Will QoS	Will メッセージの QoS レベル（0/1/2）
2	Will Flag	遺言メッセージが設定されているか
1	Clean Session	新規セッションとして接続するか
0	Reserved	常に 0

packed struct による接続フラグの表現

本プロジェクトの `types.zig` では、接続フラグを `packed struct(u8)` で定義している:

```zig
pub const ConnectFlags = packed struct(u8) {
    reserved: u1 = 0,           // bit 0: 予約（常に0）
    clean_session: bool = true, // bit 1: クリーンセッション
    will_flag: bool = false,    // bit 2: Will フラグ
    will_qos: u2 = 0,           // bits 3-4: Will QoS
    will_retain: bool = false,  // bit 5: Will Retain
    password_flag: bool = false,// bit 6: パスワードフラグ
    username_flag: bool = false,// bit 7: ユーザー名フラグ
};
```

packed struct の特徴

- `packed struct(u8)` はメモリ上で正確に1バイトを占める
- フィールドはビット0から順に配置される
- `@bitCast` で `u8` との相互変換が可能
- 手動のビットマスク操作が不要になり、可読性とバグ耐性が向上する

使用例

```zig
// フラグの作成
const flags = ConnectFlags{
    .clean_session = true,
    .username_flag = true,
    .password_flag = true,
};

// u8 に変換
const byte: u8 = @bitCast(flags); // 0xC2

// u8 からフラグに変換
const parsed: ConnectFlags = @bitCast(@as(u8, 0xC2));
// parsed.clean_session == true
// parsed.username_flag == true
// parsed.password_flag == true
```

テストで検証:

```zig
test "ConnectFlags packed layout" {
    const flags = ConnectFlags{
        .clean_session = true,
        .username_flag = true,
        .password_flag = true,
    };
    const byte: u8 = @bitCast(flags);
    try std.testing.expectEqual(0xC2, byte);
}
```

フラグバイト `0xC2` の解析:

```
1100_0010
|||| |||+-- Reserved = 0
|||| ||+--- Clean Session = 1
|||| |+---- Will Flag = 0
||||++------ Will QoS = 0
|||+-------- Will Retain = 0
||+--------- Password Flag = 1
|+---------- Username Flag = 1
```

Hex ダンプ例: CONNECT パケット

クライアントID = "zig-client"、Clean Session = true、Keep Alive = 60秒:

```
10 1A                           -- 固定ヘッダ: CONNECT, RL=26
00 04 4D 51 54 54               -- プロトコル名 "MQTT"
04                              -- プロトコルレベル 4
02                              -- 接続フラグ: Clean Session のみ
00 3C                           -- Keep Alive = 60
00 0A 7A 69 67 2D 63 6C 69     -- クライアントID "zig-client"
65 6E 74
```

ConnectPacket 構造体

`packet.zig` で CONNECT パケットのデータ構造を定義している:

```zig
pub const ConnectPacket = struct {
    protocol_name: []const u8 = types.PROTOCOL_NAME,
    protocol_level: u8 = types.PROTOCOL_LEVEL,
    flags: ConnectFlags = .{},
    keep_alive: u16 = 60,
    client_id: []const u8,
    will_topic: ?[]const u8 = null,
    will_message: ?[]const u8 = null,
    username: ?[]const u8 = null,
    password: ?[]const u8 = null,
};
```

デフォルト値が設定されているため、最小限の初期化で使える:

```zig
const pkt = ConnectPacket{
    .client_id = "my-device",
};
// protocol_name = "MQTT", protocol_level = 4, keep_alive = 60,
// flags.clean_session = true が自動的に設定される
```

CONNECT のエンコード

`codec.zig` の `encodeConnect` は `ManagedArrayList(u8)` を使って動的にバイト列を構築する:

```zig
pub fn encodeConnect(allocator: Allocator, pkt: *const ConnectPacket) ![]u8 {
    var list = ManagedArrayList(u8).init(allocator);
    errdefer list.deinit();

    // 可変ヘッダ: プロトコル名
    try appendString(&list, pkt.protocol_name);
    // プロトコルレベル
    try list.append(pkt.protocol_level);
```

```
    // 接続フラグ (packed struct -> u8)
    try list.append(@bitCast(pkt.flags));
    // Keep Alive
    try list.append(@intCast((pkt.keep_alive >> 8) & 0xFF));
    try list.append(@intCast(pkt.keep_alive & 0xFF));

    // ペイロード: クライアントID (必須)
    try appendString(&list, pkt.client_id);

    // Will Topic / Message (フラグに応じて)
    if (pkt.flags.will_flag) {
        if (pkt.will_topic) |t| try appendString(&list, t);
        if (pkt.will_message) |m| try appendString(&list, m);
    }

    // Username / Password (フラグに応じて)
    if (pkt.flags.username_flag) {
        if (pkt.username) |u| try appendString(&list, u);
    }
    if (pkt.flags.password_flag) {
        if (pkt.password) |p| try appendString(&list, p);
    }

    // 固定ヘッダを先頭に付加
    var result = ManagedArrayList(u8).init(allocator);
    errdefer result.deinit();
    var hdr_buf: [5]u8 = undefined;
    const hdr = try encodeFixedHeader(&hdr_buf, .connect, 0, @intCast(list.items.len))
;
    try result.appendSlice(hdr);
    try result.appendSlice(list.items);
    list.deinit();

    return result.toOwnedSlice();
}
```

ポイント: - `@bitCast(pkt.flags)` で `ConnectFlags` (packed struct) を `u8` に変換 - ペイロードのフィールドは**仕様で定められた順序**で書き込む必要がある - `errdefer list.deinit()` でエラー時のメモリリークを防止

CONNECT のデコード

```
pub fn decodeConnect(allocator: Allocator, data: []const u8) !ConnectPacket {
    var pos: usize = 0;

    // プロトコル名
    const proto = try decodeString(data[pos..]);
    pos += proto.bytes_consumed;
```

```zig
    // プロトコルレベル
    if (pos >= data.len) return CodecError.PacketTooShort;
    const level = data[pos];
    pos += 1;

    // 接続フラグ (u8 -> packed struct)
    if (pos >= data.len) return CodecError.PacketTooShort;
    const flags: ConnectFlags = @bitCast(data[pos]);
    pos += 1;

    // Keep Alive
    const keep_alive = try decodeU16(data[pos..]);
    pos += 2;

    // ペイロード: クライアントID
    const cid = try decodeString(data[pos..]);
    pos += cid.bytes_consumed;
    const client_id = try allocator.dupe(u8, cid.value);
    errdefer allocator.free(client_id);

    // Will Topic / Message (フラグに応じて)
    var will_topic: ?[]const u8 = null;
    var will_message: ?[]const u8 = null;
    if (flags.will_flag) {
        const wt = try decodeString(data[pos..]);
        pos += wt.bytes_consumed;
        will_topic = try allocator.dupe(u8, wt.value);

        const wm = try decodeString(data[pos..]);
        pos += wm.bytes_consumed;
        will_message = try allocator.dupe(u8, wm.value);
    }

    // Username / Password (フラグに応じて)
    var username: ?[]const u8 = null;
    if (flags.username_flag) {
        const un = try decodeString(data[pos..]);
        pos += un.bytes_consumed;
        username = try allocator.dupe(u8, un.value);
    }

    var password: ?[]const u8 = null;
    if (flags.password_flag) {
        const pw = try decodeString(data[pos..]);
        pos += pw.bytes_consumed;
        password = try allocator.dupe(u8, pw.value);
    }

    return .{
        .protocol_name = proto.value,
        .protocol_level = level,
        .flags = flags,
        .keep_alive = keep_alive,
        .client_id = client_id,
```

```
        .will_topic = will_topic,
        .will_message = will_message,
        .username = username,
        .password = password,
    };
}
```

ポイント: - `@bitCast(data[pos])` で `u8` を `ConnectFlags` (packed struct) に変換 - `allocator.dupe(u8, slice)` で文字列データをコピーし、所有権を呼び出し元に渡す - `errdefer allocator.free(...)` でデコード途中のエラー時にメモリを確実に解放

CONNACK パケット

Broker が CONNECT に応答して送る。常に固定長4バイトである。

```
[固定ヘッダ]
  Byte 1: 0x20 (タイプ=2, フラグ=0)
  Byte 2: 0x02 (Remaining Length = 2)

[可変ヘッダ]
  Byte 3: Session Present フラグ (0 or 1)
  Byte 4: 戻りコード
```

戻りコード

```zig
pub const ConnectReturnCode = enum(u8) {
    accepted = 0,              // 接続承認
    unacceptable_protocol = 1, // プロトコルバージョン不適合
    identifier_rejected = 2,   // クライアントID が不正
    server_unavailable = 3,    // サーバ利用不可
    bad_credentials = 4,       // ユーザー名/パスワード不正
    not_authorized = 5,        // 認証なし
    _,                         // 未知の値も受け入れる
};
```

`_` を定義することで、未知の戻りコード値を受信しても安全にデコードできる。

Hex ダンプ例

```
成功:     20 02 00 00   -- Session Present=0, Return Code=0
拒否:     20 02 00 04   -- Session Present=0, Return Code=4 (認証エラー)
```

CONNACK のエンコード/デコード

```
pub fn encodeConnack(buf: []u8, pkt: *const ConnackPacket) ![]const u8 {
    if (buf.len < 4) return CodecError.PacketTooShort;
    buf[0] = 0x20; // CONNACK type=2, flags=0
    buf[1] = 0x02; // remaining length = 2
    buf[2] = if (pkt.session_present) @as(u8, 0x01) else @as(u8, 0x00);
    buf[3] = @intFromEnum(pkt.return_code);
    return buf[0..4];
}

pub fn decodeConnack(data: []const u8) !ConnackPacket {
    if (data.len < 2) return CodecError.PacketTooShort;
    return .{
        .session_present = (data[0] & 0x01) != 0,
        .return_code = @enumFromInt(data[1]),
    };
}
```

CONNACK は固定長であるため、バッファベースのシンプルな実装になっている。 アロケータは不要である。

Packet tagged union によるパケット管理

`packet.zig` では、全パケット種別を `union(PacketType)` で統一的に管理している:

```
pub const Packet = union(PacketType) {
    reserved: void,
    connect: ConnectPacket,
    connack: ConnackPacket,
    publish: PublishPacket,
    puback: PubackPacket,
    pubrec: PubackPacket,
    pubrel: PubackPacket,
    pubcomp: PubackPacket,
    subscribe: SubscribePacket,
    suback: SubackPacket,
    unsubscribe: UnsubscribePacket,
    unsuback: UnsubackPacket,
    pingreq: void,
    pingresp: void,
    disconnect: void,
    reserved2: void,
};
```

パターンマッチで処理を分岐

```
fn handlePacket(pkt: Packet) void {
    switch (pkt) {
        .connect => |data| {
            std.log.info("CONNECT from: {s}", .{data.client_id});
        },
        .pingreq => {
            std.log.info("PINGREQ received", .{});
        },
        else => {},
    }
}
```

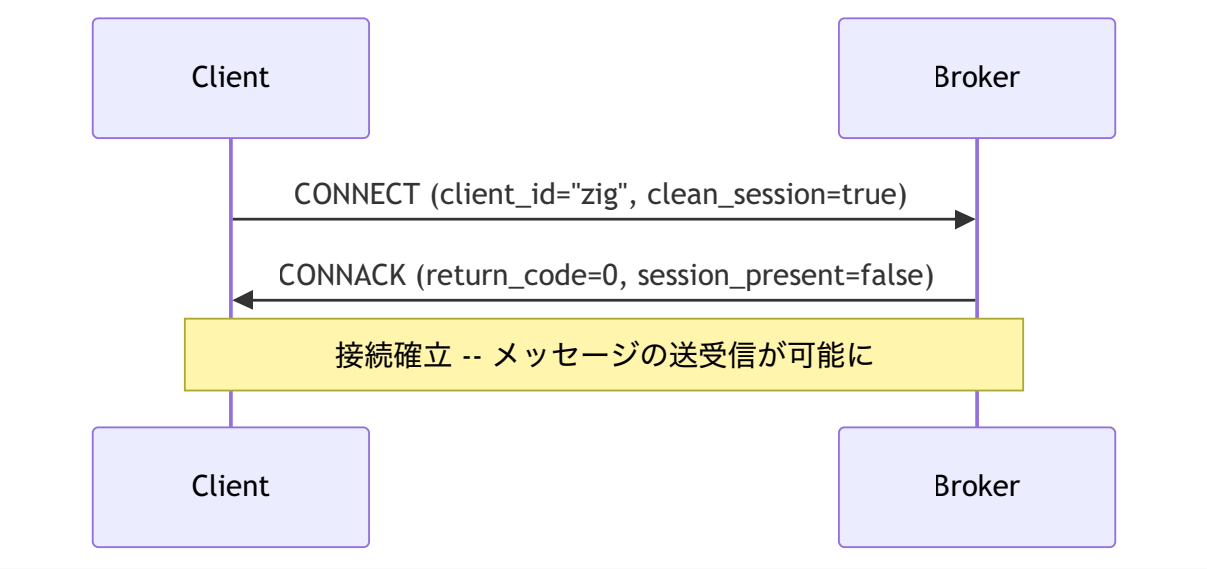
タグ付き共用体のメリット: - 全パケット種別を**コンパイル時に網羅的にチェック**できる - 各パケットに固有のデータ構造を安全に関連付けられる - `switch` で `else` を省略すると、未処理のケースがコンパイルエラーになる

パケットのメモリ解放

`freePacket` 関数がパケット種別に応じたメモリ解放を行う:

```
pub fn freePacket(allocator: Allocator, packet_val: *const Packet) void {
    switch (packet_val.*) {
        .connect => |p| {
            allocator.free(p.client_id);
            if (p.will_topic) |t| allocator.free(t);
            if (p.will_message) |m| allocator.free(m);
            if (p.username) |u| allocator.free(u);
            if (p.password) |pw| allocator.free(pw);
        },
        .publish => |p| {
            allocator.free(p.topic);
            allocator.free(p.payload);
        },
        else => {},
    }
}
```

接続フローの図解



テスト

```
test "CONNECT: encode and decode round-trip" {
    const allocator = std.testing.allocator;
    const pkt = ConnectPacket{
        .client_id = "test-client",
        .flags = .{
            .clean_session = true,
            .username_flag = true,
            .password_flag = true,
        },
        .keep_alive = 60,
        .username = "user",
        .password = "pass",
    };
    const encoded = try encodeConnect(allocator, &pkt);
    defer allocator.free(encoded);

    const header = try decodeFixedHeader(encoded);
    const data = encoded[header.header_size..];
    const decoded = try decodeConnect(allocator, data);
    defer {
        allocator.free(decoded.client_id);
        if (decoded.username) |u| allocator.free(u);
        if (decoded.password) |p| allocator.free(p);
    }
```

```zig
    try std.testing.expectEqualStrings("test-client", decoded.client_id);
    try std.testing.expectEqual(true, decoded.flags.clean_session);
    try std.testing.expectEqual(@as(u16, 60), decoded.keep_alive);
    try std.testing.expectEqualStrings("user", decoded.username.?);
    try std.testing.expectEqualStrings("pass", decoded.password.?);
}

test "CONNACK: encode and decode" {
    var buf: [4]u8 = undefined;
    const pkt = ConnackPacket{
        .session_present = true,
        .return_code = .accepted,
    };
    const encoded = try encodeConnack(&buf, &pkt);
    try std.testing.expectEqual(@as(usize, 4), encoded.len);

    const decoded = try decodeConnack(encoded[2..]);
    try std.testing.expectEqual(true, decoded.session_present);
    try std.testing.expectEqual(ConnectReturnCode.accepted, decoded.return_code);
}
```

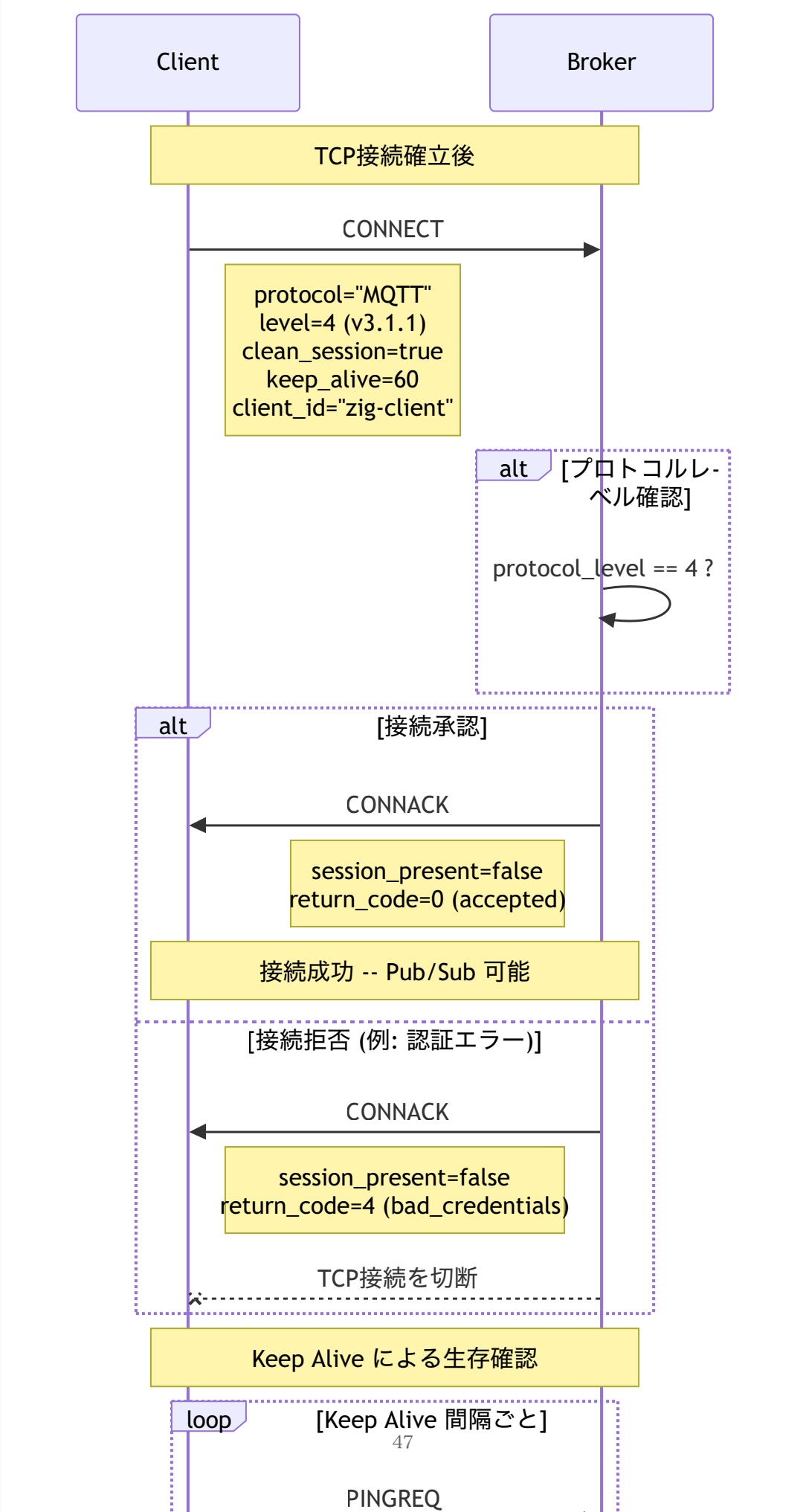
`std.testing.allocator` を使うことで、テスト終了時にメモリリークが自動検出される。

まとめ

- CONNECT は**プロトコル名・レベル・接続フラグ・Keep Alive + ペイロード**で構成される
- 接続フラグの8ビットは `packed struct(u8)` で安全に扱え、`@bitCast` で `u8` と相互変換できる
- CONNACK は固定長4バイトで、戻りコードにより接続の成否を通知する
- `codec.zig` では `ManagedArrayList(u8)` でバイト列を動的に構築し、`errdefer` でメモリリークを防止している
- `packet.zig` の `union(PacketType)` で全パケット種別を型安全に管理する

次のチャプターでは、PUBLISH パケットと QoS フローを学ぶ。

図


Client
Broker
TCP接続確立後
CONNECT
protocol="MQTT"
level=4 (v3.1.1)
clean_session=true
keep_alive=60
client_id="zig-client"
alt
[プロトコルレ-
ベル確認]
protocol_level == 4 ?
alt
[接続承認]
CONNACK
session_present=false
return_code=0 (accepted)
接続成功 -- Pub/Sub 可能
[接続拒否 (例: 認証エラー)]
CONNACK
session_present=false
return_code=4 (bad_credentials)
TCP接続を切断
Keep Alive による生存確認
loop
[Keep Alive 間隔ごと]
PINGREQ

第5章 PUBLISHフロー

メッセージ送受信とQoS

学習目標

- PUBLISH パケットの完全な構造（トピック、QoS、Retain、DUP、Packet ID、ペイロード）を理解する
- QoS 0（fire-and-forget）と QoS 1（PUBACK ハンドシェイク）のフローの違いを把握する
- 本プロジェクトの `codec.zig` での PUBLISH エンコード/デコードを読み解く
- Zig のオプショナル型（`?u16`）とエラーハンドリングを実践する
- `ManagedArrayList` を使った動的なパケット構築パターンを理解する

PUBLISH パケットの構造

PUBLISH はメッセージを配信するための中核パケットである。

```
[固定ヘッダ]
  Byte 1:  [タイプ=3: 4bit][DUP: 1bit][QoS: 2bit][Retain: 1bit]
  Byte 2+: Remaining Length

[可変ヘッダ]
 トピック名:     MQTT文字列（2バイト長 + UTF-8データ）
  Packet ID:     2バイト（QoS >= 1 の場合のみ）

[ペイロード]
 メッセージ本文:  任意のバイト列（残りの長さから算出）
```

固定ヘッダのフラグビット

PUBLISH パケットの第1バイトは他のパケットと異なり、フラグに意味がある:

```
Bit:  7  6  5  4  3     2  1     0
      0  0  1  1  DUP   QoS     Retain
      (タイプ=3)
```

フラグ	ビット	説明
DUP	3	再送パケットかどうか（QoS > 0 で使用）

フラグ	ビット	説明
QoS	2-1	配信品質レベル（0, 1, 2）
Retain	0	Broker がこのメッセージを保持するか

QoS 列挙型

本プロジェクトでは `types.zig` で QoS を列挙型として定義している:

```
pub const QoS = enum(u2) {
    at_most_once = 0,  // QoS 0: 最大1回配信（fire and forget）
    at_least_once = 1, // QoS 1: 最低1回配信（PUBACK で確認）
    exactly_once = 2,  // QoS 2: 正確に1回配信（4ステップ）

    pub fn fromInt(val: u2) QoS {
        return @enumFromInt(val);
    }
};
```

PublishPacket 構造体

`packet.zig` で PUBLISH パケットのデータ構造を定義している:

```
pub const PublishPacket = struct {
    dup: bool = false,
    qos: QoS = .at_most_once,
    retain: bool = false,
    topic: []const u8,
    packet_id: ?u16 = null,  // QoS 0 の場合は null
    payload: []const u8,
};
```

オプショナル型 `?u16` のポイント

`packet_id` が `?u16` である理由: - QoS 0 では Packet ID フィールドが**パケットに存在しない** - `null` で「フィールドが存在しない」ことを型レベルで表現できる - `if (packet_id) |id|` パターンで安全にアンラップする

Hex ダンプ例

QoS 0: トピック "sensor/temp"、ペイロード "25.3"

```
30 11                           -- PUBLISH, DUP=0, QoS=0, Retain=0, RL=17
00 0B 73 65 6E 73 6F 72 2F     -- トピック名 "sensor/temp" (11バイト)
74 65 6D 70
32 35 2E 33                     -- ペイロード "25.3"
```

Packet ID は**存在しない**（QoS 0 のため）。

QoS 1: トピック "sensor/temp"、Packet ID = 1、ペイロード "25.3"

```
32 13                           -- PUBLISH, DUP=0, QoS=1, Retain=0, RL=19
00 0B 73 65 6E 73 6F 72 2F     -- トピック名 "sensor/temp"
74 65 6D 70
00 01                           -- Packet ID = 1
32 35 2E 33                     -- ペイロード "25.3"
```

第1バイト `0x32` の解析:

```
0011_0010
||||_|||+-- Retain = 0
||||_|+---- QoS の下位ビット = 1
||||_+----- QoS の上位ビット = 0   -> QoS = 01 = 1
|||+------- DUP = 0
++++------- タイプ = 3 (PUBLISH)
```

QoS 0: Fire-and-Forget

最もシンプルな配信方法。送信者は確認応答を待たない。

```
Publisher --PUBLISH(QoS=0)--> Broker --PUBLISH(QoS=0)--> Subscriber
```

- メッセージロスの可能性がある
- 最もオーバーヘッドが少ない
- センサーデータの定期送信など、多少のロスが許容されるケースに適する
- Packet ID が不要なため、パケットサイズが2バイト小さい

QoS 1: At Least Once（PUBACK ハンドシェイク）

送信者は PUBACK を受信するまでメッセージを保持し、タイムアウト時に再送する。

```
Publisher --PUBLISH(QoS=1, id=1)--> Broker
Broker   --PUBACK(id=1)----------> Publisher
Broker   --PUBLISH(QoS=1, id=5)--> Subscriber
Subscriber--PUBACK(id=5)----------> Broker
```

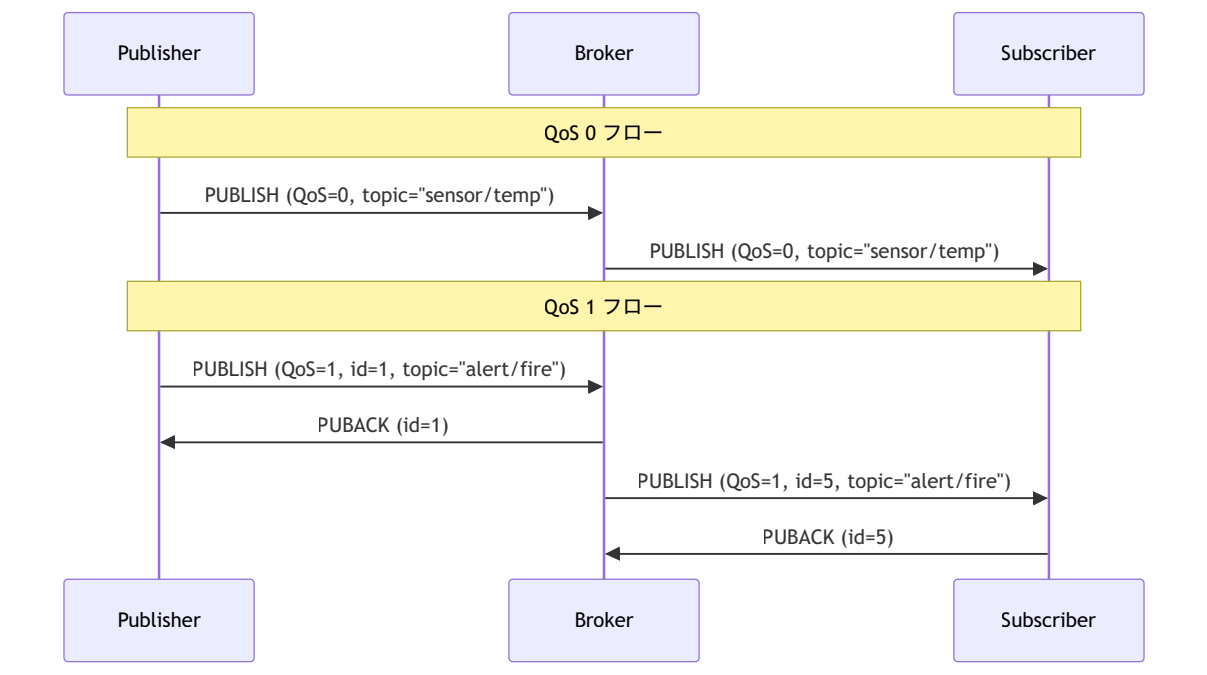
- メッセージは**少なくとも1回**は配信される
- 重複配信の可能性がある（DUP フラグで識別）
- アラート通知など、確実な配信が必要なケースに適する
- Broker はメッセージを受信した時点で即座に購読者へ転送する

PUBACK パケット

PUBACK は固定長4バイトのシンプルなパケットである:

```
40 02 00 01
|  |  +--+-- Packet ID = 1
|  +-------- Remaining Length = 2
+----------- タイプ=4 (PUBACK), フラグ=0
```

QoS フローの図解



PUBLISH のエンコード

`codec.zig` の `encodePublish` は `ManagedArrayList` を使って動的にバイト列を構築する:

```
pub fn encodePublish(allocator: Allocator, pkt: *const PublishPacket) ![]u8 {
    var list = ManagedArrayList(u8).init(allocator);
    errdefer list.deinit();

    // 可変ヘッダ: トピック名
    try appendString(&list, pkt.topic);

    // Packet ID (QoS > 0 の場合のみ)
    if (pkt.qos != .at_most_once) {
        const pid = pkt.packet_id orelse return CodecError.MalformedPacket;
        try list.append(@intCast((pid >> 8) & 0xFF));
        try list.append(@intCast(pid & 0xFF));
    }

    // ペイロード
    try list.appendSlice(pkt.payload);

    // 固定ヘッダのフラグを構築
    var flags: u4 = 0;
    if (pkt.dup) flags |= 0x08;
```

```
    flags |= @as(u4, @intFromEnum(pkt.qos)) << 1;
    if (pkt.retain) flags |= 0x01;

    // 固定ヘッダ + ペイロードを結合
    var result = ManagedArrayList(u8).init(allocator);
    errdefer result.deinit();
    var hdr_buf: [5]u8 = undefined;
    const hdr = try encodeFixedHeader(&hdr_buf, .publish, flags, @intCast(list.items.l
en));
    try result.appendSlice(hdr);
    try result.appendSlice(list.items);
    list.deinit();

    return result.toOwnedSlice();
}
```

ポイント: - `pkt.packet_id orelse return CodecError.MalformedPacket` で QoS > 0 なのに Packet ID がない場合をエラーにする - フラグビットは手動で組み立てる（DUP, QoS, Retain を個別にセット） - 2つの `ManagedArrayList` を使い、まずペイロードを構築してから Remaining Length を確定する

PUBLISH のデコード

```
pub fn decodePublish(allocator: Allocator, flags: u4, data: []const u8) !
PublishPacket {
    // フラグビットの解析
    const dup = (flags & 0x08) != 0;
    const qos = QoS.fromInt(@intCast((flags >> 1) & 0x03));
    const retain = (flags & 0x01) != 0;

    var pos: usize = 0;

    // トピック名
    const topic_result = try decodeString(data[pos..]);
    pos += topic_result.bytes_consumed;
    const topic = try allocator.dupe(u8, topic_result.value);
    errdefer allocator.free(topic);

    // Packet ID (QoS >= 1 の場合のみ)
    var packet_id: ?u16 = null;
    if (qos != .at_most_once) {
        packet_id = try decodeU16(data[pos..]);
        pos += 2;
    }

    // 残りがペイロード
    const payload = try allocator.dupe(u8, data[pos..]);

    return .{
```

```
        .dup = dup,
        .qos = qos,
        .retain = retain,
        .topic = topic,
        .packet_id = packet_id,
        .payload = payload,
    };
}
```

ポイント: - 固定ヘッダのフラグ（`u4`）は呼び出し元がデコード済みのものを渡す - `QoS.fromInt()` で `u2` を QoS 列挙型に安全に変換 - `allocator.dupe` でトピックとペイロードをコピーし、呼び出し元に所有権を移転 - `errdefer allocator.free(topic)` でペイロードのコピー失敗時にトピックのメモリを確実に解放

PUBACK のエンコード/デコード

```
pub fn encodePuback(buf: []u8, pkt: *const PubackPacket) ![]const u8 {
    if (buf.len < 4) return CodecError.PacketTooShort;
    buf[0] = 0x40;  // タイプ=4, フラグ=0
    buf[1] = 0x02;  // Remaining Length = 2
    buf[2] = @intCast((pkt.packet_id >> 8) & 0xFF);
    buf[3] = @intCast(pkt.packet_id & 0xFF);
    return buf[0..4];
}

pub fn decodePuback(data: []const u8) !PubackPacket {
    if (data.len < 2) return CodecError.PacketTooShort;
    return .{ .packet_id = try decodeU16(data) };
}
```

PUBACK は固定長であるため、バッファベースの実装でアロケータは不要。

PUBLISH 処理のパターン

Broker が PUBLISH を受信した際の典型的な処理フロー:

```
fn handlePublish(data: PublishPacket, io: std.Io) !void {
    // 1. QoS 1 の場合は PUBACK を送信者に返す
    if (data.packet_id) |id| {
        var buf: [4]u8 = undefined;
        const puback_pkt = PubackPacket{ .packet_id = id };
        const puback = try encodePuback(&buf, &puback_pkt);
        try transport.send(puback);
    }
```

```
    // 2. トピックにマッチする全購読者にメッセージを転送
    //  (topic.zig の topicMatchesFilter を使用)

    // 3. Retain フラグが true の場合、メッセージを保持
    if (data.retain) {
        // retain_store に保存
    }
}
```

オプショナル型による QoS 分岐

```
fn processPublish(data: PublishPacket) !void {
    // QoS 1 の場合のみ PUBACK を送る
    if (data.packet_id) |id| {
        try sendPuback(id);
    }
    // QoS 0 の場合は packet_id == null なので何もしない
}
```

Zig の `if (optional) |unwrapped|` 構文により、 null チェックとアンラップが同時に行える。

テスト

```
test "PUBLISH: encode and decode round-trip" {
    const allocator = std.testing.allocator;
    const pkt = PublishPacket{
        .topic = "sensor/temp",
        .payload = "25.5",
        .qos = .at_least_once,
        .packet_id = 42,
        .retain = true,
    };
    const encoded = try encodePublish(allocator, &pkt);
    defer allocator.free(encoded);

    const header = try decodeFixedHeader(encoded);
    const data = encoded[header.header_size..];
    const decoded = try decodePublish(allocator, header.flags, data);
    defer {
        allocator.free(decoded.topic);
        allocator.free(decoded.payload);
    }

    try std.testing.expectEqualStrings("sensor/temp", decoded.topic);
    try std.testing.expectEqualStrings("25.5", decoded.payload);
    try std.testing.expectEqual(QoS.at_least_once, decoded.qos);
    try std.testing.expectEqual(@as(?u16, 42), decoded.packet_id);
```

```zig
    try std.testing.expectEqual(true, decoded.retain);
}

test "PUBACK: encode and decode" {
    var buf: [4]u8 = undefined;
    const pkt = PubackPacket{ .packet_id = 1234 };
    const encoded = try encodePuback(&buf, &pkt);
    const decoded = try decodePuback(encoded[2..]);
    try std.testing.expectEqual(@as(u16, 1234), decoded.packet_id);
}
```

テストでは `std.testing.allocator` を使うことで、 `defer` で全てのメモリが正しく解放されることを自動検証している。

エラーハンドリングのパターン

PUBLISH のデコードでは複数のエラーが発生しうる:

```zig
const CodecError = error{
    // ...
    MalformedPacket,   // QoS > 0 なのに Packet ID がない
    PacketTooShort,    // データが不足
    OutOfMemory,       // メモリ割り当て失敗
};
```

Zig のエラーユニオン (`!`) により、エラーの型が明示される。 呼び出し元は `try` で伝播するか、 `catch` でハンドリングする:

```zig
const decoded = decodePublish(allocator, flags, data) catch |err| {
    switch (err) {
        error.PacketTooShort => std.log.err("パケットが短すぎます", .{}),
        error.OutOfMemory => std.log.err("メモリ不足", .{}),
        else => std.log.err("デコードエラー: {s}", .{@errorName(err)}),
    }
    return;
};
```

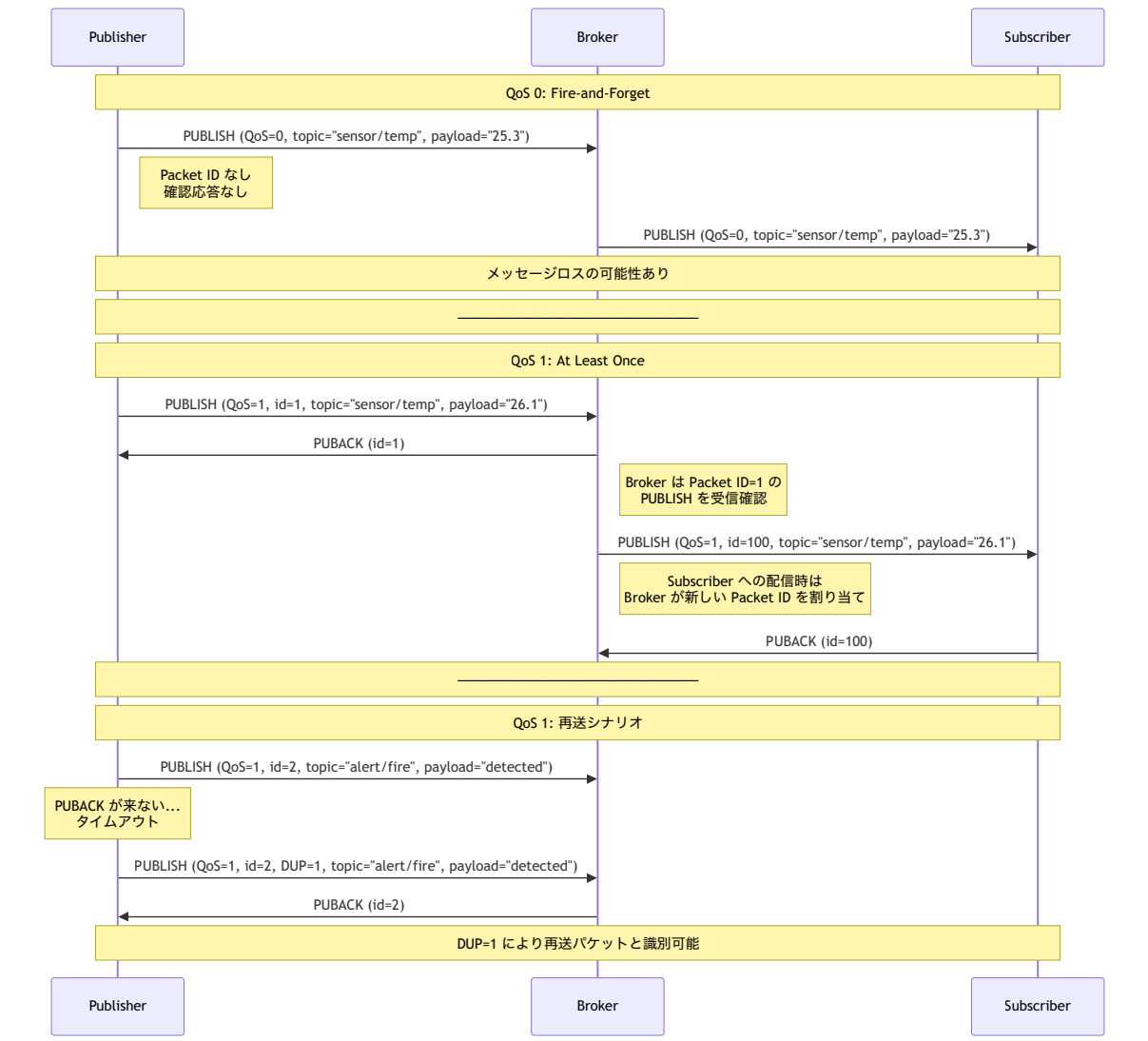
まとめ

- PUBLISH パケットは**フラグビット（DUP/QoS/Retain）+ トピック + オプショナル Packet ID + ペイロード**で構成される
- QoS 0 は確認なしの送りっぱなし、QoS 1 は PUBACK による最低1回の配信保証

- `?u16`（オプショナル型）は Packet ID の有無を型安全に表現し、`if (opt) |val|` で安全にアンラップする
- `ManagedArrayList(u8)` で動的にバイト列を構築し、`errdefer` でエラー時のメモリリークを防止する
- `QoS` 列挙型（`enum(u2)`）により、QoS レベルを型安全に扱える

次のチャプターでは、SUBSCRIBE / UNSUBSCRIBE パケットを学ぶ。

図



第6章 SUBSCRIBE / UNSUBSCRIBE

トピック購読の仕組み

学習目標

- SUBSCRIBE パケットの構造（Packet ID + トピックフィルタリスト）を理解する
- SUBACK パケットの戻りコードの意味を把握する
- UNSUBSCRIBE / UNSUBACK のフローを理解する
- Zig 0.16 の `AlignedManaged` ArrayList パターンを習得する
- `errdefer` によるエラー時のメモリ安全性を理解する
- Broker 側の購読管理の仕組みを把握する

SUBSCRIBE パケット

クライアントが Broker に対してトピックの購読を要求するパケット。

```
[固定ヘッダ]
  Byte 1: 0x82 (タイプ=8, フラグ=0010)
  Byte 2+: Remaining Length

[可変ヘッダ]
  Packet ID: 2バイト (ビッグエンディアン u16)

[ペイロード]
 トピックフィルタ1: MQTT文字列 + QoS (1バイト)
 トピックフィルタ2: MQTT文字列 + QoS (1バイト)
  ... (1つ以上必須)
```

> **注意:** SUBSCRIBE の固定ヘッダフラグは仕様上 `0010` (= 0x02) でなければならない。 他の値を受け取った場合は不正パケットとして接続を切断する。

TopicFilter 構造体

本プロジェクトの `packet.zig` では、購読要素を以下の構造体で表す:

```zig
pub const TopicFilter = struct {
    filter: []const u8,
    qos: QoS,
};
```

`filter` はトピックフィルタ文字列（ワイルドカード `+` / `#` を含む場合がある）、 `qos` は要求する最大 QoS レベルである。

Hex ダンプ例: SUBSCRIBE

トピック "sensor/temp" を QoS 1 で、"sensor/humidity" を QoS 0 で同時購読:

```
82 1E                           -- 固定ヘッダ: SUBSCRIBE, RL=30
00 0A                           -- Packet ID = 10

00 0B 73 65 6E 73 6F 72 2F     -- トピック "sensor/temp" (11バイト)
74 65 6D 70
01                              -- 要求 QoS = 1

00 0F 73 65 6E 73 6F 72 2F     -- トピック "sensor/humidity" (15バイト)
68 75 6D 69 64 69 74 79
00                              -- 要求 QoS = 0
```

ManagedArrayList: Zig 0.16 の動的配列

本プロジェクトでは `std.array_list.AlignedManaged(T, null)` を使っている。 これは Zig 0.16 で提供される**アロケータ内蔵の動的配列**である。

定義

```zig
fn ManagedArrayList(comptime T: type) type {
    return std.array_list.AlignedManaged(T, null);
}
```

基本的な使い方

```zig
const allocator = init.gpa;

// 初期化（アロケータを保持する）
var list = ManagedArrayList(TopicFilter).init(allocator);
```

```
defer list.deinit(); // メモリ解放

// 要素の追加
try list.append(.{ .filter = "sensor/#", .qos = .at_least_once });
try list.append(.{ .filter = "alert/+", .qos = .at_most_once });

// 要素数の取得
const count = list.items.len; // 2

// スライスとして所有権を移転
const owned = try list.toOwnedSlice();
// 呼び出し元が allocator.free(owned) で解放する責任を持つ
```

ArrayList との違い

特徴	`std.ArrayList(T)`	`AlignedManaged(T, null)`
アロケータ	`init(allocator)` で保持	`init(allocator)` で保持
アライメント	デフォルト	明示的に指定可能（null = デフォルト）
Zig 0.16 推奨	--	推奨

SUBSCRIBE のエンコード

```
pub fn encodeSubscribe(allocator: Allocator, pkt: *const SubscribePacket) ![]u8 {
    var list = ManagedArrayList(u8).init(allocator);
    errdefer list.deinit();

    // Packet ID
    try list.append(@intCast((pkt.packet_id >> 8) & 0xFF));
    try list.append(@intCast(pkt.packet_id & 0xFF));

    // トピックフィルタのリスト
    for (pkt.topics) |tf| {
        try appendString(&list, tf.filter);
        try list.append(@intFromEnum(tf.qos));
    }

    // 固定ヘッダ + ペイロードを結合
    var result = ManagedArrayList(u8).init(allocator);
    errdefer result.deinit();
    var hdr_buf: [5]u8 = undefined;
    const hdr = try encodeFixedHeader(&hdr_buf, .subscribe, 0x02,
@intCast(list.items.len));
    try result.appendSlice(hdr);
    try result.appendSlice(list.items);
```

```
    list.deinit();

    return result.toOwnedSlice();
}
```

ポイント: - 固定ヘッダのフラグは仕様上 `0x02` 固定 - `@intFromEnum(tf.qos)` で QoS 列挙型を `u8` に変換 - 2つの `ManagedArrayList` を使い、まずペイロードを構築してから Remaining Length を確定する

SUBSCRIBE のデコード

```
pub fn decodeSubscribe(allocator: Allocator, data: []const u8) !SubscribePacket {
    if (data.len < 2) return CodecError.PacketTooShort;
    var pos: usize = 0;

    // Packet ID
    const packet_id = try decodeU16(data[pos..]);
    pos += 2;

    // トピックフィルタのリストを動的に構築
    var topics = ManagedArrayList(TopicFilter).init(allocator);
    errdefer {
        for (topics.items) |tf| allocator.free(tf.filter);
        topics.deinit();
    }

    while (pos < data.len) {
        // トピック文字列を読む
        const filter_result = try decodeString(data[pos..]);
        pos += filter_result.bytes_consumed;

        if (pos >= data.len) return CodecError.PacketTooShort;

        // QoS バイトを読む
        const qos_byte = data[pos];
        pos += 1;

        // フィルタ文字列をコピーして追加
        const filter_copy = try allocator.dupe(u8, filter_result.value);
        try topics.append(.{
            .filter = filter_copy,
            .qos = QoS.fromInt(@intCast(qos_byte & 0x03)),
        });
    }

    return .{
        .packet_id = packet_id,
        .topics = try topics.toOwnedSlice(),
```

```
    };
}
```

errdefer パターンの詳細

```
var topics = ManagedArrayList(TopicFilter).init(allocator);
errdefer {
    // エラー時: 既に追加した全フィルタ文字列を解放
    for (topics.items) |tf| allocator.free(tf.filter);
    // リスト自体のメモリも解放
    topics.deinit();
}
```

この `errdefer` ブロックは以下の状況で発動する: - `decodeString` がエラーを返した場合 - `allocator.dupe` がメモリ不足で失敗した場合 - `topics.append` がメモリ不足で失敗した場合

成功時は `toOwnedSlice()` で所有権が呼び出し元に移転するため、 `errdefer` は発動しない。

SUBACK パケット

Broker が SUBSCRIBE に応答して送る。各トピックフィルタに対する結果を返す。

```
[固定ヘッダ]
  Byte 1: 0x90 (タイプ=9, フラグ=0)
  Byte 2+: Remaining Length

[可変ヘッダ]
  Packet ID: 2バイト (SUBSCRIBE と同じ値)

[ペイロード]
 戻りコード1: 1バイト
 戻りコード2: 1バイト
  ... (SUBSCRIBE のトピック数と同じ数)
```

SUBACK 戻りコード

```
pub const SubackReturnCode = enum(u8) {
    success_qos0 = 0x00, // 成功: 最大 QoS 0 で承認
    success_qos1 = 0x01, // 成功: 最大 QoS 1 で承認
    success_qos2 = 0x02, // 成功: 最大 QoS 2 で承認
    failure = 0x80,       // 失敗: 購読拒否
    _,                    // 未知の値
};
```

> Broker は要求された QoS より低い QoS を付与することがある。 例: QoS 2 を要求しても、Broker が QoS 1 までしかサポートしない場合は `0x01` を返す。

Hex ダンプ例

```
90 04                    -- 固定ヘッダ: SUBACK, RL=4
00 0A                    -- Packet ID = 10 (SUBSCRIBE と同じ)
01                       -- トピック1: QoS 1 で承認
00                       -- トピック2: QoS 0 で承認
```

SUBACK のエンコード/デコード

エンコード

```
pub fn encodeSuback(allocator: Allocator, pkt: *const SubackPacket) ![]u8 {
    const payload_len: u32 = @intCast(2 + pkt.return_codes.len);

    var result = ManagedArrayList(u8).init(allocator);
    errdefer result.deinit();

    var hdr_buf: [5]u8 = undefined;
    const hdr = try encodeFixedHeader(&hdr_buf, .suback, 0, payload_len);
    try result.appendSlice(hdr);

    // Packet ID
    try result.append(@intCast((pkt.packet_id >> 8) & 0xFF));
    try result.append(@intCast(pkt.packet_id & 0xFF));

    // 戻りコード
    for (pkt.return_codes) |rc| {
        try result.append(@intFromEnum(rc));
    }

    return result.toOwnedSlice();
}
```

デコード

```
pub fn decodeSuback(allocator: Allocator, data: []const u8) !SubackPacket {
    if (data.len < 3) return CodecError.PacketTooShort;
    const packet_id = try decodeU16(data[0..]);
```

```
    const codes_data = data[2..];
    const return_codes = try allocator.alloc(SubackReturnCode, codes_data.len);
    for (codes_data, 0..) |byte, i| {
        return_codes[i] = @enumFromInt(byte);
    }
    return .{
        .packet_id = packet_id,
        .return_codes = return_codes,
    };
}
```

`@enumFromInt(byte)` で各バイトを `SubackReturnCode` に変換する。 `_` が定義されているため、未知の値でもパニックせずにデコードできる。

UNSUBSCRIBE パケット

購読を解除するパケット。SUBSCRIBE と似た構造だが、QoS フィールドがない。

```
[固定ヘッダ]
  Byte 1: 0xA2 (タイプ=10, フラグ=0010)
  Byte 2+: Remaining Length

[可変ヘッダ]
  Packet ID: 2バイト

[ペイロード]
 トピックフィルタ1: MQTT文字列
 トピックフィルタ2: MQTT文字列
  ... (1つ以上必須)
```

Hex ダンプ例

```
A2 0F                           -- UNSUBSCRIBE, RL=15
00 0B                           -- Packet ID = 11
00 0B 73 65 6E 73 6F 72 2F     -- "sensor/temp" (11バイト)
74 65 6D 70
```

UNSUBSCRIBE のデコード

```
pub fn decodeUnsubscribe(allocator: Allocator, data: []const u8) !UnsubscribePacket {
    if (data.len < 2) return CodecError.PacketTooShort;
    var pos: usize = 0;
    const packet_id = try decodeU16(data[pos..]);
    pos += 2;
```

```
    var topics = ManagedArrayList([]const u8).init(allocator);
    errdefer {
        for (topics.items) |t| allocator.free(t);
        topics.deinit();
    }

    while (pos < data.len) {
        const s = try decodeString(data[pos..]);
        pos += s.bytes_consumed;
        try topics.append(try allocator.dupe(u8, s.value));
    }

    return .{
        .packet_id = packet_id,
        .topics = try topics.toOwnedSlice(),
    };
}
```

SUBSCRIBE と同じく `ManagedArrayList` と `errdefer` パターンを使用する。 QoS フィールドがないため、トピック文字列のみを読み取る。

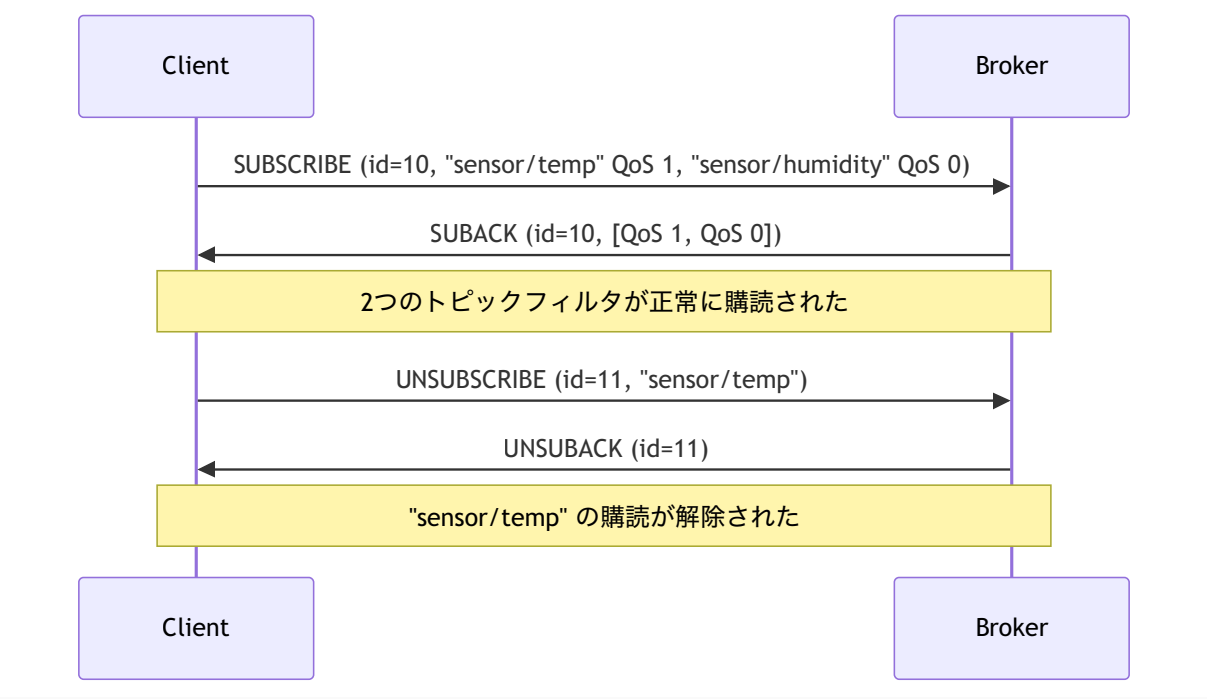
UNSUBACK パケット

UNSUBSCRIBE に対する応答。固定長4バイト。

```
B0 02 00 0B
|  |  +--+-- Packet ID = 11
|  +-------- Remaining Length = 2
+----------- タイプ=11 (UNSUBACK), フラグ=0
```

```
pub fn encodeUnsuback(buf: []u8, pkt: *const UnsubackPacket) ![]const u8 {
    if (buf.len < 4) return CodecError.PacketTooShort;
    buf[0] = 0xB0; // UNSUBACK type=11, flags=0
    buf[1] = 0x02;
    buf[2] = @intCast((pkt.packet_id >> 8) & 0xFF);
    buf[3] = @intCast(pkt.packet_id & 0xFF);
    return buf[0..4];
}
```

購読フローの図解



Broker の購読管理

Broker はクライアントごとの購読情報を管理する必要がある。 `HashMap` と `ManagedArrayList` を組み合わせて実装する:

```
const Subscription = struct {
    client_id: []const u8,
    qos: QoS,
};

const SubscriptionManager = struct {
    subscriptions: std.StringHashMap(ManagedArrayList(Subscription)),
    allocator: Allocator,

    pub fn init(allocator: Allocator) SubscriptionManager {
        return .{
            .subscriptions = std.StringHashMap(
                ManagedArrayList(Subscription),
            ).init(allocator),
            .allocator = allocator,
        };
    }
```

```zig
    pub fn subscribe(
        self: *SubscriptionManager,
        filter: []const u8,
        client_id: []const u8,
        qos: QoS,
    ) !void {
        const result = try self.subscriptions.getOrPut(filter);
        if (!result.found_existing) {
            result.value_ptr.* = ManagedArrayList(Subscription).init(self.allocator);
        }
        try result.value_ptr.append(.{
            .client_id = client_id,
            .qos = qos,
        });
    }

    pub fn unsubscribe(
        self: *SubscriptionManager,
        filter: []const u8,
        client_id: []const u8,
    ) void {
        if (self.subscriptions.getPtr(filter)) |subs| {
            var i: usize = 0;
            while (i < subs.items.len) {
                if (std.mem.eql(u8, subs.items[i].client_id, client_id)) {
                    _ = subs.swapRemove(i);
                } else {
                    i += 1;
                }
            }
        }
    }
};
```

メモリ管理のパターン

```zig
// テスト時: std.testing.allocator でメモリリーク検出
test "subscribe and unsubscribe" {
    const allocator = std.testing.allocator;
    var manager = SubscriptionManager.init(allocator);
    defer manager.deinit();

    try manager.subscribe("sensor/+", "client-1", .at_least_once);
    try manager.subscribe("sensor/+", "client-2", .at_most_once);

    manager.unsubscribe("sensor/+", "client-1");
    // テスト終了時にメモリリークがあれば自動検出される
}
```

テスト

```
test "SUBSCRIBE: encode and decode round-trip" {
    const allocator = std.testing.allocator;
    const topics = [_]TopicFilter{
        .{ .filter = "sensor/+/temp", .qos = .at_least_once },
        .{ .filter = "alert/#", .qos = .at_most_once },
    };
    const pkt = SubscribePacket{
        .packet_id = 100,
        .topics = &topics,
    };
    const encoded = try encodeSubscribe(allocator, &pkt);
    defer allocator.free(encoded);

    const header = try decodeFixedHeader(encoded);
    const data = encoded[header.header_size..];
    const decoded = try decodeSubscribe(allocator, data);
    defer {
        for (decoded.topics) |tf| allocator.free(tf.filter);
        allocator.free(decoded.topics);
    }

    try std.testing.expectEqual(@as(u16, 100), decoded.packet_id);
    try std.testing.expectEqual(@as(usize, 2), decoded.topics.len);
    try std.testing.expectEqualStrings("sensor/+/temp", decoded.topics[0].filter);
    try std.testing.expectEqual(QoS.at_least_once, decoded.topics[0].qos);
    try std.testing.expectEqualStrings("alert/#", decoded.topics[1].filter);
    try std.testing.expectEqual(QoS.at_most_once, decoded.topics[1].qos);
}
```

テストでは: - `std.testing.allocator` を使いメモリリークを自動検出 - `defer` でデコード結果のメモリを確実に解放 - ラウンドトリップ（エンコード → デコード → 比較）で正確性を検証

メモリ解放の全体像

`packet.zig` の `freePacket` 関数がパケット種別に応じた解放を行う:

```
pub fn freePacket(allocator: Allocator, packet_val: *const Packet) void {
    switch (packet_val.*) {
        .subscribe => |p| {
            for (p.topics) |tf| {
                allocator.free(tf.filter);
            }
            allocator.free(p.topics);
        },
        .unsubscribe => |p| {
```

```
            for (p.topics) |t| {
                allocator.free(t);
            }
            allocator.free(p.topics);
        },
        .suback => |p| {
            allocator.free(p.return_codes);
        },
        // ... 他のパケット種別 ...
        else => {},
    }
}
```

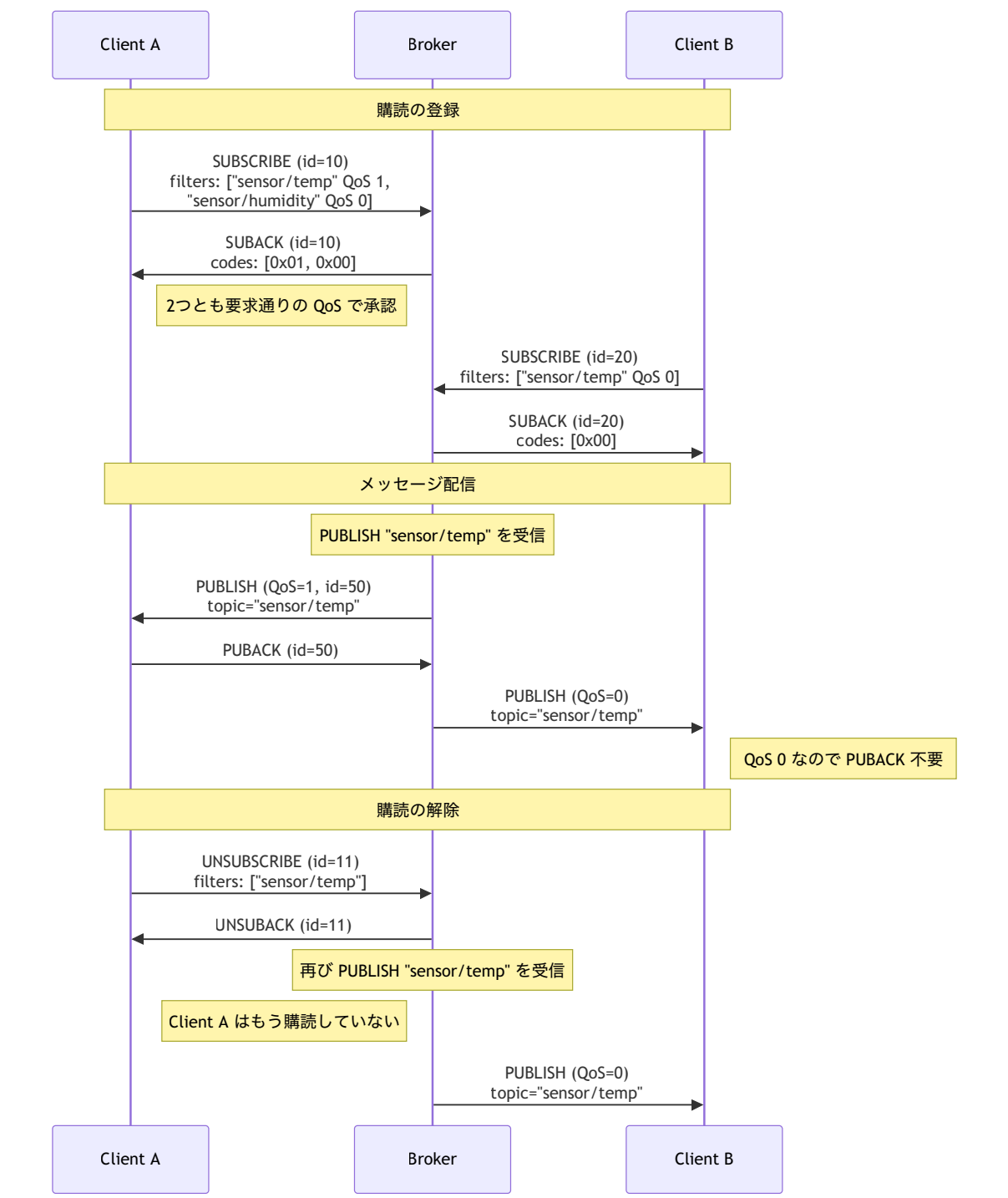
`tagged union` の `switch` で全パケット種別を網羅的に処理できる。

まとめ

- SUBSCRIBE は **Packet ID + トピックフィルタ/QoS のリスト** で構成される
- SUBACK は各フィルタに対する承認 QoS（または拒否 `0x80`）を返す
- UNSUBSCRIBE / UNSUBACK は購読解除のペアで、Packet ID で対応付ける
- Zig 0.16 の `AlignedManaged(T, null)` はアロケータ内蔵の動的配列であり、SUBSCRIBE のような可変長ペイロードの構築・デコードに適している
- `errdefer` パターンにより、デコード途中のエラーでもメモリリークを防止できる
- Broker 側は `HashMap` + `ManagedArrayList` で購読情報を管理する

次のチャプターでは、トピックフィルタのワイルドカードマッチングを学ぶ。

図


Client A
Broker
Client B
購読の登録
SUBSCRIBE (id=10)
filters: ["sensor/temp" QoS 1,
"sensor/humidity" QoS 0]
SUBACK (id=10)
codes: [0x01, 0x00]
2つとも要求通りの QoS で承認
SUBSCRIBE (id=20)
filters: ["sensor/temp" QoS 0]
SUBACK (id=20)
codes: [0x00]
メッセージ配信
PUBLISH "sensor/temp" を受信
PUBLISH (QoS=1, id=50)
topic="sensor/temp"
PUBACK (id=50)
PUBLISH (QoS=0)
topic="sensor/temp"
QoS 0 なので PUBACK 不要
購読の解除
UNSUBSCRIBE (id=11)
filters: ["sensor/temp"]
UNSUBACK (id=11)
再び PUBLISH "sensor/temp" を受信
Client A はもう購読していない
PUBLISH (QoS=0)
topic="sensor/temp"
Client A
Broker
Client B

第III部 ブローカーロジック

ブローカーの中核機能を実装します

第7章 トピックワイルドカード

+と#のマッチングアルゴリズム

学習目標

- MQTT のトピック名（Topic Name）とトピックフィルタ（Topic Filter）の違いを理解する
- 単一レベルワイルドカード `+` の動作を把握する
- 複数レベルワイルドカード `#` の動作を把握する
- `$SYS` トピックの特別な扱いを理解する
- 本プロジェクトの `topic.zig` の実装を読み解く
- Zig の `std.testing` を活用したテスト手法を習得する

トピック名 vs トピックフィルタ

トピック名（Topic Name）

PUBLISH パケットで使われる。ワイルドカードを含んではならない。

```
sensor/temperature        -- 有効
factory/line1/status      -- 有効
sensor/+/data             -- 無効（ワイルドカードは使えない）
```

トピックフィルタ（Topic Filter）

SUBSCRIBE / UNSUBSCRIBE で使われる。ワイルドカードを含むことができる。

```
sensor/temperature        -- 完全一致
sensor/+/data             -- 単一レベルワイルドカード
sensor/#                  -- 複数レベルワイルドカード
```

トピックのレベル構造

トピックは `/` で区切られた**レベル**で構成される:

```
sensor/temperature/celsius
  |         |          +-- レベル3
```

```
  |          +------------- レベル2
  +----------------------- レベル1
```

単一レベルワイルドカード: `+`

`+` は**ちょうど1つのレベル**にマッチする。

マッチ例

フィルタ	トピック名	マッチ?
`sensor/+/data`	`sensor/temp/data`	Yes
`sensor/+/data`	`sensor/humidity/data`	Yes
`sensor/+/data`	`sensor/a/b/data`	No
`+/temperature`	`sensor/temperature`	Yes
`+/+`	`sensor/temperature`	Yes
`+`	`sensor`	Yes
`+`	`sensor/temperature`	No

ポイント: - `+` は**1つのレベル全体**を置換する - 空のレベルにもマッチする（例: `+/+` は `/finance` にマッチ -- 先頭が空レベル） - レベル区切り `/` を超えることはない

複数レベルワイルドカード: `#`

`#` は**0個以上のレベル**にマッチする。フィルタの**最後**にのみ使用できる。

マッチ例

フィルタ	トピック名	マッチ?
`sensor/#`	`sensor`	Yes
`sensor/#`	`sensor/temp`	Yes
`sensor/#`	`sensor/temp/celsius`	Yes

フィルタ	トピック名	マッチ?
`#`	`sensor/temp`	Yes
`#`	`any/topic/at/all`	Yes
`sensor/temp/#`	`sensor/temp`	Yes
`sensor/temp/#`	`sensor/temp/celsius`	Yes
`sensor/#/data`	(無効なフィルタ)	--

ポイント: - `#` は**フィルタの末尾でなければならない** - `#` の前には `/` が必要（ただし `#` 単体は例外） - **0レベル**にもマッチする（`sensor/#` は `sensor` 自体にマッチ）

`$SYS` トピックの特別扱い

`$` で始まるトピックは**システムトピック**であり、ワイルドカードフィルタからは除外される。

```
フィルタ "#" は "$SYS/broker/clients" にマッチしない
フィルタ "+/broker/clients" は "$SYS/broker/clients" にマッチしない
フィルタ "$SYS/#" は "$SYS/broker/clients" にマッチする
フィルタ "$SYS/broker/+" は "$SYS/broker/clients" にマッチする
```

理由: `#` や `+` が全てのシステムトピックにマッチすると、 クライアントが意図せず大量のシステムメッセージを受信してしまうため。

---

topic.zig の実装

本プロジェクトの `src/mqtt/topic.zig` にトピック関連の全ロジックを実装している。

トピック名のバリデーション

PUBLISH 用のトピック名にはワイルドカードを含んではならない:

```
pub fn isValidTopicName(name: []const u8) bool {
    if (name.len == 0 or name.len > 65535) return false;
    for (name) |c| {
        if (c == '+' or c == '#') return false;
        if (c == 0) return false; // NULL 文字禁止
    }
```

```
    return true;
}
```

トピックフィルタのバリデーション

SUBSCRIBE 用のフィルタにはワイルドカードのルールがある:

```
pub fn isValidTopicFilter(filter: []const u8) bool {
    if (filter.len == 0 or filter.len > 65535) return false;

    var iter = TopicLevelIterator.init(filter);
    var level_count: usize = 0;
    while (iter.next()) |level| {
        level_count += 1;
        // '#' は最後のレベルかつ単独でなければならない
        if (std.mem.indexOfScalar(u8, level, '#')) |_| {
            if (level.len != 1) return false;
            if (iter.next() != null) return false;
            return true;
        }
        // '+' は単独のレベルでなければならない
        if (std.mem.indexOfScalar(u8, level, '+')) |_| {
            if (level.len != 1) return false;
        }
        // NULL 文字禁止
        if (std.mem.indexOfScalar(u8, level, 0)) |_| return false;
    }
    return level_count > 0;
}
```

バリデーションルール: - `#` はフィルタ末尾のレベルにのみ出現可能で、そのレベルは `#` のみ - `+` はレベル全体を占める必要がある（`sensor/te+mp` は不正） - NULL 文字は禁止 - 空文字列は不正

TopicLevelIterator

トピック文字列を `/` で分割するイテレータ:

```
pub const TopicLevelIterator = struct {
    data: []const u8,
    pos: usize,
    done: bool,

    pub fn init(topic: []const u8) TopicLevelIterator {
        return .{
            .data = topic,
            .pos = 0,
            .done = false,
```

```
        };
    }

    pub fn next(self: *TopicLevelIterator) ?[]const u8 {
        if (self.done) return null;

        const start = self.pos;
        while (self.pos < self.data.len and self.data[self.pos] != '/') {
            self.pos += 1;
        }

        const level = self.data[start..self.pos];

        if (self.pos < self.data.len) {
            self.pos += 1; // '/' をスキップ
        } else {
            self.done = true;
        }

        return level;
    }
};
```

Zig には組み込みのイテレータ構文がないが、 `next()` メソッドが `?T` （オプショナル型）を返す慣用パターンで `while` ループと組み合わせて使う:

```
var iter = TopicLevelIterator.init("sensor/temp/celsius");
while (iter.next()) |level| {
    std.debug.print("レベル: {s}\n", .{level});
}
// 出力:
// レベル: sensor
// レベル: temp
// レベル: celsius
```

標準ライブラリとの比較

`std.mem.splitScalar` でも同様の分割が可能:

```
var it = std.mem.splitScalar(u8, "sensor/temp/celsius", '/');
while (it.next()) |level| {
    // ...
}
```

本プロジェクトでは `done` フラグを持つ独自イテレータを使うことで、 末尾の `/` やエッジケースの制御を明示的に行っている。

---

topicMatchesFilter 関数

本プロジェクトの核心となるマッチングロジック:

```
pub fn topicMatchesFilter(topic: []const u8, filter: []const u8) bool {
    // $SYS チェック: ワイルドカードで始まるフィルタは $ トピックにマッチしない
    if (topic.len > 0 and topic[0] == '$') {
        if (filter.len > 0 and (filter[0] == '+' or filter[0] == '#')) {
            return false;
        }
    }

    var topic_iter = TopicLevelIterator.init(topic);
    var filter_iter = TopicLevelIterator.init(filter);

    while (true) {
        const filter_level = filter_iter.next();
        const topic_level = topic_iter.next();

        if (filter_level == null and topic_level == null) {
            return true; // 両方とも終了 -> マッチ
        }

        if (filter_level) |fl| {
            // '#' は残りの全レベルにマッチ
            if (std.mem.eql(u8, fl, "#")) {
                return true;
            }

            if (topic_level) |tl| {
                // '+' は任意の1レベルにマッチ
                if (std.mem.eql(u8, fl, "+")) {
                    continue;
                }
                // 完全一致
                if (std.mem.eql(u8, fl, tl)) {
                    continue;
                }
                return false; // レベルが不一致
            } else {
                return false; // フィルタにまだレベルがあるがトピックが終了
            }
        } else {
            return false; // トピックにまだレベルがあるがフィルタが終了
        }
    }
}
```

アルゴリズムの流れ

1. `$SYS` トピックの特別扱い: フィルタが `+` や `#` で始まる場合は即座に `false`
2. トピックとフィルタを同時にレベルごとに走査
3. フィルタレベルが `#` なら残り全てにマッチ -> `true`
4. フィルタレベルが `+` なら1レベルスキップ -> `continue`
5. フィルタレベルとトピックレベルが一致 -> `continue`
6. 両方とも `null`（同時に終了） -> `true`
7. それ以外 -> `false`

テストケース

`topic.zig` には `std.testing` を使った網羅的なテストが含まれている:

バリデーションテスト

```
test "isValidTopicName: basic" {
    try std.testing.expect(isValidTopicName("sensor/temp"));
    try std.testing.expect(isValidTopicName("a"));
    try std.testing.expect(isValidTopicName("/"));
    try std.testing.expect(isValidTopicName("a/b/c"));
    try std.testing.expect(!isValidTopicName(""));
    try std.testing.expect(!isValidTopicName("sensor/+/temp"));
    try std.testing.expect(!isValidTopicName("sensor/#"));
}

test "isValidTopicFilter: basic" {
    try std.testing.expect(isValidTopicFilter("sensor/temp"));
    try std.testing.expect(isValidTopicFilter("#"));
    try std.testing.expect(isValidTopicFilter("+"));
    try std.testing.expect(isValidTopicFilter("sensor/+/temp"));
    try std.testing.expect(isValidTopicFilter("sensor/#"));
    try std.testing.expect(!isValidTopicFilter(""));
    try std.testing.expect(!isValidTopicFilter("sensor/temp#"));
    try std.testing.expect(!isValidTopicFilter("sensor/#/temp"));
    try std.testing.expect(!isValidTopicFilter("sensor/te+mp"));
}
```

完全一致テスト

```
test "topicMatchesFilter: exact match" {
    try std.testing.expect(topicMatchesFilter("sensor/temp", "sensor/temp"));
    try std.testing.expect(!topicMatchesFilter("sensor/temp", "sensor/humidity"));
```

```
    try std.testing.expect(!topicMatchesFilter("sensor/temp", "sensor"));
}
```

単一レベルワイルドカードテスト

```
test "topicMatchesFilter: single-level wildcard (+)" {
    try std.testing.expect(topicMatchesFilter("sensor/temp", "sensor/+"));
    try std.testing.expect(topicMatchesFilter("sensor/humidity", "sensor/+"));
    try std.testing.expect(!topicMatchesFilter("sensor/a/b", "sensor/+"));
    try std.testing.expect(topicMatchesFilter("a/b/c", "+/b/c"));
    try std.testing.expect(topicMatchesFilter("a/b/c", "+/+/+"));
}
```

複数レベルワイルドカードテスト

```
test "topicMatchesFilter: multi-level wildcard (#)" {
    try std.testing.expect(topicMatchesFilter("sensor/temp", "sensor/#"));
    try std.testing.expect(topicMatchesFilter("sensor/a/b/c", "sensor/#"));
    try std.testing.expect(topicMatchesFilter("sensor", "sensor/#"));
    try std.testing.expect(topicMatchesFilter("anything", "#"));
    try std.testing.expect(topicMatchesFilter("a/b/c", "#"));
}
```

$SYS 除外テスト

```
test "topicMatchesFilter: $SYS exclusion" {
    try std.testing.expect(!topicMatchesFilter("$SYS/info", "#"));
    try std.testing.expect(!topicMatchesFilter("$SYS/info", "+/info"));
    try std.testing.expect(topicMatchesFilter("$SYS/info", "$SYS/#"));
    try std.testing.expect(topicMatchesFilter("$SYS/info", "$SYS/info"));
}
```

エッジケーステスト

```
test "topicMatchesFilter: edge cases" {
    // 空レベル
    try std.testing.expect(topicMatchesFilter("/finance", "/finance"));
    try std.testing.expect(topicMatchesFilter("/finance", "/+"));
    try std.testing.expect(topicMatchesFilter("/finance", "+/+"));
    // sensor/# は sensor 自体にもマッチ
    try std.testing.expect(topicMatchesFilter("sport", "sport/#"));
}
```

イテレータのテスト

```
test "TopicLevelIterator" {
    var iter = TopicLevelIterator.init("a/b/c");
    try std.testing.expectEqualStrings("a", iter.next().?);
    try std.testing.expectEqualStrings("b", iter.next().?);
    try std.testing.expectEqualStrings("c", iter.next().?);
    try std.testing.expectEqual(@as(?[]const u8, null), iter.next());

    // 先頭スラッシュ
    var iter2 = TopicLevelIterator.init("/a");
    try std.testing.expectEqualStrings("", iter2.next().?);
    try std.testing.expectEqualStrings("a", iter2.next().?);
    try std.testing.expectEqual(@as(?[]const u8, null), iter2.next());
}
```

テストの実行

```
zig build test
```

std.testing の主要な関数

関数	説明
`expect(ok)`	`ok` が `true` でなければテスト失敗
`expectEqual(expected, actual)`	値が一致しなければテスト失敗
`expectEqualStrings(expected, actual)`	文字列が一致しなければテスト失敗
`expectEqualSlices(T, expected, actual)`	スライスが一致しなければテスト失敗
`std.testing.allocator`	テスト用アロケータ（メモリリーク検出付き）

`std.testing.expect` は `!void` を返すため、`try` と組み合わせて使う:

```
test "example" {
    try std.testing.expect(true);  // OK
    try std.testing.expect(false); // テスト失敗
}
```

マッチングの計算量

- トピックのレベル数を N、フィルタのレベル数を M とすると、マッチングは **O(min(N, M))** で完了する
- `#` が出現した時点で即座に `true` を返せるため、最良ケースは O(1)
- Broker が多数のフィルタを持つ場合、全フィルタに対して線形にマッチングするため、フィルタ数 K に対して **O(K * N)** になる
- 大規模な Broker ではトライ木（Trie）を使って最適化することが多い

Broker での使用例

PUBLISH 受信時に、全購読フィルタに対してマッチングを行う:

```
fn distributeMessage(
    topic: []const u8,
    payload: []const u8,
    subscriptions: *SubscriptionManager,
) void {
    var iter = subscriptions.subscriptions.iterator();
    while (iter.next()) |entry| {
        const filter = entry.key_ptr.*;
        if (topicMatchesFilter(topic, filter)) {
            // マッチしたフィルタの全購読者にメッセージを転送
            for (entry.value_ptr.items) |sub| {
                // sub.client_id に対して PUBLISH を送信
                _ = sub;
            }
        }
    }
}
```

`topicMatchesFilter` は純粋関数（副作用なし）であるため、 テストが容易でスレッドセーフである。

まとめ

- **トピック名**は PUBLISH で使用し、ワイルドカードを含まない
- **トピックフィルタ**は SUBSCRIBE で使用し、`+`（単一レベル）と `#`（複数レベル）のワイルドカードを使える
- `$SYS` で始まるトピックは、先頭がワイルドカードのフィルタからは除外される

- `topicMatchesFilter` はレベルごとに逐次比較するシンプルなアルゴリズムで実装でき、計算量は O(min(N, M))
- `isValidTopicName` / `isValidTopicFilter` でバリデーションを行い、不正なトピックを事前に排除する
- `std.testing` で `expect` / `expectEqual` / `expectEqualStrings` を使い、テストケースを網羅的に記述する

これで MQTT v3.1.1 の主要な概念とパケット構造の学習は完了である。 ここまでの知識を組み合わせることで、Pure Zig 0.16 による MQTT Broker / Client の実装を理解し、拡張できるようになる。

第8章 QoS配信保証

パケットID管理と再送

学習目標

- MQTT の QoS 0, QoS 1, QoS 2 の違いを理解する
- パケットIDの管理方法（atomic カウンタによる単調増加）を実装できる
- In-flight メッセージの追跡に HashMap を活用できる
- Zig 0.16 の `std.atomic.Value` を用いたスレッドセーフなカウンタを理解する

QoS とは

MQTT では、メッセージ配信の信頼性を3段階で指定できる。 パブリッシャーがブローカーへ送る際の QoS と、ブローカーがサブスクライバーへ転送する際の QoS は独立して扱われる。

レベル	名称	配信保証
QoS 0	At most once	最大1回。ロスの可能性あり
QoS 1	At least once	最低1回。重複の可能性あり
QoS 2	Exactly once	正確に1回。最も重い

QoS 0: At most once（火と忘れ）

QoS 0 は最もシンプルなフローである。PUBLISH パケットを送信したら、それで完了となる。 確認応答は一切行われない。

```
Publisher ——— PUBLISH (QoS 0) ———> Broker ——— PUBLISH (QoS 0) ———> Subscriber
```

パケットIDフィールドは QoS 0 では存在しない。

実装上のポイント

```
fn handlePublish(self: *Connection, io: *std.Io, packet: PublishPacket) !void {
    if (packet.qos == .at_most_once) {
```

```
        // QoS 0: 確認応答なし。即座にルーティングする
        try self.router.route(io, packet.topic, packet.payload, .at_most_once);
        return;
    }
    // QoS 1 以上の処理は後述
}
```

QoS 0 は IoT センサーの温度データなど、多少のロスが許容されるユースケースに適している。

QoS 1: At least once

QoS 1 では、送信側は PUBLISH を送り、受信側は PUBACK で確認応答を返す。 PUBACK を受信するまで、送信側はメッセージを保持し、タイムアウト後に再送する。

```
Publisher                     Broker                      Subscriber
   |                            |                            |
   |── PUBLISH (QoS 1, id=1) ─>|                            |
   |                            |── PUBLISH (QoS 1, id=7) ─>|
   |                            |<── PUBACK (id=7) ──────────|
   |<── PUBACK (id=1) ─────────|                            |
```

重要な点として、パブリッシャーからブローカーへの packet_id と、ブローカーからサブスクライバーへの packet_id は別の値になる。

パケットIDの管理

パケットIDは 1 から 65535 の範囲で単調増加させる。0 は使用しない。 Zig 0.16 の `std.atomic.Value` を使えば、複数スレッドから安全にIDを生成できる。

```
const PacketIdGenerator = struct {
    counter: std.atomic.Value(u16),

    pub fn init() PacketIdGenerator {
        return .{
            .counter = std.atomic.Value(u16).init(0),
        };
    }

    /// スレッドセーフにパケットIDを生成する
    /// CAS (Compare-And-Swap) ループにより競合時にリトライする
    pub fn next(self: *PacketIdGenerator) u16 {
        while (true) {
            const current = self.counter.load(.acquire);
            const new_val = if (current == 65535) 1 else current + 1;
            if (self.counter.cmpxchgWeak(
                current,
```

```
                new_val,
                .release,
                .monotonic,
            ) == null) {
                return new_val;
            }
            // 競合発生時はループしてリトライ
        }
    }
};
```

`cmpxchgWeak` は、 `current` が実際の値と一致する場合のみ `new_val` を書き込む。 他スレッドが先に更新していた場合は `null` 以外を返すため、ループで再試行する。

メモリオーダリングの選択

オーダリング	用途
`.acquire`	load 時: この読み取り以降の操作が先行しない
`.release`	store 時: この書き込み以前の操作が後続しない
`.monotonic`	失敗時: 最小限の保証で十分

パケットID生成では厳密な順序保証は不要だが、値の一貫性は必要であるため、acquire/release を使う。

In-flight メッセージの追跡

PUBACK を受け取るまでメッセージを保持するため、HashMap を使う。

```
const InFlightMessage = struct {
    topic: []const u8,
    payload: []const u8,
    sent_at: i64,
    retry_count: u8,
};

const InFlightMap = std.HashMap(
    u16,
    InFlightMessage,
    std.hash_map.AutoContext(u16),
    80,
);
```

QoS 1 メッセージの送信と追跡

```
fn sendQos1(
    self: *Connection,
    io: *std.Io,
    topic: []const u8,
    payload: []const u8,
) !void {
    const packet_id = self.packet_id_gen.next();

    // In-flight マップに登録
    try self.in_flight.put(packet_id, .{
        .topic = topic,
        .payload = payload,
        .sent_at = std.time.milliTimestamp(),
        .retry_count = 0,
    });

    // PUBLISH パケットを送信
    try self.sendPublishPacket(io, topic, payload, packet_id, .at_least_once);
}

fn handlePuback(self: *Connection, packet_id: u16) void {
    // 確認応答を受信したので、in-flight マップから削除
    _ = self.in_flight.remove(packet_id);
}
```

再送ロジック

一定時間内に PUBACK が届かない場合、DUP フラグを立てて再送する。

```
const retry_timeout_ms: i64 = 5000; // 5秒

fn retryInFlightMessages(self: *Connection, io: *std.Io) !void {
    const now = std.time.milliTimestamp();
    var iter = self.in_flight.iterator();

    while (iter.next()) |entry| {
        const msg = entry.value_ptr;
        const elapsed = now - msg.sent_at;

        if (elapsed > retry_timeout_ms) {
            msg.retry_count += 1;
            msg.sent_at = now;
            // DUP=true で再送
            try self.sendPublishPacket(
                io,
                msg.topic,
                msg.payload,
                entry.key_ptr.*,
```

```
                .at_least_once,
            );
        }
    }
}
```

再送タイマーの実装には Zig 0.16 の sleep API を活用できる。

```zig
fn retryLoop(self: *Connection, io: *std.Io) !void {
    while (!self.should_stop.load(.acquire)) {
        std.Io.Clock.Duration.sleep(.{
            .clock = .awake,
            .raw = .fromMilliseconds(1000),
        }, io);
        try self.retryInFlightMessages(io);
    }
}
```

QoS 2: Exactly once（概念説明）

QoS 2 は4ステップのハンドシェイクにより、正確に1回の配信を保証する。 本プロジェクトでは実装しないが、概念を理解しておくことは重要である。

```
Publisher                    Broker
   |                           |
   |—— PUBLISH (QoS 2, id=1) —>|
   |<—— PUBREC  (id=1) ————————|   <- 受信確認
   |—— PUBREL   (id=1) ————————>|   <- 解放要求
   |<—— PUBCOMP (id=1) ————————|   <- 完了通知
```

各ステップの役割:

1. **PUBLISH**: パブリッシャーがメッセージを送信
2. **PUBREC** (Publish Received): ブローカーが受信を確認。この時点でメッセージを保存
3. **PUBREL** (Publish Release): パブリッシャーが解放を指示。ブローカーはサブスクライバーへ転送
4. **PUBCOMP** (Publish Complete): ブローカーが完了を通知。双方が状態をクリア

QoS 2 を実装しない理由は、オーバーヘッドが大きく、多くの IoT ユースケースでは QoS 1 で十分なためである。

PUBACK パケットのエンコード/デコード

PUBACK は非常にシンプルな固定長パケットである。

```
バイト1: 0x40 (パケットタイプ=4, フラグ=0)
バイト2: 0x02 (残りの長さ=2)
バイト3-4: パケットID (u16 big-endian)
```

```
fn encodePuback(packet_id: u16) [4]u8 {
    return .{
        0x40,                           // PUBACK パケットタイプ
        0x02,                           // 残りの長さ
        @intCast(packet_id >> 8),       // パケットID上位バイト
        @intCast(packet_id & 0xFF),     // パケットID下位バイト
    };
}

fn decodePuback(data: []const u8) !u16 {
    if (data.len < 4) return error.IncompletePacket;
    if (data[0] != 0x40) return error.InvalidPacketType;
    return @as(u16, data[2]) << 8 | @as(u16, data[3]);
}
```

サブスクリプションの QoS ダウングレード

サブスクライバーが QoS 1 でサブスクライブしていても、パブリッシャーが QoS 0 で送信した場合、ブローカーは QoS 0 で転送する。配信 QoS は両者の最小値になる。

```
fn effectiveQos(publish_qos: QoS, subscription_qos: QoS) QoS {
    return @enumFromInt(@min(
        @intFromEnum(publish_qos),
        @intFromEnum(subscription_qos),
    ));
}
```

このルールにより、パブリッシャーが意図しない高い QoS で配信されることを防ぐ。

atomic パターンのまとめ

Zig 0.16 で利用可能な atomic 操作の一覧:

```zig
const AtomicU16 = std.atomic.Value(u16);

// 初期化
var counter = AtomicU16.init(0);

// 読み取り
const val = counter.load(.acquire);

// 書き込み
counter.store(42, .release);

// CAS (Compare-And-Swap)
// 成功時は null、失敗時は実際の値を返す
if (counter.cmpxchgWeak(expected, desired, .release, .monotonic)) |actual| {
    // 競合発生: actual が現在の値
} else {
    // 成功: expected -> desired に更新された
}

// fetchAdd / fetchSub も利用可能
const old = counter.fetchAdd(1, .monotonic);
```

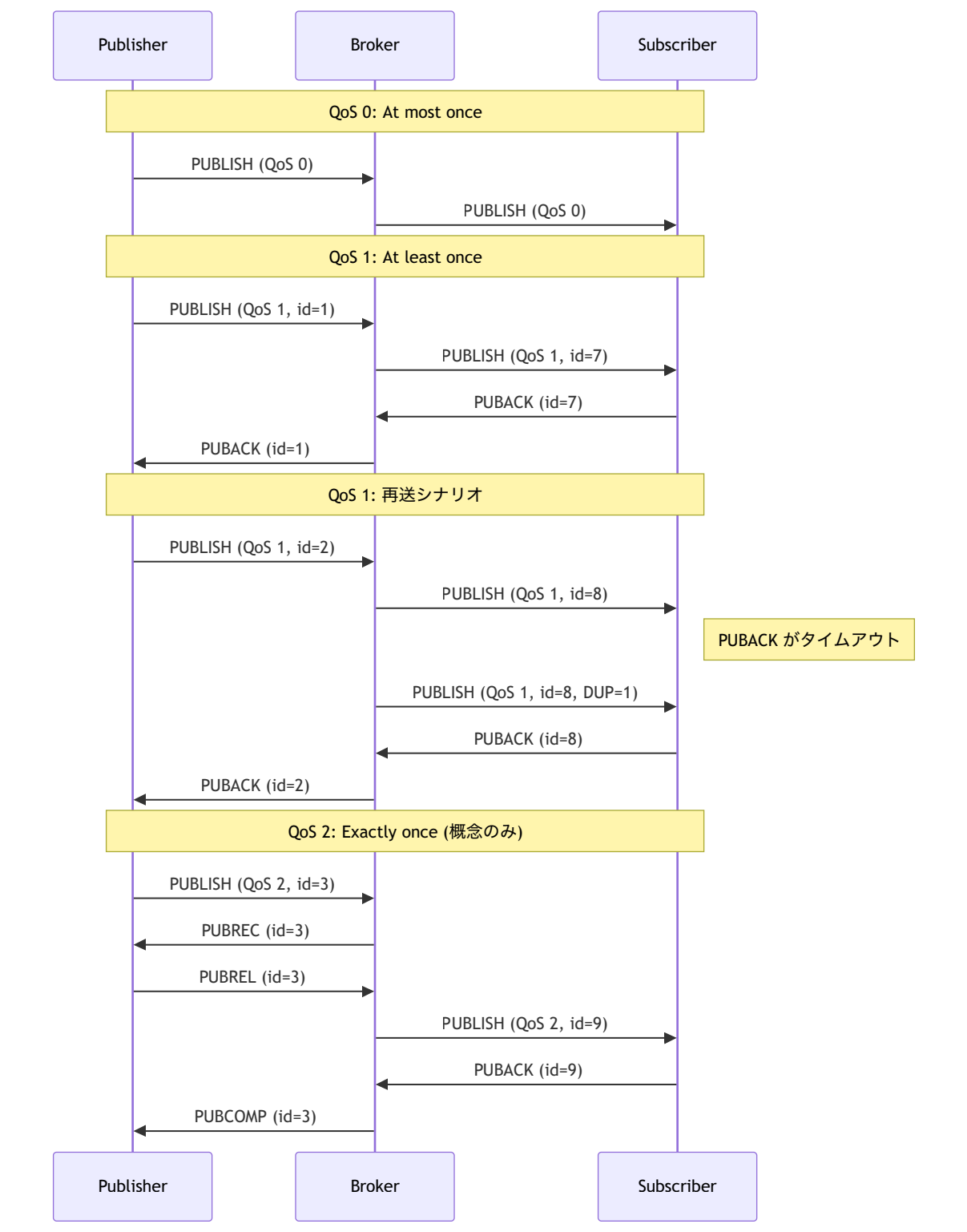
まとめ

項目	QoS 0	QoS 1	QoS 2
確認応答	なし	PUBACK	4-way ハンドシェイク
パケットID	なし	必要	必要
重複配信	なし	あり得る	なし
実装複雑度	低	中	高

- `std.atomic.Value` で CAS ループによるスレッドセーフなID生成を実現する
- In-flight HashMap でタイムアウト再送を管理する
- QoS ダウングレードにより、配信 QoS は publish と subscription の最小値になる

次章では、接続を維持するためのキープアライブの仕組みを学ぶ。

図


Publisher
Broker
Subscriber
QoS 0: At most once
PUBLISH (QoS 0)
PUBLISH (QoS 0)
QoS 1: At least once
PUBLISH (QoS 1, id=1)
PUBLISH (QoS 1, id=7)
PUBACK (id=7)
PUBACK (id=1)
QoS 1: 再送シナリオ
PUBLISH (QoS 1, id=2)
PUBLISH (QoS 1, id=8)
PUBACK がタイムアウト
PUBLISH (QoS 1, id=8, DUP=1)
PUBACK (id=8)
PUBACK (id=2)
QoS 2: Exactly once (概念のみ)
PUBLISH (QoS 2, id=3)
PUBREC (id=3)
PUBREL (id=3)
PUBLISH (QoS 2, id=9)
PUBACK (id=9)
PUBCOMP (id=3)
Publisher
Broker
Subscriber

第9章 キープアライブ

PINGREQ/PINGRESPとタイムアウト

学習目標

- MQTT のキープアライブの目的と仕組みを理解する
- PINGREQ / PINGRESP パケットの構造を把握する
- サーバー側のタイムアウト検出（1.5倍ルール）を実装できる
- クライアント側の PINGREQ 送信タイミングを設計できる
- Zig 0.16 の `std.Io.Clock.Duration.sleep` API を活用できる

キープアライブとは

TCP 接続は、データの送受信がなくても維持される。しかし、ネットワーク障害やクライアントの異常終了を検知するには、定期的な死活確認が必要になる。

MQTT では CONNECT パケット内の `keep_alive` フィールド（秒単位、u16）でこの間隔を指定する。

```
keep_alive = 60 の場合:
- クライアントは 60秒以内に何らかのパケットを送信する義務がある
- サーバーは 90秒（1.5倍）以内にパケットを受信しなければ切断する
```

`keep_alive = 0` の場合、キープアライブは無効となる。

PINGREQ と PINGRESP

データを送信する必要がないときでも接続を維持するため、専用の軽量パケットが用意されている。

PINGREQ（クライアント -> サーバー）

```
バイト1: 0xC0（パケットタイプ=12, フラグ=0）
バイト2: 0x00（残りの長さ=0）
```

PINGRESP（サーバー -> クライアント）

```
バイト1: 0xD0（パケットタイプ=13, フラグ=0）
バイト2: 0x00（残りの長さ=0）
```

どちらも2バイトの固定長であり、ペイロードは持たない。

```
const PINGREQ_PACKET = [_]u8{ 0xC0, 0x00 };
const PINGRESP_PACKET = [_]u8{ 0xD0, 0x00 };
```

サーバー側の実装

タイムアウト検出

MQTT 仕様（3.1.2.10）では、サーバーはキープアライブ値の1.5倍の時間内にクライアントからパケットを受信しなければ、接続を切断しなければならないと定められている。

```
const Connection = struct {
    keep_alive_s: u16,
    last_packet_received: i64,
    // ...

    /// キープアライブのデッドラインを計算する（ナノ秒）
    fn keepAliveDeadlineNs(self: *const Connection) ?i64 {
        if (self.keep_alive_s == 0) return null; // 無効

        const timeout_ns: i64 = @as(i64, self.keep_alive_s) * std.time.ns_per_s * 3 /
2;
        return self.last_packet_received + timeout_ns;
    }

    /// パケット受信時に呼び出す
    fn updateLastReceived(self: *Connection) void {
        self.last_packet_received = std.time.nanoTimestamp();
    }

    /// タイムアウトしているかチェック
    fn isKeepAliveExpired(self: *const Connection) bool {
        const deadline = self.keepAliveDeadlineNs() orelse return false;
        return std.time.nanoTimestamp() > deadline;
    }
};
```

PINGREQ の処理

サーバーが PINGREQ を受信したら、即座に PINGRESP を返す。Zig 0.16 では writer に `io` パラメータが必要である。

```
fn handlePingreq(self: *Connection, io: *std.Io) !void {
    self.updateLastReceived();

    var write_buf: [4096]u8 = undefined;
    var writer = self.stream.writer(io, &write_buf);
    try writer.interface.writeAll(&PINGRESP_PACKET);
    try writer.interface.flush();
}
```

接続ループへの統合

メインの読み取りループでは、定期的にキープアライブのタイムアウトをチェックする。

```
fn connectionLoop(self: *Connection, io: *std.Io) !void {
    var read_buf: [4096]u8 = undefined;
    var reader = self.stream.reader(io, &read_buf);

    while (true) {
        // キープアライブチェック
        if (self.isKeepAliveExpired()) {
            std.log.info("キープアライブタイムアウト: {s}", .{self.client_id});
            return; // 接続を閉じる
        }

        // ソケットからデータを読む
        var header_buf: [1]u8 = undefined;
        reader.interface.readSliceAll(&header_buf) catch |err| {
            if (err == error.EndOfStream) return;
            return err;
        };

        self.updateLastReceived();
        const packet_type: u4 = @intCast(header_buf[0] >> 4);

        switch (packet_type) {
            3 => try self.handlePublish(io, header_buf[0]),
            8 => try self.handleSubscribe(io),
            12 => try self.handlePingreq(io),
            14 => return, // DISCONNECT
            else => return error.UnsupportedPacketType,
        }
    }
}
```

Zig 0.16 のスリープ API

Zig 0.16 では、スリープは `std.Io.Clock.Duration.sleep` を使用する。 これは Io ランタイムと統合されたスリープであり、 `io` パラメータが必要になる。

基本的なスリープ

```
// 1秒スリープ
std.Io.Clock.Duration.sleep(.{
    .clock = .awake,
    .raw = .fromMilliseconds(1000),
}, io);

// 500ミリ秒スリープ
std.Io.Clock.Duration.sleep(.{
    .clock = .awake,
    .raw = .fromMilliseconds(500),
}, io);
```

clock の種類

clock	説明
`.awake`	システムがスリープ中はカウントしない。一般用途に適する
`.boot`	システムのブート時刻からの経過。スリープ中もカウントする

キープアライブのタイマーには `.awake` が適切である。

タイムスタンプの取得

```
// 壁時計時間（エポックからのナノ秒）
const wall_ns = std.time.nanoTimestamp();

// ミリ秒単位の壁時計時間
const wall_ms = std.time.milliTimestamp();
```

クライアント側の実装

クライアントは、 `keep_alive` 期間内にパケットを送信しなかった場合、PINGREQ を送る。一般的には `keep_alive` の半分から3分の2の間隔で送信する。

```
const Client = struct {
    keep_alive_s: u16,
    last_packet_sent: i64,
    stream: anytype,

    /// 必要なら PINGREQ を送信する
    fn maybeSendPing(self: *Client, io: *std.Io) !void {
        if (self.keep_alive_s == 0) return;

        const now = std.time.nanoTimestamp();
        const interval_ns: i64 = @as(i64, self.keep_alive_s) * std.time.ns_per_s *
2 / 3;

        if (now - self.last_packet_sent > interval_ns) {
            var write_buf: [4096]u8 = undefined;
            var writer = self.stream.writer(io, &write_buf);
            try writer.interface.writeAll(&PINGREQ_PACKET);
            try writer.interface.flush();
            self.last_packet_sent = now;
        }
    }

    /// 任意のパケット送信時に呼ぶ
    fn markSent(self: *Client) void {
        self.last_packet_sent = std.time.nanoTimestamp();
    }
};
```

PINGREQ 定期送信ループ

クライアント側では別のタスクとして PING 送信ループを回すことが一般的である。 Zig 0.16 の sleep API を使った実装例:

```
fn pingLoop(client: *Client, io: *std.Io, should_stop: *std.atomic.Value(bool)) void {
    while (!should_stop.load(.acquire)) {
        // keep_alive の半分の間隔でチェック
        const interval_ms: u64 = @as(u64, client.keep_alive_s) * 500;
        std.Io.Clock.Duration.sleep(.{
            .clock = .awake,
            .raw = .fromMilliseconds(interval_ms),
        }, io);

        client.maybeSendPing(io) catch |err| {
```

```
            std.log.err("PING 送信エラー: {}", .{err});
            return;
        };
    }
}
```

PINGRESP の受信確認

クライアントは PINGREQ を送信した後、合理的な時間内に PINGRESP を受信することを期待する。受信できなかった場合、接続が失われたと判断してよい。

```
fn handlePingresp(self: *Client) void {
    // PINGRESP を正常に受信
    self.last_pingresp_received = std.time.nanoTimestamp();
    std.log.debug("PINGRESP 受信", .{});
}

fn isPingrespOverdue(self: *const Client) bool {
    if (self.last_pingreq_sent == 0) return false;
    const elapsed = std.time.nanoTimestamp() - self.last_pingreq_sent;
    // PINGREQ 送信後5秒以内に PINGRESP がなければ異常とみなす
    return elapsed > 5 * std.time.ns_per_s;
}
```

タイムアウト検出のパターン

Zig 0.16 では、タイムアウト検出に `std.atomic.Value(bool)` と sleep を組み合わせるパターンが有効である。

```
fn keepAliveMonitor(
    conn: *Connection,
    io: *std.Io,
    should_stop: *std.atomic.Value(bool),
) void {
    while (!should_stop.load(.acquire)) {
        std.Io.Clock.Duration.sleep(.{
            .clock = .awake,
            .raw = .fromMilliseconds(1000),
        }, io);

        if (conn.isKeepAliveExpired()) {
            std.log.info("キープアライブ超過を検出: {s}", .{conn.client_id});
            conn.forceClose();
            return;
        }
```

```
    }
}
```

タイムアウト値の設計指針

用途	推奨 keep_alive	理由
モバイルアプリ	30-60秒	バッテリー消費とのバランス
サーバー間通信	10-30秒	素早い障害検知
センサーデバイス	60-300秒	低電力・低帯域
LAN 内通信	10-20秒	高速な障害検知が可能

エッジケース

keep_alive = 0

キープアライブが無効の場合、サーバーはタイムアウト切断を行わない。TCP 自体のタイムアウトに依存する。

パケット送受信がキープアライブを兼ねる

PUBLISH や SUBSCRIBE など、通常のパケット送受信もキープアライブのカウンタをリセットする。PINGREQ は他にパケットを送信する必要がないときだけ使えばよい。

ネットワーク遅延の考慮

1.5倍ルールは、ネットワーク遅延を考慮したマージンである。クライアントが期限ぎりぎりに送信しても、ネットワーク遅延で到着が遅れる可能性がある。サーバー側では十分な余裕を持つべきである。

半開き接続の検出

TCP の半開き接続（一方が切断を認識していない状態）は、キープアライブなしでは検出できない。キープアライブを有効にすることで、このような異常を確実に検出できる。

まとめ

- キープアライブは TCP 接続の死活監視を行う MQTT の重要な仕組みである
- PINGREQ/PINGRESP は2バイトの軽量パケットで、オーバーヘッドは最小限
- サーバーは `keep_alive * 1.5` のタイムアウトで切断判定を行う
- Zig 0.16 では `std.Io.Clock.Duration.sleep` で Io ランタイム統合のスリープを行う
- `std.atomic.Value(bool)` と sleep の組み合わせでタイムアウト監視ループを実装する

次章では、ブローカー全体のアーキテクチャ設計を学ぶ。

第IV部 統合

コンポーネントを組み合わせて完成させます

第10章 ブローカーの全体設計

コンポーネントとスレッドモデル

学習目標

- MQTT ブローカーの主要コンポーネントと Zig 0.16 のアーキテクチャを理解する
- `std.Io.Threaded` による Io ランタイムの初期化を把握する
- `std.process.Init`（Juicy Main パターン）でエントリポイントを設計できる
- `std.Io.RwLock` と `std.Io.Mutex` の使い分けを理解する
- Thread-per-connection モデルと graceful shutdown を実装できる
- server.zig, connection.zig, session.zig の責務分担を理解する

アーキテクチャ概要

本プロジェクトのブローカーは以下の主要コンポーネントから構成される。

```
+-------------------------------------------------------+
|                    MqttBroker                         |
|                                                       |
|  +-----------+   +------------------+                |
|  |  Listener  |-->| ConnectionHandler | (per thread) |
|  +-----------+   +--------+---------+                |
|                           |                          |
|            +--------------+--------------+           |
|            v              v              v           |
|  +--------------+ +------------+ +-------------+     |
|  |SessionManager| |   Router   | | RetainStore |     |
|  | (RwLock)     | | (RwLock)   | | (Mutex)     |     |
|  +--------------+ +------------+ +-------------+     |
|                                                       |
|  Graceful shutdown: atomic.Value(bool) + Io.Event    |
+-------------------------------------------------------+
```

コンポーネント	ファイル	責務	同期プリミティブ
Listener	server.zig	TCP 接続の受け入れ	-
ConnectionHandler	connection.zig	個別クライアントのパケット処理	-
SessionManager	session.zig	セッションの作成・検索・破棄	`Io.RwLock`

コンポーネント	ファイル	責務	同期プリミティブ
Router	router.zig	トピックマッチングとメッセージ転送	`Io.RwLock`
RetainStore	retain.zig	Retained メッセージの保存と配信	`Io.Mutex`

Juicy Main パターン: std.process.Init

Zig 0.16 では、`main` 関数のシグネチャが `std.process.Init` を受け取る形に変わった。 これにより、アロケータと Io ランタイムが標準的に提供される。

```zig
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    // init.gpa: GeneralPurposeAllocator (プロセス標準のアロケータ)
    // init.io:  Io ランタイム (ネットワーク、スリープ等に必要)
    const allocator = init.gpa;
    const io = init.io;

    var broker = MqttBroker.init(allocator);
    defer broker.deinit();

    try broker.run(io, 1883);
}
```

init.gpa と init.io の役割

フィールド	型	用途
`init.gpa`	`std.mem.Allocator`	汎用メモリアロケータ
`init.io`	`*std.Io`	Io ランタイム（ネット、スリープ、同期プリミティブ）

Zig 0.16 の全ての I/O 操作は `io` パラメータを要求する。 ネットワーク接続、reader/writer の生成、Mutex のロック、スリープなど、あらゆる I/O 関連操作に `io` が必要である。

std.Io.Threaded: Io ランタイムの初期化

テストや特殊な環境では、手動で Io ランタイムを構築する必要がある。

```
var threaded = std.Io.Threaded.init(allocator, .{});
const io = threaded.io();
```

`std.Io.Threaded` はスレッドプール付きの Io ランタイムを提供する。 通常の `main` 関数では `init.io` を使えばよいが、テスト内でブローカーを起動する場合などに `Threaded` を直接使う。

server.zig: リスナー

サーバーはメインスレッドで TCP ソケットを listen し、新しい接続ごとにスレッドを生成する。 Zig 0.16 では、ネットワーク API に `io` パラメータが必要になる。

```
const MqttBroker = struct {
    allocator: std.mem.Allocator,
    session_manager: SessionManager,
    retain_store: RetainStore,
    should_stop: std.atomic.Value(bool),
    shutdown_event: std.Io.Event,

    pub fn init(allocator: std.mem.Allocator) MqttBroker {
        return .{
            .allocator = allocator,
            .session_manager = SessionManager.init(allocator),
            .retain_store = RetainStore.init(allocator),
            .should_stop = std.atomic.Value(bool).init(false),
            .shutdown_event = .unset,
        };
    }

    pub fn run(self: *MqttBroker, io: *std.Io, port: u16) !void {
        const address = std.Io.net.IpAddress.parse("0.0.0.0", port);
        var listener = try address.listen(io, .{
            .reuse_address = true,
        });
        defer listener.close(io);

        std.log.info("MQTT ブローカー起動: ポート {d}", .{port});

        while (!self.should_stop.load(.acquire)) {
            const conn = try listener.accept(io);
            // 接続ごとに新しいスレッドを生成
            const thread = try std.Thread.spawn(.{}, handleConnection, .{
                self, conn, io,
            });
            thread.detach();
        }
    }

    fn handleConnection(
        self: *MqttBroker,
```

```
        conn: anytype,
        io: *std.Io,
    ) void {
        var handler = ConnectionHandler.init(
            self.allocator,
            conn,
            &self.session_manager,
            &self.retain_store,
            io,
        );
        defer handler.deinit();
        handler.run() catch |err| {
            std.log.err("接続エラー: {}", .{err});
        };
    }
};
```

connection.zig: 接続ハンドラ

各スレッドで1つのクライアント接続を担当する。reader/writer の生成に `io` が必要である。

```
const ConnectionHandler = struct {
    allocator: std.mem.Allocator,
    stream: anytype,
    session_manager: *SessionManager,
    retain_store: *RetainStore,
    io: *std.Io,
    client_id: ?[]const u8 = null,
    keep_alive_s: u16 = 0,
    last_received: i64 = 0,

    pub fn run(self: *ConnectionHandler) !void {
        var read_buf: [4096]u8 = undefined;
        var write_buf: [4096]u8 = undefined;
        var reader = self.stream.reader(self.io, &read_buf);
        var writer = self.stream.writer(self.io, &write_buf);

        // 最初のパケットは CONNECT でなければならない
        try self.handleConnect(&reader, &writer);

        // メインループ: パケットを読み続ける
        while (true) {
            if (self.isKeepAliveExpired()) {
                std.log.info("キープアライブ切れ: {s}", .{
                    self.client_id orelse "unknown",
                });
                return;
            }

            var header_buf: [1]u8 = undefined;
```

```
            reader.interface.readSliceAll(&header_buf) catch |err| {
                if (err == error.EndOfStream) return;
                return err;
            };

            self.last_received = std.time.nanoTimestamp();
            const packet_type: u4 = @intCast(header_buf[0] >> 4);

            switch (packet_type) {
                3 => try self.handlePublish(header_buf[0], &reader, &writer),
                8 => try self.handleSubscribe(&reader, &writer),
                10 => try self.handleUnsubscribe(&reader, &writer),
                12 => {
                    // PINGREQ -> PINGRESP
                    try writer.interface.writeAll(&PINGRESP_PACKET);
                    try writer.interface.flush();
                },
                14 => return, // DISCONNECT
                else => return error.UnsupportedPacketType,
            }
        }
    }
};
```

同期プリミティブの使い分け

Zig 0.16 では、同期プリミティブにも `io` パラメータが必要になる。

std.Io.RwLock: 読み書きロック

SessionManager のように読み取りが頻繁で書き込みが稀な場合に使う。

```
// 定義
rwlock: std.Io.RwLock = .init,

// 読み取りロック（複数スレッドが同時に取得可能）
rwlock.lockSharedUncancelable(io);
defer rwlock.unlockShared(io);
// ... 読み取り操作 ...

// 書き込みロック（排他的: 他の読み取り/書き込みを全てブロック）
rwlock.lockUncancelable(io);
defer rwlock.unlock(io);
// ... 書き込み操作 ...
```

std.Io.Mutex: 排他ロック

RetainStore のように読み書きの比率が同程度、または書き込みが頻繁な場合に使う。

```
// 定義
mutex: std.Io.Mutex = .init,

// ロック
mutex.lockUncancelable(io);
defer mutex.unlock(io);
// ... 排他操作 ...
```

使い分けの指針

プリミティブ	適用場面	本プロジェクトでの利用
`Io.RwLock`	読み取り多、書き込み少	SessionManager（購読者検索が頻繁）
`Io.Mutex`	読み書き均等、または書き込み多	RetainStore、接続リスト

Graceful Shutdown パターン

`std.atomic.Value(bool)` と `std.Io.Event` を組み合わせて、安全なシャットダウンを実現する。

```
const MqttBroker = struct {
    should_stop: std.atomic.Value(bool),
    shutdown_event: std.Io.Event,
    // ...

    pub fn shutdown(self: *MqttBroker, io: *std.Io) void {
        // 停止フラグをセット
        self.should_stop.store(true, .release);
        // イベントをシグナルして待機中のスレッドを起こす
        self.shutdown_event.set(io);
    }
};
```

シャットダウンの流れ

```
1. 外部からシグナルを受信（Ctrl+C など）
2. should_stop.store(true, .release) で停止フラグをセット
3. shutdown_event.set(io) で待機中のスレッドを起こす
```

```
4. 各スレッドが should_stop をチェックして終了
5. メインスレッドで全リソースをクリーンアップ
```

Event の基本操作

```
// 初期化
var event: std.Io.Event = .unset;

// シグナル送信（待機中のスレッドを起こす）
event.set(io);

// 待機（シグナルされるまでブロック）
event.waitUncancelable(io);
```

Thread-per-connection モデル

設計思想

各クライアント接続に対して1つの OS スレッドを割り当てるモデルである。

```
メインスレッド（accept ループ）
   |
   +-- スレッド1: クライアントA の処理
   +-- スレッド2: クライアントB の処理
   +-- スレッド3: クライアントC の処理
   +-- ...
```

利点

- **実装がシンプル**: 各スレッドはブロッキング I/O を使える
- **デバッグしやすい**: スタックトレースが明確
- **Zig 0.16 との親和性**: `std.Thread.spawn` で直接生成

欠点

- **スケーラビリティ**: 数千接続で OS スレッドの限界に達する
- **メモリ使用量**: 各スレッドにスタック領域が必要（デフォルト数MB）

本プロジェクトは教育目的であるため、シンプルさを優先してこのモデルを採用している。

```
// 接続の受け入れとスレッド生成
const conn = try listener.accept(io);
const thread = try std.Thread.spawn(.{}, handleConnection, .{
    self, conn, io,
});
thread.detach();
```

共有状態のロック設計

ロック保持時間の最小化

ロック中にブロッキング I/O を行うと、他のスレッドが全てブロックされる。 必要なデータだけコピーして即座にアンロックするのが鉄則である。

```
// 悪い例: ロック中にネットワーク I/O を行う
self.rwlock.lockSharedUncancelable(io);
defer self.rwlock.unlockShared(io);
try self.stream.writer(io, &buf).interface.writeAll(data); // 全スレッドがブロック

// 良い例: 必要なデータだけコピーして即座にアンロック
var subscribers_buf: [64]SubscriberInfo = undefined;
const count = blk: {
    self.rwlock.lockSharedUncancelable(io);
    defer self.rwlock.unlockShared(io);
    break :blk self.collectSubscribers(topic, &subscribers_buf);
};
// ロック外で I/O を行う
for (subscribers_buf[0..count]) |sub| {
    try sub.connection.sendPublish(io, topic, payload);
}
```

コンポーネント間の相互作用

PUBLISH メッセージを受信してからサブスクライバーに配信するまでの流れ:

```
1. connection.zig: クライアントから PUBLISH パケットを受信・デコード
2. retain.zig:     Retain フラグが立っていれば RetainStore に保存 (Mutex)
3. session.zig:    SessionManager から購読者を検索 (RwLock 読み取り)
4. connection.zig: 各サブスクライバーの接続に PUBLISH を転送
```

```
fn handlePublish(
    self: *ConnectionHandler,
    io: *std.Io,
```

```
    first_byte: u8,
    reader: anytype,
    writer: anytype,
) !void {
    const packet = try self.decodePublish(first_byte, reader);

    // Retained メッセージの保存 (Mutex で保護)
    if (packet.retain) {
        try self.retain_store.store(io, packet.topic, packet.payload, packet.qos);
    }

    // サブスクライバーへの転送 (RwLock 読み取りで保護)
    var buf: [128]SubscriberInfo = undefined;
    const count = self.session_manager.getSubscribers(io, packet.topic, &buf);

    for (buf[0..count]) |sub| {
        const effective_qos = @min(packet.qos, sub.max_qos);
        sub.connection.sendPublish(io, packet.topic, packet.payload, effective_qos) ca
tch |err| {
            std.log.warn("転送エラー: {}", .{err});
        };
    }

    // QoS 1 なら PUBACK を返す
    if (packet.qos >= 1) {
        try self.sendPuback(writer, packet.packet_id);
    }
}
```

session.zig の概要

SessionManager は `Io.RwLock` で保護する。購読者検索（読み取り）が圧倒的に多いためである。

```
const SessionManager = struct {
    rwlock: std.Io.RwLock = .init,
    sessions: std.StringHashMap(*Session),
    allocator: std.mem.Allocator,

    /// 購読者の検索（読み取りロック）
    pub fn getSubscribers(
        self: *SessionManager,
        io: *std.Io,
        topic: []const u8,
        result_buf: []SubscriberInfo,
    ) usize {
        self.rwlock.lockSharedUncancelable(io);
        defer self.rwlock.unlockShared(io);
        // ... 検索処理 ...
    }
```

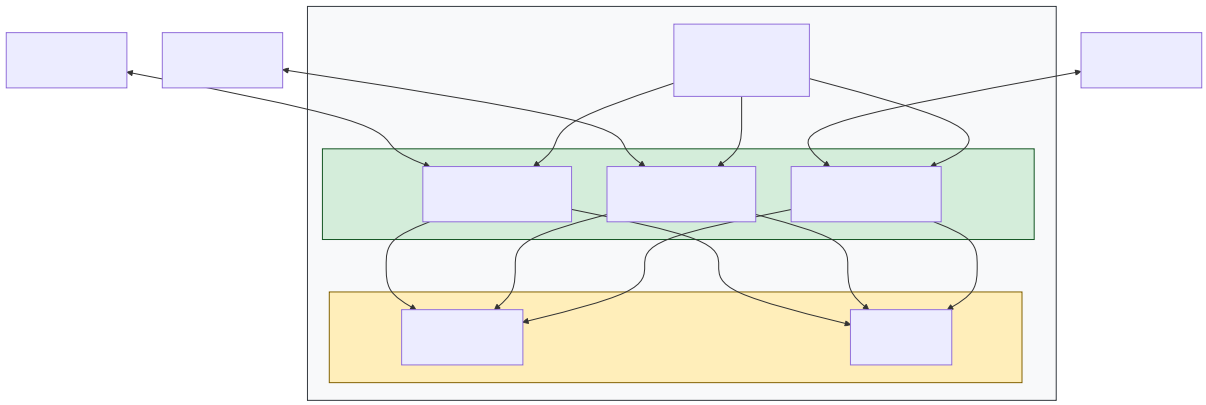
```zig
    /// セッションの作成（書き込みロック）
    pub fn getOrCreate(
        self: *SessionManager,
        io: *std.Io,
        client_id: []const u8,
        clean: bool,
    ) !*Session {
        self.rwlock.lockUncancelable(io);
        defer self.rwlock.unlock(io);
        // ... 作成処理 ...
    }
};
```

まとめ

- Zig 0.16 の `std.process.Init`（Juicy Main）で `gpa` と `io` を受け取る
- `std.Io.Threaded` はテスト環境などで手動 Io ランタイム構築に使う
- `std.Io.RwLock` は読み取り頻度が高い SessionManager に、`std.Io.Mutex` は RetainStore に使う
- 全ての I/O 操作（ネットワーク、ロック、スリープ）に `io` パラメータが必要
- `std.atomic.Value(bool)` + `std.Io.Event` で graceful shutdown を実現する
- ロック保持時間を最小化し、ロック外で I/O を行うことが並行性能の鍵である

次章では、セッション管理の詳細な実装を学ぶ。

図



第11章 セッション管理

Clean SessionとクライアントID

学習目標

- MQTT の Clean Session フラグの動作を理解する
- 永続的サブスクリプションの仕組みを把握する
- クライアントテイクオーバー（同一 client_id の再接続）を実装できる
- Zig 0.16 の `std.Io.RwLock` を用いた SessionManager を設計できる
- `errdefer` によるリソースリーク防止を理解する

セッションとは

MQTT におけるセッションとは、ブローカーがクライアントごとに保持する状態の集合である。

セッションに含まれる情報: - クライアントID - サブスクリプション一覧（トピックフィルタと QoS） - 未配信の QoS 1/2 メッセージ - 接続状態（オンライン/オフライン）

Clean Session フラグ

CONNECT パケットの Connect Flags 内にある Clean Session ビットは、セッションの扱いを決定する。

clean_session = true（クリーンセッション）

```
クライアント接続時:
1. 既存のセッションがあれば破棄する
2. 新しい空のセッションを作成する

クライアント切断時:
1. セッションを完全に破棄する
```

一時的な接続に適している。再接続時にサブスクリプションの再登録が必要。

clean_session = false（永続セッション）

```
クライアント接続時:
1. 既存のセッションがあればそれを再利用する
   -> CONNACK の Session Present フラグ = 1
2. なければ新規作成する
   -> CONNACK の Session Present フラグ = 0

クライアント切断時:
1. セッションを保持し続ける
2. サブスクリプションは維持される
```

再接続時にサブスクリプションが維持されるため、メッセージの取りこぼしを防げる。

RwLock ベースの SessionManager

SessionManager はブローカー内で最もアクセス頻度が高い共有状態である。 メッセージルーティング時に毎回購読者を検索するため、読み取りが圧倒的に多い。 Zig 0.16 の `std.Io.RwLock` を使うことで、読み取り操作を並行に実行できる。

データ構造

```zig
const Session = struct {
    client_id: []const u8,
    subscriptions: std.StringHashMap(u2), // topic_filter -> max_qos
    connected: bool,
    connection: ?*ConnectionHandler,
    clean_session: bool,
    will_message: ?WillMessage,

    fn deinit(self: *Session, allocator: std.mem.Allocator) void {
        // サブスクリプションのキーを解放
        var iter = self.subscriptions.keyIterator();
        while (iter.next()) |key| {
            allocator.free(key.*);
        }
        self.subscriptions.deinit();
        allocator.free(self.client_id);
    }
};

const SessionManager = struct {
    rwlock: std.Io.RwLock = .init,
    sessions: std.StringHashMap(*Session),
    allocator: std.mem.Allocator,
```

```
    pub fn init(allocator: std.mem.Allocator) SessionManager {
        return .{
            .sessions = std.StringHashMap(*Session).init(allocator),
            .allocator = allocator,
        };
    }

    pub fn deinit(self: *SessionManager, io: *std.Io) void {
        self.rwlock.lockUncancelable(io);
        defer self.rwlock.unlock(io);

        var iter = self.sessions.valueIterator();
        while (iter.next()) |session_ptr| {
            session_ptr.*.deinit(self.allocator);
            self.allocator.destroy(session_ptr.*);
        }
        self.sessions.deinit();
    }
};
```

RwLock の読み取りと書き込みの使い分け

```
// 読み取りロック: 複数スレッドが同時に取得可能
// -> 購読者検索、セッション情報参照
rwlock.lockSharedUncancelable(io);
defer rwlock.unlockShared(io);

// 書き込みロック: 排他的に取得
// -> セッション作成、削除、サブスクリプション変更
rwlock.lockUncancelable(io);
defer rwlock.unlock(io);
```

getOrCreate: セッションの取得または作成

```
const GetOrCreateResult = struct {
    session: *Session,
    session_existed: bool,
};

pub fn getOrCreate(
    self: *SessionManager,
    io: *std.Io,
    client_id: []const u8,
    clean_session: bool,
) !GetOrCreateResult {
    // 書き込みロック（作成の可能性があるため）
    self.rwlock.lockUncancelable(io);
    defer self.rwlock.unlock(io);
```

```zig
    if (self.sessions.get(client_id)) |existing| {
        if (clean_session) {
            // 既存セッションをクリア
            existing.clearSubscriptions(self.allocator);
        }
        return .{ .session = existing, .session_existed = true };
    }

    // 新しいセッションを作成
    const session = try self.allocator.create(Session);
    errdefer self.allocator.destroy(session);

    const owned_id = try self.allocator.dupe(u8, client_id);
    errdefer self.allocator.free(owned_id);

    session.* = .{
        .client_id = owned_id,
        .subscriptions = std.StringHashMap(u2).init(self.allocator),
        .connected = false,
        .connection = null,
        .clean_session = clean_session,
        .will_message = null,
    };

    try self.sessions.put(owned_id, session);
    return .{ .session = session, .session_existed = false };
}
```

CONNECT 処理での利用

```zig
fn handleConnect(self: *ConnectionHandler, io: *std.Io) !void {
    const connect = try self.decodeConnect();

    const result = try self.session_manager.getOrCreate(
        io,
        connect.client_id,
        connect.clean_session,
    );

    // CONNACK で Session Present を通知
    try self.sendConnack(io, .{
        .session_present = result.session_existed and !connect.clean_session,
        .return_code = .accepted,
    });

    self.client_id = connect.client_id;
}
```

購読者検索: 読み取りロック

メッセージルーティング時に呼ばれる。読み取りロックにより複数スレッドが同時に検索できる。

```
pub fn getSubscribers(
    self: *SessionManager,
    io: *std.Io,
    topic: []const u8,
    result_buf: []SubscriberInfo,
) usize {
    // 読み取りロック: 複数スレッドが並行して検索可能
    self.rwlock.lockSharedUncancelable(io);
    defer self.rwlock.unlockShared(io);

    var count: usize = 0;
    var iter = self.sessions.valueIterator();
    while (iter.next()) |session| {
        if (!session.*.connected) continue;
        if (session.*.subscriptions.get(topic)) |qos| {
            if (count < result_buf.len) {
                result_buf[count] = .{
                    .connection = session.*.connection.?,
                    .max_qos = qos,
                };
                count += 1;
            }
        }
    }
    return count;
}
```

パフォーマンス効果

読み取りロックの利点は、複数のメッセージルーティングが同時に実行できることである。

```
スレッド1: PUBLISH 受信 -> getSubscribers (読み取りロック) -> 転送
スレッド2: PUBLISH 受信 -> getSubscribers (読み取りロック) -> 転送
スレッド3: PUBLISH 受信 -> getSubscribers (読み取りロック) -> 転送
   ^^ これらが全て同時に実行される

スレッド4: CONNECT 受信 -> getOrCreate (書き込みロック) -> 排他的
   ^^ 書き込み中はスレッド1-3もブロックされる
```

errdefer によるリソースリーク防止

`errdefer` は、スコープがエラーで抜ける場合のみ実行されるクリーンアップである。 リソースを段階的に確保する際、途中でエラーが発生してもリークしない。

```
pub fn createSession(
    self: *SessionManager,
    client_id: []const u8,
) !*Session {
    // ステップ1: Session 構造体を確保
    const session = try self.allocator.create(Session);
    errdefer self.allocator.destroy(session);
    // ステップ2以降でエラーなら session を解放

    // ステップ2: client_id をコピー
    const owned_id = try self.allocator.dupe(u8, client_id);
    errdefer self.allocator.free(owned_id);
    // ステップ3以降でエラーなら owned_id を解放

    // ステップ3: HashMap に登録
    try self.sessions.put(owned_id, session);
    // ここでエラーなら owned_id と session の両方が解放される

    return session;
    // 成功時は errdefer は実行されない
}
```

`errdefer` は Zig のエラーハンドリングの核心的な機能であり、手動メモリ管理でもリソースリークを確実に防げる。

クライアントテイクオーバー

同じ client_id で新しい接続が来た場合、既存の接続を切断して新しい接続に切り替える。 これは MQTT 仕様で定められた動作である。

```
pub fn connectClient(
    self: *SessionManager,
    io: *std.Io,
    client_id: []const u8,
    new_conn: *ConnectionHandler,
    clean_session: bool,
) !*Session {
    // 書き込みロック
    self.rwlock.lockUncancelable(io);
    defer self.rwlock.unlock(io);
```

```
    if (self.sessions.get(client_id)) |existing| {
        if (existing.connected) {
            // 既存の接続を強制切断（テイクオーバー）
            if (existing.connection) |old_conn| {
                std.log.info("クライアントテイクオーバー: {s}", .{client_id});
                old_conn.forceDisconnect();
            }
        }
        existing.connection = new_conn;
        existing.connected = true;

        if (clean_session) {
            existing.clearSubscriptions(self.allocator);
            existing.clean_session = true;
        }
        return existing;
    }

    // 新規セッション作成
    return try self.createNewSession(client_id, new_conn, clean_session);
}
```

テイクオーバーの流れ

```
1. クライアントA が client_id="sensor-01" で接続中
2. クライアントB が同じ client_id="sensor-01" で接続を試みる
3. ブローカーはクライアントA の接続を強制切断
4. クライアントB のセッションとして登録
5. clean_session に応じてサブスクリプションを引き継ぐ or クリア
```

サブスクリプションの管理

サブスクリプションの追加・削除は書き込みロックが必要である。

```
pub fn addSubscription(
    self: *SessionManager,
    io: *std.Io,
    client_id: []const u8,
    topic_filter: []const u8,
    max_qos: u2,
) !void {
    // 書き込みロック
    self.rwlock.lockUncancelable(io);
    defer self.rwlock.unlock(io);

    const session = self.sessions.get(client_id) orelse return error.SessionNotFound;
```

```
    // トピックフィルタを複製して保存
    const owned_filter = try self.allocator.dupe(u8, topic_filter);
    errdefer self.allocator.free(owned_filter);

    // 既存のエントリがあれば古いキーを解放
    if (session.subscriptions.fetchRemove(topic_filter)) |removed| {
        self.allocator.free(removed.key);
    }

    try session.subscriptions.put(owned_filter, max_qos);
}

pub fn removeSubscription(
    self: *SessionManager,
    io: *std.Io,
    client_id: []const u8,
    topic_filter: []const u8,
) !void {
    // 書き込みロック
    self.rwlock.lockUncancelable(io);
    defer self.rwlock.unlock(io);

    const session = self.sessions.get(client_id) orelse return error.SessionNotFound;

    if (session.subscriptions.fetchRemove(topic_filter)) |removed| {
        self.allocator.free(removed.key);
    }
}
```

切断時のセッション処理

```
pub fn disconnectClient(
    self: *SessionManager,
    io: *std.Io,
    client_id: []const u8,
    graceful: bool,
) void {
    // 書き込みロック
    self.rwlock.lockUncancelable(io);
    defer self.rwlock.unlock(io);

    const session = self.sessions.get(client_id) orelse return;

    session.connected = false;
    session.connection = null;

    if (session.clean_session) {
        // クリーンセッション: 完全に破棄
        _ = self.sessions.remove(client_id);
        session.deinit(self.allocator);
```

```
        self.allocator.destroy(session);
    }
    // 永続セッション: サブスクリプションを保持して残す

    if (!graceful) {
        // 非正常切断: Will メッセージを発行（第12章で詳述）
    }
}
```

StringHashMap の活用

Zig の `std.StringHashMap` は `[]const u8` をキーとする HashMap である。 文字列のハッシュと比較を自動で行うため、MQTT のトピックフィルタやクライアントIDの管理に最適である。

```
// 初期化
var map = std.StringHashMap(u32).init(allocator);
defer map.deinit();

// 追加
try map.put("key", 42);

// 取得
if (map.get("key")) |value| {
    std.log.info("値: {d}", .{value});
}

// 削除（キーと値のペアを返す）
if (map.fetchRemove("key")) |entry| {
    // entry.key, entry.value が取得可能
}
```

注意: `put` に渡すキーのライフタイムは、HashMap のエントリが存在する間有効でなければならない。スタック上の一時文字列をキーにすると、ダングリングポインタになる。 必ず `allocator.dupe` でコピーしてからキーとして使うこと。

まとめ

- Clean Session フラグにより、一時的/永続的セッションを使い分ける
- `std.Io.RwLock` で読み取りと書き込みを分離し、並行性能を向上させる
- `lockSharedUncancelable(io)` / `unlockShared(io)` で読み取りロック
- `lockUncancelable(io)` / `unlock(io)` で書き込みロック
- `errdefer` を活用して、段階的なリソース確保でもリークを防ぐ
- クライアントテイクオーバーで同一 client_id の排他制御を行う

- `StringHashMap` + `allocator.dupe` でトピックフィルタを安全に管理する

次章では、Retained メッセージと Will メッセージについて学ぶ。

第12章 Retained/Willメッセージ

保持メッセージと遺言

学習目標

- Retained メッセージの保存・配信の仕組みを理解する
- 空ペイロードによる Retained メッセージの削除を実装できる
- Will メッセージの目的と非正常切断時の動作を理解する
- Zig 0.16 の `std.Io.Mutex` による RetainStore の保護を実装できる
- `allocator.dupe` によるメモリ所有権の管理を活用できる

Retained メッセージとは

Retained メッセージは、トピックごとにブローカーが保持する「最後のメッセージ」である。新しいサブスクライバーが接続した際に、即座にそのトピックの最新値を受け取れる。

ユースケース

- 温度センサーの最新値: サブスクライバーは接続直後に現在の温度を知りたい
- デバイスのオンライン状態: `devices/sensor-01/status` に "online" を retain で保持

動作の流れ

```
1. Publisher が PUBLISH (retain=1, topic="room/temp", payload="25.3") を送信
2. Broker が RetainStore にこのメッセージを保存
3. 通常のサブスクライバーにも PUBLISH を転送 (retain=0 で)
4. 後から Subscriber が "room/temp" をサブスクライブ
5. Broker は即座に保存済みの retained メッセージを PUBLISH (retain=1) で送信
```

RetainStore の実装

RetainStore は `std.Io.Mutex` で保護する。読み取りと書き込みの頻度が同程度であり、RwLock より Mutex の方が適切である。

基本構造

```
const RetainedMessage = struct {
    topic: []const u8,
    payload: []const u8,
    qos: u2,
};

const RetainStore = struct {
    mutex: std.Io.Mutex = .init,
    messages: std.StringHashMap(RetainedMessage),
    allocator: std.mem.Allocator,

    pub fn init(allocator: std.mem.Allocator) RetainStore {
        return .{
            .messages = std.StringHashMap(RetainedMessage).init(allocator),
            .allocator = allocator,
        };
    }

    pub fn deinit(self: *RetainStore, io: *std.Io) void {
        self.mutex.lockUncancelable(io);
        defer self.mutex.unlock(io);

        var iter = self.messages.iterator();
        while (iter.next()) |entry| {
            self.allocator.free(entry.key_ptr.*);
            self.allocator.free(entry.value_ptr.payload);
        }
        self.messages.deinit();
    }
};
```

メッセージの保存

```
pub fn store(
    self: *RetainStore,
    io: *std.Io,
    topic: []const u8,
    payload: []const u8,
    qos: u2,
) !void {
    self.mutex.lockUncancelable(io);
    defer self.mutex.unlock(io);

    // 空ペイロード = retained メッセージの削除
    if (payload.len == 0) {
        if (self.messages.fetchRemove(topic)) |removed| {
            self.allocator.free(removed.key);
            self.allocator.free(removed.value.payload);
```

```
        }
        return;
    }

    // 既存のメッセージがあれば古いデータを解放
    if (self.messages.fetchRemove(topic)) |removed| {
        self.allocator.free(removed.key);
        self.allocator.free(removed.value.payload);
    }

    // 新しいデータを複製して保存
    const owned_topic = try self.allocator.dupe(u8, topic);
    errdefer self.allocator.free(owned_topic);

    const owned_payload = try self.allocator.dupe(u8, payload);
    errdefer self.allocator.free(owned_payload);

    try self.messages.put(owned_topic, .{
        .topic = owned_topic,
        .payload = owned_payload,
        .qos = qos,
    });
}
```

メッセージの取得

新しいサブスクリプション登録時に、マッチする Retained メッセージを返す。

```
pub fn getMatching(
    self: *RetainStore,
    io: *std.Io,
    topic_filter: []const u8,
    buf: []RetainedMessage,
) usize {
    self.mutex.lockUncancelable(io);
    defer self.mutex.unlock(io);

    var count: usize = 0;
    var iter = self.messages.iterator();

    while (iter.next()) |entry| {
        if (topicMatchesFilter(entry.key_ptr.*, topic_filter)) {
            if (count < buf.len) {
                buf[count] = entry.value_ptr.*;
                count += 1;
            }
        }
    }
    return count;
}
```

空ペイロードによる削除

MQTT 仕様では、retain フラグが立った PUBLISH で空のペイロードを送ると、そのトピックの Retained メッセージが削除される。

```
// クライアント側: retained メッセージを削除する
try client.publish(io, "room/temp", "", .{ .retain = true });
```

これはセンサーがオフラインになった際に、古い値を残さないために使われる。

削除の内部処理

```
// store() 内の該当部分
if (payload.len == 0) {
    if (self.messages.fetchRemove(topic)) |removed| {
        // 所有権を持つデータを確実に解放
        self.allocator.free(removed.key);
        self.allocator.free(removed.value.payload);
    }
    return;
}
```

`fetchRemove` は削除と同時にキーと値を返す。所有権を持つメモリを確実に解放するために、この戻り値を利用する。

サブスクリプション時の Retained メッセージ配信

```
fn handleSubscribe(self: *ConnectionHandler, io: *std.Io) !void {
    const sub = try self.decodeSubscribe();

    for (sub.topic_filters) |filter| {
        // サブスクリプションを登録
        try self.session_manager.addSubscription(
            io,
            self.client_id.?,
            filter.topic,
            filter.max_qos,
        );

        // マッチする retained メッセージを配信
        var retained_buf: [32]RetainedMessage = undefined;
        const count = self.retain_store.getMatching(
            io,
```

```
            filter.topic,
            &retained_buf,
        );

        for (retained_buf[0..count]) |msg| {
            const effective_qos = @min(msg.qos, filter.max_qos);
            try self.sendPublish(io, msg.topic, msg.payload, effective_qos, .{
                .retain = true, // retained メッセージであることを示す
            });
        }
    }

    try self.sendSuback(io, sub.packet_id, sub.topic_filters);
}
```

Will メッセージとは

Will メッセージは、クライアントが非正常切断した場合にブローカーが自動的にパブリッシュするメッセージである。CONNECT パケット内で事前に設定する。

ユースケース

- デバイスの死活監視: `devices/sensor-01/status` に "offline" を Will メッセージとして設定
- アラート通知: 重要なクライアントの切断を他のクライアントに通知

CONNECT パケットでの指定

```
Connect Flags:
  bit 2: Will Flag = 1
  bit 3-4: Will QoS = 0-2
  bit 5: Will Retain = 0 or 1

ペイロード (Will Flag=1 の場合) :
  - Will Topic (UTF-8 文字列)
  - Will Message (バイナリデータ)
```

```
const WillMessage = struct {
    topic: []const u8,
    payload: []const u8,
    qos: u2,
    retain: bool,
};

fn decodeWill(connect_flags: u8, reader: anytype) !?WillMessage {
    if (connect_flags & 0x04 == 0) return null; // Will Flag なし
```

```
    const will_qos: u2 = @intCast((connect_flags >> 3) & 0x03);
    const will_retain = (connect_flags & 0x20) != 0;
    const will_topic = try readUtf8String(reader);
    const will_payload = try readBinaryData(reader);

    return .{
        .topic = will_topic,
        .payload = will_payload,
        .qos = will_qos,
        .retain = will_retain,
    };
}
```

Will メッセージの発行条件

Will メッセージは以下の場合に発行される:

1. **ネットワーク障害**: TCP 接続が異常切断された
2. **キープアライブ超過**: 1.5倍のタイムアウト以内にパケットが届かなかった
3. **プロトコルエラー**: 不正なパケットを受信した

Will メッセージが発行されない場合:

1. **正常な DISCONNECT**: クライアントが DISCONNECT パケットを送信して切断した

```
fn connectionClosed(
    self: *ConnectionHandler,
    io: *std.Io,
    reason: DisconnectReason,
) void {
    switch (reason) {
        .graceful_disconnect => {
            // DISCONNECT パケットを受信: Will メッセージは発行しない
            self.session_manager.disconnectClient(io, self.client_id.?, true);
        },
        .network_error, .keepalive_timeout, .protocol_error => {
            // 非正常切断: Will メッセージを発行
            if (self.session) |session| {
                if (session.will_message) |will| {
                    self.publishWill(io, will);
                }
            }
            self.session_manager.disconnectClient(io, self.client_id.?, false);
        },
    }
}
```

Will メッセージの転送

```
fn publishWill(self: *ConnectionHandler, io: *std.Io, will: WillMessage) void {
    // Retained フラグが立っていれば保存
    if (will.retain) {
        self.retain_store.store(io, will.topic, will.payload, will.qos) catch {};
    }

    // サブスクライバーに転送
    var buf: [128]SubscriberInfo = undefined;
    const count = self.session_manager.getSubscribers(io, will.topic, &buf);
    for (buf[0..count]) |sub| {
        const effective_qos = @min(will.qos, sub.max_qos);
        sub.connection.sendPublish(
            io,
            will.topic,
            will.payload,
            effective_qos,
        ) catch {};
    }
}
```

メモリ所有権と allocator.dupe

Zig では GC がないため、メモリの所有権を明確にする必要がある。

問題: ダングリングポインタ

```
// 悪い例: パケットバッファの一部を直接保存
fn badStore(self: *RetainStore, io: *std.Io, packet: *const Packet) !void {
    self.mutex.lockUncancelable(io);
    defer self.mutex.unlock(io);

    // packet.payload はパケットバッファの一部を指している
    // パケット処理が終わるとバッファは解放される
    try self.messages.put(packet.topic, .{
        .payload = packet.payload, // ダングリングポインタ!
    });
}
```

解決: allocator.dupe でコピー

```
// 正しい例: データを複製して所有権を RetainStore に移す
fn goodStore(
```

```
    self: *RetainStore,
    io: *std.Io,
    topic: []const u8,
    payload: []const u8,
) !void {
    self.mutex.lockUncancelable(io);
    defer self.mutex.unlock(io);

    const owned_payload = try self.allocator.dupe(u8, payload);
    errdefer self.allocator.free(owned_payload);

    const owned_topic = try self.allocator.dupe(u8, topic);
    errdefer self.allocator.free(owned_topic);

    try self.messages.put(owned_topic, .{
        .topic = owned_topic,
        .payload = owned_payload,
    });
    // 所有権は RetainStore に移った
    // 解放は RetainStore.deinit() または上書き時に行う
}
```

`allocator.dupe(u8, slice)` は、スライスの内容を新しく確保したメモリにコピーする。戻り値のスライスはアロケータが所有するため、不要になったら `allocator.free` で解放する。

所有権の注意点

パターン	安全性	説明
直接参照	危険	元のバッファが解放されるとダングリング
`allocator.dupe`	安全	独立したメモリに複製
`errdefer` + `dupe`	安全	エラー時も確実に解放

Retained + Will の組み合わせ

Will メッセージを retain 付きで設定すると、クライアントの死活状態を永続的に通知できる。

```
1. クライアント接続時:
   - Will: topic="status/sensor-01", payload="offline", retain=true
   - 接続後に publish: topic="status/sensor-01", payload="online", retain=true

2. 正常動作中:
   - "status/sensor-01" の retained 値は "online"

3. 非正常切断時:
```

```
    - Will メッセージが発行され、retained 値が "offline" に更新される
    - 新しいサブスクライバーは "offline" を受け取る
```

この パターンはデバイス管理において非常に強力である。サブスクライバーは接続するだけで、全デバイスの現在のステータスを即座に取得できる。

Mutex と RwLock の選択基準（再確認）

RetainStore には `Io.Mutex` を使い、SessionManager には `Io.RwLock` を使う。 その理由を改めて整理する。

```
// RetainStore: Mutex を使用
// 理由: store（書き込み）と getMatching（読み取り）の頻度が同程度
// Retain メッセージはそもそも数が少なく、ロック競合が低い
mutex: std.Io.Mutex = .init,

// SessionManager: RwLock を使用
// 理由: getSubscribers（読み取り）が圧倒的に多い
// 全 PUBLISH メッセージで読み取りが発生するため、読み取り並行性が重要
rwlock: std.Io.RwLock = .init,
```

まとめ

- Retained メッセージはトピックごとの「最新値」をブローカーに保持する仕組み
- 空ペイロードで retained メッセージを削除できる
- Will メッセージは非正常切断時のみ発行される遺言メッセージ
- RetainStore は `std.Io.Mutex` で保護する（`lockUncancelable(io)` / `unlock(io)`）
- `allocator.dupe` でデータをコピーし、所有権を明確に管理する
- `errdefer` と組み合わせることで、エラー時のリソースリークを防ぐ
- Retained + Will の組み合わせでデバイスの死活監視が実現できる

次章では、複数クライアントを統合した動作デモを構築する。

第V部 デモとテスト

統合デモとテスト手法を学びます

第13章 複数クライアントの統合

1プロセスデモとメッセージルーティング

学習目標

- ブローカー・パブリッシャー・サブスクライバーを統合して動作させる
- 各実行バイナリで Juicy Main パターン（`std.process.Init`）を使用する
- メッセージルーティングの End-to-End フローを理解する
- 複数ターミナルでのデモ実行手順を習得する
- mosquitto クライアントとの相互運用テストを実施できる

プロジェクト構成

本プロジェクトでは3つの実行可能バイナリを生成する。

```
mqtt_zig/
+-- build.zig
+-- src/
    +-- broker/
    |   +-- main.zig          # ブローカーのエントリポイント
    |   +-- server.zig        # TCP リスナー
    |   +-- connection.zig    # 接続ハンドラ
    |   +-- session.zig       # セッション管理
    |   +-- retain.zig        # Retained メッセージ
    |   +-- topic.zig         # トピックマッチング
    +-- publisher/
    |   +-- main.zig          # パブリッシャーのエントリポイント
    +-- subscriber/
        +-- main.zig          # サブスクライバーのエントリポイント
```

build.zig の構成

```zig
const std = @import("std");

pub fn build(b: *std.Build) void {
    const target = b.standardTargetOptions(.{});
    const optimize = b.standardOptimizeOption(.{});

    // ブローカー
    const broker = b.addExecutable(.{
```

```
        .name = "mqtt-broker",
        .root_source_file = b.path("src/broker/main.zig"),
        .target = target,
        .optimize = optimize,
    });
    b.installArtifact(broker);

    // パブリッシャー
    const publisher = b.addExecutable(.{
        .name = "mqtt-pub",
        .root_source_file = b.path("src/publisher/main.zig"),
        .target = target,
        .optimize = optimize,
    });
    b.installArtifact(publisher);

    // サブスクライバー
    const subscriber = b.addExecutable(.{
        .name = "mqtt-sub",
        .root_source_file = b.path("src/subscriber/main.zig"),
        .target = target,
        .optimize = optimize,
    });
    b.installArtifact(subscriber);

    // 個別ビルドステップ
    const broker_step = b.step("broker", "ブローカーをビルド");
    broker_step.dependOn(&broker.step);

    const pub_step = b.step("pub", "パブリッシャーをビルド");
    pub_step.dependOn(&publisher.step);

    const sub_step = b.step("sub", "サブスクライバーをビルド");
    sub_step.dependOn(&subscriber.step);
}
```

Juicy Main: 全バイナリ共通のエントリポイント

Zig 0.16 では、3つの実行バイナリ全てで `std.process.Init` を受け取る Juicy Main パターンを使う。

ブローカーの main.zig

```
const std = @import("std");
const MqttBroker = @import("server.zig").MqttBroker;

pub fn main(init: std.process.Init) !void {
    const allocator = init.gpa;
    const io = init.io;
```

```
    var broker = MqttBroker.init(allocator);
    defer broker.deinit(io);

    std.log.info("MQTT ブローカー起動: ポート 1883", .{});
    try broker.run(io, 1883);
}
```

パブリッシャーの main.zig

```
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    const allocator = init.gpa;
    const io = init.io;

    // ブローカーに接続
    const address = std.Io.net.IpAddress.parse("127.0.0.1", 1883);
    var stream = try address.connect(io, .{});
    defer stream.close(io);

    var read_buf: [4096]u8 = undefined;
    var write_buf: [4096]u8 = undefined;
    var reader = stream.reader(io, &read_buf);
    var writer = stream.writer(io, &write_buf);

    // CONNECT 送信
    try sendConnect(&writer, "mqtt-pub-client", .{
        .keep_alive = 30,
        .clean_session = true,
    });
    try writer.interface.flush();

    // CONNACK 受信
    const connack = try receiveConnack(&reader);
    if (connack.return_code != .accepted) {
        std.log.err("接続拒否: {}", .{connack.return_code});
        return;
    }
    std.log.info("CONNACK 受信: session_present={}", .{connack.session_present});

    // PUBLISH 送信
    // コマンドライン引数からトピックとメッセージを取得する想定
    const topic = "sensor/temp";
    const message = "25.3";

    try sendPublish(&writer, topic, message, .{
        .qos = 0,
        .retain = false,
    });
    try writer.interface.flush();
    std.log.info("PUBLISH 送信: topic={s}, payload={s}", .{ topic, message });
```

```
    // DISCONNECT 送信
    try writer.interface.writeAll(&[_]u8{ 0xE0, 0x00 });
    try writer.interface.flush();

    _ = allocator; // 将来的にコマンドライン解析で使用
}
```

サブスクライバーの main.zig

```
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    const allocator = init.gpa;
    const io = init.io;

    // ブローカーに接続
    const address = std.Io.net.IpAddress.parse("127.0.0.1", 1883);
    var stream = try address.connect(io, .{});
    defer stream.close(io);

    var read_buf: [4096]u8 = undefined;
    var write_buf: [4096]u8 = undefined;
    var reader = stream.reader(io, &read_buf);
    var writer = stream.writer(io, &write_buf);

    // CONNECT
    try sendConnect(&writer, "mqtt-sub-client", .{
        .keep_alive = 60,
        .clean_session = true,
    });
    try writer.interface.flush();
    _ = try receiveConnack(&reader);

    // SUBSCRIBE
    const topic_filter = "sensor/#";
    try sendSubscribe(&writer, topic_filter, 1);
    try writer.interface.flush();
    _ = try receiveSuback(&reader);
    std.log.info("サブスクライブ: {s}", .{topic_filter});

    std.log.info("メッセージ待受中... (Ctrl+C で終了)", .{});

    // メッセージ受信ループ
    while (true) {
        var header: [1]u8 = undefined;
        reader.interface.readSliceAll(&header) catch |err| {
            if (err == error.EndOfStream) return;
            return err;
        };

        const packet_type: u4 = @intCast(header[0] >> 4);
```

```
        switch (packet_type) {
            3 => { // PUBLISH
                const pub_msg = try decodePublish(header[0], &reader);
                std.log.info(
                    "受信: topic={s}, payload={s}, qos={d}",
                    .{ pub_msg.topic, pub_msg.payload, pub_msg.qos },
                );

                // QoS 1 なら PUBACK を返す
                if (pub_msg.qos >= 1) {
                    try sendPuback(&writer, pub_msg.packet_id);
                    try writer.interface.flush();
                }
            },
            13 => {}, // PINGRESP: 無視
            else => {
                std.log.warn("予期しないパケット: type={d}", .{packet_type});
            },
        }
    }

    _ = allocator;
}
```

デモ実行手順

ステップ1: ビルド

```
# 全バイナリをビルド
zig build

# または個別にビルド
zig build broker
zig build pub
zig build sub
```

ステップ2: ブローカーを起動（ターミナル1）

```
$ ./zig-out/bin/mqtt-broker
info: MQTT ブローカー起動: ポート 1883
info: クライアント接続待ち...
```

ステップ3: サブスクライバーを起動（ターミナル2）

```
$ ./zig-out/bin/mqtt-sub
info: CONNACK 受信: session_present=false
info: サブスクライブ: sensor/#
info: メッセージ待受中... (Ctrl+C で終了)
```

ステップ4: パブリッシャーでメッセージ送信（ターミナル3）

```
$ ./zig-out/bin/mqtt-pub
info: CONNACK 受信: session_present=false
info: PUBLISH 送信: topic=sensor/temp, payload=25.3
```

ステップ5: サブスクライバー側で受信を確認（ターミナル2）

```
info: 受信: topic=sensor/temp, payload=25.3, qos=0
```

メッセージルーティングの End-to-End フロー

```
Publisher                    Broker                       Subscriber
   |                           |                               |
   |-- CONNECT --------------->|                               |
   |<-- CONNACK ---------------|                               |
   |                           |                               |
   |                           |<------------- CONNECT -------|
   |                           |------------- CONNACK -------->|
   |                           |<---------- SUBSCRIBE --------|
   |                           |------------ SUBACK --------->|
   |                           |                               |
   |-- PUBLISH --------------->|                               |
   |                           | (トピックマッチング)           |
   |                           |-- PUBLISH ------------------->|
   |                           |                               |
   |-- DISCONNECT ------------>|                               |
```

ブローカー内部のルーティング処理

```
fn routeMessage(
    self: *Broker,
    io: *std.Io,
```

```
    source_client_id: []const u8,
    topic: []const u8,
    payload: []const u8,
    qos: u2,
    retain: bool,
) !void {
    // 1. Retained メッセージの処理 (Mutex)
    if (retain) {
        try self.retain_store.store(io, topic, payload, qos);
    }

    // 2. マッチするサブスクライバーを検索 (RwLock 読み取り)
    var subscribers_buf: [256]SubscriberInfo = undefined;
    const count = self.session_manager.getSubscribers(io, topic, &subscribers_buf);

    // 3. 各サブスクライバーに転送 (ロック外)
    for (subscribers_buf[0..count]) |sub| {
        // 自分自身には送らない
        if (std.mem.eql(u8, sub.client_id, source_client_id)) continue;

        const effective_qos = @min(qos, sub.max_qos);
        sub.connection.sendPublish(
            io,
            topic,
            payload,
            effective_qos,
        ) catch |err| {
            std.log.warn("転送失敗 ({s}): {}", .{ sub.client_id, err });
        };
    }

    std.log.info(
        "ルーティング完了: topic={s}, 配信先={d}クライアント",
        .{ topic, count },
    );
}
```

複数サブスクライバーのテスト

ワイルドカードトピックの動作を確認するために、複数のサブスクライバーを起動する。

```
# ターミナル2: 全センサーデータを購読 (sensor/# にサブスクライブ)
./zig-out/bin/mqtt-sub

# ターミナル3: mosquitto_sub で温度データのみ購読
mosquitto_sub -h 127.0.0.1 -p 1883 -t "sensor/temp"

# ターミナル4: 湿度データを送信
mosquitto_pub -h 127.0.0.1 -p 1883 -t "sensor/humidity" -m "60%"
# -> ターミナル2 のみが受信する (sensor/# がマッチ)
```

```
# ターミナル5: 温度データを送信
mosquitto_pub -h 127.0.0.1 -p 1883 -t "sensor/temp" -m "25.3"
# -> ターミナル2 と ターミナル3 の両方が受信する
```

mosquitto クライアントとの相互運用テスト

本ブローカーは MQTT v3.1.1 準拠であるため、mosquitto の公式クライアントツールでもテストできる。

基本テスト

```
# mosquitto_sub で購読
mosquitto_sub -h 127.0.0.1 -p 1883 -t "sensor/#" -v

# 別ターミナルで mosquitto_pub で発行
mosquitto_pub -h 127.0.0.1 -p 1883 -t "sensor/temp" -m "25.3"
```

QoS 1 テスト

```
# QoS 1 で購読
mosquitto_sub -h 127.0.0.1 -p 1883 -t "qos/test" -q 1 -v

# QoS 1 で発行
mosquitto_pub -h 127.0.0.1 -p 1883 -t "qos/test" -m "reliable" -q 1
```

Retained メッセージテスト

```
# retained メッセージを保存
mosquitto_pub -h 127.0.0.1 -p 1883 -t "status/device" -m "online" -r

# 後から購読すると、即座に retained メッセージを受信
mosquitto_sub -h 127.0.0.1 -p 1883 -t "status/device" -v -C 1
# 出力: status/device online
```

Will メッセージテスト

```
# Will メッセージ付きで接続（Ctrl+C で強制切断）
mosquitto_sub -h 127.0.0.1 -p 1883 -t "test/#" \
  --will-topic "status/sub-client" \
```

```
  --will-payload "disconnected" \
  --will-qos 1

# 別ターミナルで Will メッセージを監視
mosquitto_sub -h 127.0.0.1 -p 1883 -t "status/#" -v

# 最初のターミナルで Ctrl+C -> Will メッセージが配信される
```

自作クライアントと mosquitto の混在テスト

```
# zig-out のサブスクライバーと mosquitto_pub を組み合わせる
./zig-out/bin/mqtt-sub &
mosquitto_pub -h 127.0.0.1 -p 1883 -t "sensor/temp" -m "from-mosquitto"

# mosquitto_sub と zig-out のパブリッシャーを組み合わせる
mosquitto_sub -h 127.0.0.1 -p 1883 -t "sensor/#" -v &
./zig-out/bin/mqtt-pub
```

相互運用テストは、プロトコル実装の正しさを検証する上で非常に有効である。 自作のクライアントとサードパーティのクライアントが問題なく通信できれば、MQTT 仕様に準拠していることの強い証拠となる。

トラブルシューティング

接続できない場合

```
# ブローカーが起動しているか確認
lsof -i :1883

# ファイアウォールの確認（macOS）
sudo pfctl -sr | grep 1883
```

メッセージが届かない場合

1. サブスクライバーの topic filter がパブリッシャーの topic に一致しているか確認
2. ブローカーのログで「ルーティング完了」メッセージを確認
3. QoS レベルが一致しているか確認

mosquitto がインストールされていない場合

```
# macOS
brew install mosquitto

# Ubuntu/Debian
sudo apt install mosquitto-clients
```

まとめ

- 全バイナリが `std.process.Init`（Juicy Main）で `gpa` と `io` を受け取る
- `std.Io.net.IpAddress.parse` でアドレスを解析し、`connect(io, .{})` で接続する
- reader/writer は `stream.reader(io, &buf)` / `stream.writer(io, &buf)` で生成する
- mosquitto クライアントとの混在テストでプロトコル準拠を検証できる
- ルーティング処理では RwLock と Mutex を適切に使い分けてロック競合を最小化する

次章では、統合テストの手法を学ぶ。

第14章 統合テスト

TCPを使ったテストパターン

学習目標

- MQTT ブローカーの統合テスト手法を理解する
- Zig 0.16 の `std.Io.Threaded` をテスト内で活用できる
- `std.testing.allocator` によるメモリリーク検出を理解する
- ブローカーの起動、接続、パブリッシュ、検証の一連のフローをテストで自動化できる
- エッジケース（不正パケット、高速接続/切断）のテスト設計ができる

テスト戦略

MQTT ブローカーのテストは3層で構成する。

レベル	対象	手法
ユニットテスト	個別モジュール（パケットエンコード、トピックマッチング）	`test` ブロック
統合テスト	ブローカー + クライアント間の通信	テスト内でブローカーを起動
相互運用テスト	外部ツール（mosquitto）との互換性	シェルスクリプト

Zig 0.16 の std.testing

基本的なアサーション

```
const std = @import("std");
const testing = std.testing;

test "パケットID生成" {
    var gen = PacketIdGenerator.init();

    try testing.expectEqual(@as(u16, 1), gen.next());
    try testing.expectEqual(@as(u16, 2), gen.next());
```

```
    try testing.expectEqual(@as(u16, 3), gen.next());
}

test "トピックマッチング" {
    try testing.expect(topicMatches("sensor/temp", "sensor/temp"));
    try testing.expect(topicMatches("sensor/temp", "sensor/#"));
    try testing.expect(topicMatches("sensor/temp", "+/temp"));
    try testing.expect(!topicMatches("sensor/temp", "sensor/humidity"));
}
```

std.testing.allocator: メモリリーク検出

`std.testing.allocator` はテスト終了時に全ての割り当てが解放されているかを検証する。リークがあればテストが失敗する。

```
test "セッション作成と破棄" {
    const allocator = std.testing.allocator;

    var sm = SessionManager.init(allocator);
    defer sm.deinit();

    const result = try sm.getOrCreate("client-01", true);
    try testing.expect(!result.session_existed);

    try sm.addSubscription("client-01", "sensor/#", 1);

    // 切断してクリーンアップ
    sm.disconnectClient("client-01", true);

    // test allocator がリークを自動検出
    // deinit で全メモリが解放されなければテスト失敗
}
```

std.Io.Threaded: テスト内での Io ランタイム

テスト内でブローカーを起動するには、Io ランタイムが必要である。 `std.process.Init` はテストでは使えないため、 `std.Io.Threaded` を直接構築する。

```
test "Io ランタイムの構築" {
    const allocator = std.testing.allocator;

    // テスト用の Io ランタイムを初期化
    var threaded = std.Io.Threaded.init(allocator, .{});
    const io = threaded.io();

    // io を使ってネットワーク操作やロック操作が可能
```

```
    _ = io;
}
```

`std.Io.Threaded` は、通常の `main` 関数で `init.io` が提供するものと同等の Io ランタイムを生成する。テスト環境ではこれが唯一の Io ランタイム取得方法である。

統合テストの実装

テスト用ブローカーの起動

テスト内でブローカーをバックグラウンドスレッドとして起動する。

```
const TestBroker = struct {
    broker: *MqttBroker,
    thread: std.Thread,
    port: u16,
    allocator: std.mem.Allocator,
    io: *std.Io,

    fn start(allocator: std.mem.Allocator, io: *std.Io) !TestBroker {
        const broker = try allocator.create(MqttBroker);
        broker.* = MqttBroker.init(allocator);

        // ポート 0 を指定すると OS が空きポートを割り当てる
        const port: u16 = 0;

        const thread = try std.Thread.spawn(.{}, runBroker, .{
            broker, io, port,
        });

        // ブローカーの起動を待つ
        std.Io.Clock.Duration.sleep(.{
            .clock = .awake,
            .raw = .fromMilliseconds(100),
        }, io);

        return .{
            .broker = broker,
            .thread = thread,
            .port = broker.actual_port,
            .allocator = allocator,
            .io = io,
        };
    }

    fn stop(self: *TestBroker) void {
        self.broker.shutdown(self.io);
        self.thread.join();
        self.broker.deinit(self.io);
```

```
        self.allocator.destroy(self.broker);
    }

    fn runBroker(broker: *MqttBroker, io: *std.Io, port: u16) void {
        broker.run(io, port) catch |err| {
            std.log.err("テストブローカーエラー: {}", .{err});
        };
    }
};
```

graceful shutdown の活用

テスト後に確実にブローカーを停止するために、graceful shutdown パターンを使う。

```
fn stop(self: *TestBroker) void {
    // atomic フラグで停止を通知
    self.broker.should_stop.store(true, .release);
    // Event でブロック中のスレッドを起こす
    self.broker.shutdown_event.set(self.io);
    // スレッドの完了を待つ
    self.thread.join();
    // リソースの解放
    self.broker.deinit(self.io);
    self.allocator.destroy(self.broker);
}
```

接続・パブリッシュ・受信の統合テスト

```
test "パブリッシュしたメッセージがサブスクライバーに届く" {
    const allocator = std.testing.allocator;
    var threaded = std.Io.Threaded.init(allocator, .{});
    const io = threaded.io();

    // ブローカー起動
    var broker = try TestBroker.start(allocator, io);
    defer broker.stop();

    // サブスクライバー接続
    const sub_address = std.Io.net.IpAddress.parse("127.0.0.1", broker.port);
    var sub_stream = try sub_address.connect(io, .{});
    defer sub_stream.close(io);

    var sub_read_buf: [4096]u8 = undefined;
    var sub_write_buf: [4096]u8 = undefined;
    var sub_reader = sub_stream.reader(io, &sub_read_buf);
    var sub_writer = sub_stream.writer(io, &sub_write_buf);
```

```
    try sendConnect(&sub_writer, "sub-client", .{ .clean_session = true });
    try sub_writer.interface.flush();
    const sub_connack = try receiveConnack(&sub_reader);
    try testing.expectEqual(ConnackCode.accepted, sub_connack.return_code);

    try sendSubscribe(&sub_writer, "test/topic", 0);
    try sub_writer.interface.flush();
    _ = try receiveSuback(&sub_reader);

    // パブリッシャー接続
    const pub_address = std.Io.net.IpAddress.parse("127.0.0.1", broker.port);
    var pub_stream = try pub_address.connect(io, .{});
    defer pub_stream.close(io);

    var pub_read_buf: [4096]u8 = undefined;
    var pub_write_buf: [4096]u8 = undefined;
    var pub_reader = pub_stream.reader(io, &pub_read_buf);
    var pub_writer = pub_stream.writer(io, &pub_write_buf);

    try sendConnect(&pub_writer, "pub-client", .{ .clean_session = true });
    try pub_writer.interface.flush();
    _ = try receiveConnack(&pub_reader);

    // メッセージ送信
    try sendPublish(&pub_writer, "test/topic", "hello mqtt", .{ .qos = 0 });
    try pub_writer.interface.flush();

    // サブスクライバー側で受信確認
    const received = try receivePublish(&sub_reader);
    try testing.expectEqualStrings("test/topic", received.topic);
    try testing.expectEqualStrings("hello mqtt", received.payload);
}
```

Retained メッセージのテスト

```
test "Retained メッセージが新規サブスクライバーに届く" {
    const allocator = std.testing.allocator;
    var threaded = std.Io.Threaded.init(allocator, .{});
    const io = threaded.io();

    var broker = try TestBroker.start(allocator, io);
    defer broker.stop();

    // パブリッシャーが retained メッセージを送信
    var pub_stream = try connectToBroker(io, broker.port, "pub-client");
    defer pub_stream.close(io);

    var pub_write_buf: [4096]u8 = undefined;
    var pub_writer = pub_stream.writer(io, &pub_write_buf);
    try sendPublish(&pub_writer, "status/device", "online", .{
        .qos = 0,
        .retain = true,
```

```
    });
    try pub_writer.interface.flush();

    // 少し待ってからサブスクライバーが接続
    std.Io.Clock.Duration.sleep(.{
        .clock = .awake,
        .raw = .fromMilliseconds(50),
    }, io);

    var sub_stream = try connectToBroker(io, broker.port, "sub-client");
    defer sub_stream.close(io);

    var sub_read_buf: [4096]u8 = undefined;
    var sub_write_buf: [4096]u8 = undefined;
    var sub_reader = sub_stream.reader(io, &sub_read_buf);
    var sub_writer = sub_stream.writer(io, &sub_write_buf);

    try sendSubscribe(&sub_writer, "status/device", 0);
    try sub_writer.interface.flush();
    _ = try receiveSuback(&sub_reader);

    // サブスクライブ直後に retained メッセージを受信するはず
    const received = try receivePublish(&sub_reader);
    try testing.expectEqualStrings("status/device", received.topic);
    try testing.expectEqualStrings("online", received.payload);
    try testing.expect(received.retain);
}
```

エッジケースのテスト

不正パケットの処理

ブローカーは不正なパケットを受け取ってもクラッシュしてはならない。

```
test "不正なパケットで接続が切断される" {
    const allocator = std.testing.allocator;
    var threaded = std.Io.Threaded.init(allocator, .{});
    const io = threaded.io();

    var broker = try TestBroker.start(allocator, io);
    defer broker.stop();

    const address = std.Io.net.IpAddress.parse("127.0.0.1", broker.port);
    var stream = try address.connect(io, .{});
    defer stream.close(io);

    var write_buf: [4096]u8 = undefined;
    var writer = stream.writer(io, &write_buf);
```

```
    // 不正なデータを送信（CONNECT なしで PUBLISH を送る）
    const garbage = [_]u8{ 0x30, 0x05, 0x00, 0x01, 'x', 'A', 'B' };
    try writer.interface.writeAll(&garbage);
    try writer.interface.flush();

    // ブローカーは接続を切断するはず
    var read_buf: [4096]u8 = undefined;
    var reader = stream.reader(io, &read_buf);
    var result_buf: [64]u8 = undefined;
    const n = reader.interface.readSliceAll(&result_buf) catch 0;
    try testing.expectEqual(@as(usize, 0), n);
}

test "CONNECT なしで SUBSCRIBE を送ると切断される" {
    const allocator = std.testing.allocator;
    var threaded = std.Io.Threaded.init(allocator, .{});
    const io = threaded.io();

    var broker = try TestBroker.start(allocator, io);
    defer broker.stop();

    const address = std.Io.net.IpAddress.parse("127.0.0.1", broker.port);
    var stream = try address.connect(io, .{});
    defer stream.close(io);

    var write_buf: [4096]u8 = undefined;
    var writer = stream.writer(io, &write_buf);

    // CONNECT を送らずに SUBSCRIBE を送信
    try sendSubscribe(&writer, "test/topic", 0);
    try writer.interface.flush();

    var read_buf: [4096]u8 = undefined;
    var reader = stream.reader(io, &read_buf);
    var result_buf: [64]u8 = undefined;
    const n = reader.interface.readSliceAll(&result_buf) catch 0;
    try testing.expectEqual(@as(usize, 0), n);
}
```

高速接続・切断

```
test "高速な接続と切断を繰り返してもブローカーは安定動作する" {
    const allocator = std.testing.allocator;
    var threaded = std.Io.Threaded.init(allocator, .{});
    const io = threaded.io();

    var broker = try TestBroker.start(allocator, io);
    defer broker.stop();

    // 100回の高速接続・切断
    for (0..100) |i| {
        var id_buf: [32]u8 = undefined;
```

```
        const client_id = std.fmt.bufPrint(&id_buf, "rapid-{d}", .{i}) catch unreachab
le;

        const address = std.Io.net.IpAddress.parse("127.0.0.1", broker.port);
        var stream = address.connect(io, .{}) catch continue;

        var write_buf: [4096]u8 = undefined;
        var writer = stream.writer(io, &write_buf);
        sendConnect(&writer, client_id, .{ .clean_session = true }) catch {
            stream.close(io);
            continue;
        };
        writer.interface.flush() catch {};

        // CONNACK を待たずに即切断
        stream.close(io);
    }

    // ブローカーがまだ動作していることを確認
    const verify_address = std.Io.net.IpAddress.parse("127.0.0.1", broker.port);
    var verify_stream = try verify_address.connect(io, .{});
    defer verify_stream.close(io);

    var vr_buf: [4096]u8 = undefined;
    var vw_buf: [4096]u8 = undefined;
    var verify_reader = verify_stream.reader(io, &vr_buf);
    var verify_writer = verify_stream.writer(io, &vw_buf);

    try sendConnect(&verify_writer, "verify-client", .{ .clean_session = true });
    try verify_writer.interface.flush();
    const connack = try receiveConnack(&verify_reader);
    try testing.expectEqual(ConnackCode.accepted, connack.return_code);
}
```

クライアントテイクオーバーのテスト

```
test "同一 client_id の再接続で旧接続が切断される" {
    const allocator = std.testing.allocator;
    var threaded = std.Io.Threaded.init(allocator, .{});
    const io = threaded.io();

    var broker = try TestBroker.start(allocator, io);
    defer broker.stop();

    // 最初の接続
    const addr = std.Io.net.IpAddress.parse("127.0.0.1", broker.port);
    var stream1 = try addr.connect(io, .{});
    defer stream1.close(io);

    var w1_buf: [4096]u8 = undefined;
    var r1_buf: [4096]u8 = undefined;
    var writer1 = stream1.writer(io, &w1_buf);
```

```
    var reader1 = stream1.reader(io, &r1_buf);
    try sendConnect(&writer1, "takeover-client", .{ .clean_session = true });
    try writer1.interface.flush();
    _ = try receiveConnack(&reader1);

    // 同じ client_id で2つ目の接続
    var stream2 = try addr.connect(io, .{});
    defer stream2.close(io);

    var w2_buf: [4096]u8 = undefined;
    var r2_buf: [4096]u8 = undefined;
    var writer2 = stream2.writer(io, &w2_buf);
    var reader2 = stream2.reader(io, &r2_buf);
    try sendConnect(&writer2, "takeover-client", .{ .clean_session = true });
    try writer2.interface.flush();
    _ = try receiveConnack(&reader2);

    // stream1 は切断されているはず
    std.Io.Clock.Duration.sleep(.{
        .clock = .awake,
        .raw = .fromMilliseconds(100),
    }, io);
    var result_buf: [64]u8 = undefined;
    const n = reader1.interface.readSliceAll(&result_buf) catch 0;
    try testing.expectEqual(@as(usize, 0), n);
}
```

mosquitto を使った相互運用テスト

外部の MQTT クライアントツールを使うことで、プロトコル実装の正しさを独立して検証できる。

テストスクリプト

```
#!/bin/bash
# interop_test.sh

# ブローカーを起動
./zig-out/bin/mqtt-broker &
BROKER_PID=$!
sleep 1

# テスト1: 基本的な pub/sub
echo "=== テスト1: 基本 pub/sub ==="
mosquitto_sub -h 127.0.0.1 -t "test/topic" -C 1 &
SUB_PID=$!
sleep 0.5

mosquitto_pub -h 127.0.0.1 -t "test/topic" -m "hello"
wait $SUB_PID
```

```
echo "テスト1: OK"

# テスト2: ワイルドカード
echo "=== テスト2: ワイルドカード ==="
mosquitto_sub -h 127.0.0.1 -t "sensor/#" -C 1 &
SUB_PID=$!
sleep 0.5

mosquitto_pub -h 127.0.0.1 -t "sensor/temp" -m "25.3"
wait $SUB_PID
echo "テスト2: OK"

# テスト3: Retained メッセージ
echo "=== テスト3: Retained ==="
mosquitto_pub -h 127.0.0.1 -t "retained/test" -m "stored" -r
sleep 0.5
RESULT=$(mosquitto_sub -h 127.0.0.1 -t "retained/test" -C 1 -W 2)
if [ "$RESULT" = "stored" ]; then
    echo "テスト3: OK"
else
    echo "テスト3: FAIL (期待: stored, 実際: $RESULT)"
fi

# クリーンアップ
kill $BROKER_PID
echo "=== 全テスト完了 ==="
```

テストの実行

```
# ユニットテスト + 統合テストを全て実行
zig build test

# 特定のテストファイルのみ実行
zig build test --filter "パブリッシュしたメッセージ"

# テスト中のログ出力を表示
zig build test 2>&1 | head -50
```

テスト用の build.zig 設定

```
// build.zig に追加
const tests = b.addTest(.{
    .root_source_file = b.path("src/tests/integration_test.zig"),
    .target = target,
    .optimize = optimize,
});
```

```
const test_step = b.step("test", "統合テストを実行");
test_step.dependOn(&b.addRunArtifact(tests).step);
```

テスト設計のガイドライン

良いテストの特徴

1. **独立性**: 各テストは他のテストの結果に依存しない
2. **再現性**: 何度実行しても同じ結果になる
3. **速度**: 統合テストでもタイムアウトは最小限にする
4. **明確さ**: テスト名から何をテストしているか分かる

テスト内の Io ランタイム初期化パターン

全ての統合テストで共通のパターン:

```
test "テスト名" {
    // 1. アロケータの取得
    const allocator = std.testing.allocator;

    // 2. Io ランタイムの構築
    var threaded = std.Io.Threaded.init(allocator, .{});
    const io = threaded.io();

    // 3. テスト用ブローカーの起動
    var broker = try TestBroker.start(allocator, io);
    defer broker.stop();

    // 4. テストクライアントの接続
    const address = std.Io.net.IpAddress.parse("127.0.0.1", broker.port);
    var stream = try address.connect(io, .{});
    defer stream.close(io);

    // 5. reader/writer の生成
    var read_buf: [4096]u8 = undefined;
    var write_buf: [4096]u8 = undefined;
    var reader = stream.reader(io, &read_buf);
    var writer = stream.writer(io, &write_buf);

    // 6. テスト本体
    // ...
}
```

テストで検証すべき項目

```
接続:
  [ ] 正常な CONNECT / CONNACK
  [ ] 不正なプロトコルバージョンの拒否
  [ ] 空の client_id の処理
  [ ] クライアントテイクオーバー

パブリッシュ:
  [ ] QoS 0 の配信
  [ ] QoS 1 の配信と PUBACK
  [ ] Retained メッセージの保存と配信
  [ ] 空ペイロードによる Retained 削除

サブスクリプション:
  [ ] 完全一致トピック
  [ ] + ワイルドカード
  [ ] # ワイルドカード
  [ ] UNSUBSCRIBE 後のメッセージ非配信

セッション:
  [ ] Clean Session = true
  [ ] Clean Session = false (永続セッション)
  [ ] Will メッセージの発行

エッジケース:
  [ ] 不正なパケットフォーマット
  [ ] 高速接続・切断
  [ ] 大量の同時接続
  [ ] 極端に長いトピック名
  [ ] バイナリペイロード
```

まとめ

- テスト内で `std.Io.Threaded.init(allocator, .{})` を使って Io ランタイムを構築する
- `std.testing.allocator` でメモリリークを自動検出できる
- ネットワーク接続は `std.Io.net.IpAddress.parse` + `connect(io, .{})` で行う
- reader/writer は `stream.reader(io, &buf)` / `stream.writer(io, &buf)` で生成する
- `std.atomic.Value(bool)` + `std.Io.Event` でテスト後の確実なブローカー停止を実現する
- エッジケーステストでブローカーの堅牢性を確認する
- mosquitto クライアントとの相互運用テストでプロトコル準拠を検証する

以上で本チュートリアルの全章が完了した。ここまで学んだ知識を活かして、MQTT ブローカーの実装を完成させてほしい。

第VI部 発展編

Zig 0.16の新機能を活用してブローカーを改善します

第15章 イベント駆動ブローカー

io.async()とIo.Groupによる非同期化

学習目標

- Thread-per-connection モデルとイベント駆動モデルの違いを理解する
- `io.async()` を使った非ブロッキング接続受け入れを実装できる
- `std.Io.Group` を使ったファンアウト配信を実装できる
- Io バックエンド（Threaded / Evented）の切り替え方法を理解する
- `io.async` と `io.concurrent` の使い分けを判断できる

Thread-per-connection の限界

第10章で実装した Thread-per-connection モデルは教育目的には優れているが、 本番運用では深刻なスケーラビリティ問題がある。

```
Thread-per-connection:
  クライアント 100 台  → OS スレッド 100 本（スタック 8MB × 100 = 800MB）
  クライアント 1,000 台 → OS スレッド 1,000 本（スタック 8GB、コンテキストスイッチ多発）
  クライアント 10,000 台 → OS の限界を超える

イベント駆動:
  クライアント 100 台   → 1 イベントループ + ワーカースレッド数本
  クライアント 1,000 台  → 同上（カーネルが接続を多重化）
  クライアント 10,000 台 → 同上（io_uring / kqueue が効率的に処理）
```

指標	Thread-per-connection	イベント駆動
メモリ使用量	O(N) スレッドスタック	O(N) 接続状態のみ
コンテキストスイッチ	O(N)	O(1) イベントループ
最大接続数	~数千	~数万〜数十万
実装の複雑さ	低い	中程度
デバッグ	容易	やや困難

Zig 0.16 の Io モデル

Zig 0.16 では `std.Io` がランタイムの非同期基盤を提供する。 これにより、同じコードがスレッドプールベース（Threaded）でも カーネルイベント駆動（Evented）でも動作する。

バックエンドの種類

バックエンド	内部実装	対応 OS
`std.Io.Threaded`	スレッドプール	全 OS（クロスプラットフォーム）
`std.Io.Evented`	io_uring / kqueue	Linux / macOS / BSD

バックエンドの選択

バックエンドの切り替えは初期化コードの1行を変えるだけで済む。 ビジネスロジックは一切変更不要である。

```
const std = @import("std");

// スレッドプールバックエンド（クロスプラットフォーム）
pub fn mainThreaded(init: std.process.Init) !void {
    var io = init.io; // init.io は Threaded バックエンド
    try runBroker(io, init.gpa);
}

// io_uring / kqueue バックエンド（高性能）
pub fn mainEvented(init: std.process.Init) !void {
    // Evented バックエンドを明示的に初期化
    var evented: std.Io.Evented = .init;
    defer evented.deinit();
    try runBroker(&evented, init.gpa);
}

// ビジネスロジックはバックエンドに依存しない
fn runBroker(io: anytype, gpa: std.mem.Allocator) !void {
    // 同じコードが Threaded でも Evented でも動く
    _ = io;
    _ = gpa;
}
```

io.async によるノンブロッキング accept

従来の accept ループはブロッキングであった。 `io.async()` を使うと、accept を非同期に発行し、新しい接続が来るまで他のタスクを実行できる。

従来のブロッキング accept

```
// 第10章のコード: ブロッキング
while (true) {
    const conn = try server.accept(io);            // ← ブロック
    const thread = try std.Thread.spawn(.{}, handleConnection, .{self, conn});
    thread.detach();
}
```

io.async を使った非同期 accept

```
const net = std.Io.net;

fn runAcceptLoop(self: *MqttBroker, io: anytype) !void {
    const address = net.IpAddress.parse("0.0.0.0", 1883) catch return error.AddressPar
seFailed;
    var server = try net.IpAddress.listen(&address, io, .{ .reuse_address = true });
    defer server.deinit(io);

    std.log.info("イベント駆動ブローカー起動: ポート 1883", .{});

    while (!self.stop_requested.load(.acquire)) {
        // accept を非同期に発行
        const conn = try server.accept(io);

        // 接続ハンドラを非同期タスクとして起動
        // io.async はタスクをイベントループに登録し、即座に返る
        const future = io.async(handleClient, .{ self, conn, io });
        _ = future; // fire-and-forget: 個別の await は不要
    }
}
```

`io.async()` の戻り値は future である。 `future.await(io)` を呼ぶと タスクの完了を待てるが、fire-and-forget パターンでは await しなくてもよい。

---

io.async vs io.concurrent

Zig 0.16 では2種類の非同期プリミティブが提供される。 用途に応じて使い分ける必要がある。

API	用途	スケジューリング
`io.async(fn, args)`	I/O バウンド処理	イベントループ内で協調スケジュール
`io.concurrent(fn, args)`	CPU バウンド処理	別スレッドで並列実行

使い分けの指針

```
// I/O バウンド: ネットワーク読み書き、ファイル操作
// → io.async を使う
const read_future = io.async(readPacket, .{ stream, buffer });

// CPU バウンド: 暗号化、圧縮、大量データの変換
// → io.concurrent を使う（別スレッドで実行される）
const hash_future = io.concurrent(computeHash, .{payload}) catch |err| {
    std.log.err("concurrent タスク生成失敗: {}", .{err});
    return err;
};

// 両方とも .await(io) で結果を取得
const packet = read_future.await(io);
const hash = hash_future.await(io);
```

MQTT ブローカーでは、ほとんどの処理が I/O バウンド（TCP 読み書き）であるため、`io.async` を主に使用する。パスワードのハッシュ計算のような CPU 集約処理にのみ `io.concurrent` を使う。

std.Io.Group によるファンアウト配信

PUBLISH メッセージを複数のサブスクライバーに同時配信する場合、`std.Io.Group` を使うと全配信を並行して発行し、全完了を待機できる。

Group の基本パターン

```
fn fanOutPublish(
    io: anytype,
    subscribers: []const SubscriberInfo,
    topic: []const u8,
    payload: []const u8,
) !void {
    var group: std.Io.Group = .init;

    // 各サブスクライバーへの配信を並行タスクとして登録
    for (subscribers) |sub| {
        group.async(io, sendToSubscriber, .{ sub, topic, payload, io });
```

```
    }

    // 全タスクの完了を待機
    // いずれかがエラーを返した場合、group.await もエラーを返す
    try group.await(io);
}

fn sendToSubscriber(
    sub: SubscriberInfo,
    topic: []const u8,
    payload: []const u8,
    io: anytype,
) !void {
    const effective_qos = @min(sub.max_qos, sub.publish_qos);
    const packet = encodePublish(topic, payload, effective_qos);
    var write_buf: [4096]u8 = undefined;
    var writer = sub.stream.writer(io, &write_buf);
    try writer.interface.writeAll(packet);
    try writer.interface.flush();
}
```

従来のコードとの比較

```
// 従来: 逐次配信（第10章）
// サブスクライバー N 人 → 合計 N 回の write を直列実行
for (subscribers) |sub| {
    sub.connection.sendPublish(topic, payload, qos) catch |err| {
        std.log.warn("転送エラー: {}", .{err});
    };
}

// イベント駆動: 並行配信（本章）
// サブスクライバー N 人 → N 回の write を並行発行
// Evented バックエンドなら io_uring に一括サブミットされる
var group: std.Io.Group = .init;
for (subscribers) |sub| {
    group.async(io, sendToSubscriber, .{ sub, topic, payload, io });
}
try group.await(io);
```

ファンアウト配信は MQTT ブローカーのホットパスであり、 Group による並行化の効果が最も大きい箇所である。

イベント駆動ブローカーの骨格

```
const std = @import("std");
const net = std.Io.net;
```

```
const EventDrivenBroker = struct {
    gpa: std.mem.Allocator,
    session_manager: *SessionManager,
    retain_store: *RetainStore,
    stop_requested: std.atomic.Value(bool),

    pub fn run(self: *EventDrivenBroker, io: anytype) !void {
        const address = net.IpAddress.parse("0.0.0.0", 1883) catch return error.Addres
sParseFailed;
        var server = try net.IpAddress.listen(&address, io, .{ .reuse_address = true }
);
        defer server.deinit(io);

        while (!self.stop_requested.load(.acquire)) {
            const conn = try server.accept(io);
            // 接続ハンドラを非同期タスクとして起動 (fire-and-forget)
            _ = io.async(handleClient, .{ self, conn.stream, io });
        }
    }

    fn handleClient(self: *EventDrivenBroker, stream: net.Stream, io: anytype) void {
        defer stream.close(io);

        var read_buf: [4096]u8 = undefined;
        var reader = stream.reader(io, &read_buf);

        while (!self.stop_requested.load(.acquire)) {
            var byte_buf: [1]u8 = undefined;
            reader.interface.readSliceAll(&byte_buf) catch break;
            const first_byte = byte_buf[0];
            const packet_type: u4 = @intCast(first_byte >> 4);

            switch (packet_type) {
                3 => self.handlePublish(stream, first_byte, io) catch break,
                12 => {
                    var write_buf: [4096]u8 = undefined;
                    var writer = stream.writer(io, &write_buf);
                    writer.interface.writeAll(&.{ 0xD0, 0x00 }) catch break;
                    writer.interface.flush() catch break;
                },
                14 => break, // DISCONNECT
                else => break,
            }
        }
    }

    fn handlePublish(self: *EventDrivenBroker, _: net.Stream, _: u8, io: anytype) !voi
d {
        // サブスクライバー検索 → Group でファンアウト配信
        var buf: [128]SubscriberInfo = undefined;
        const count = self.session_manager.getSubscribers("topic", &buf);

        var group: std.Io.Group = .init;
```

```
        for (buf[0..count]) |sub| {
            group.async(io, sendToSubscriber, .{ sub, "topic", "payload" });
        }
        try group.await(io);
    }

    fn sendToSubscriber(_: SubscriberInfo, _: []const u8, _: []const u8) void {}
};

const SubscriberInfo = struct { stream: net.Stream, max_qos: u2, publish_qos: u2 };
const SessionManager = struct {
    pub fn getSubscribers(_: *SessionManager, _: []const u8, _: []SubscriberInfo) usiz
e { return 0; }
};
const RetainStore = struct {};
```

パフォーマンス特性の比較

メモリ使用量

```
Thread-per-connection (1,000 接続):
 スレッドスタック:   8 MB × 1,000 = 8,000 MB
 接続状態:          ~1 KB × 1,000 =     1 MB
 合計:                              ~8,001 MB

イベント駆動 (1,000 接続):
 ワーカースレッド:   8 MB × 4     =    32 MB
 接続状態:          ~1 KB × 1,000 =     1 MB
 イベント管理:      ~64 B × 1,000 =   0.06 MB
 合計:                              ~   33 MB
```

システムコール削減（Evented バックエンド）

io_uring を使用する Evented バックエンドでは、複数の I/O 操作を 1回の `io_uring_enter` システムコールにバッチ送信できる。

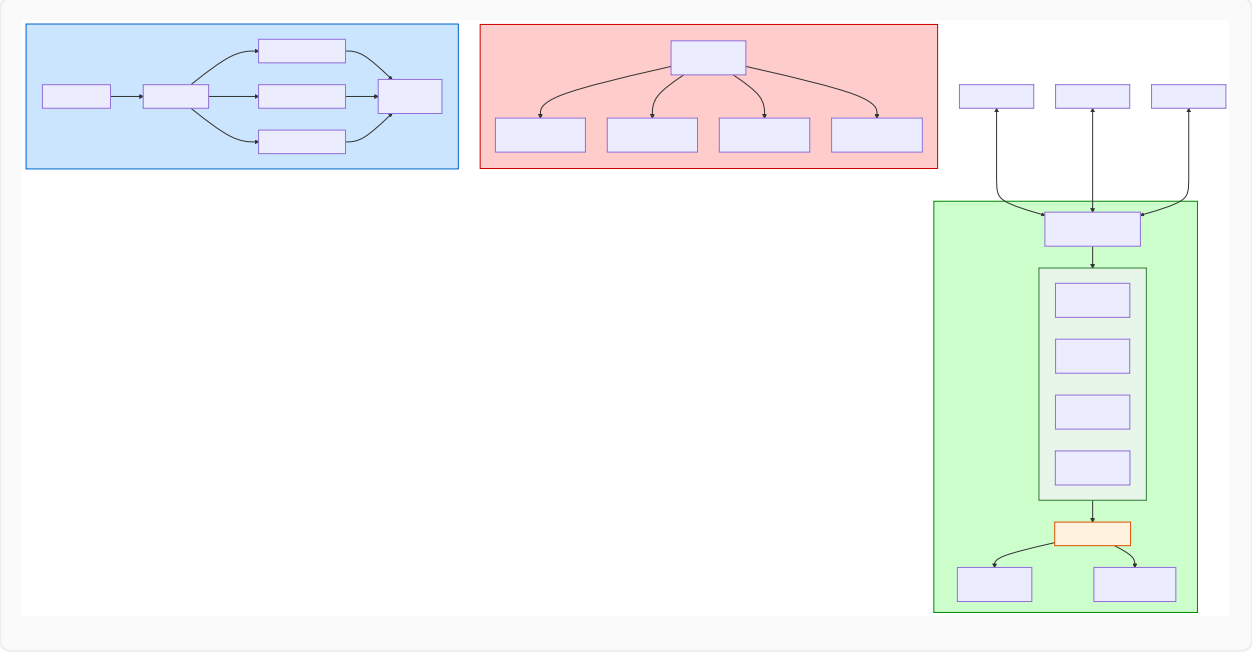
```
Thread-per-connection (10 サブスクライバーにファンアウト):
  write() × 10 = システムコール 10 回

Evented + Io.Group (10 サブスクライバーにファンアウト):
  io_uring_enter() × 1 = システムコール 1 回 (10 件の SQE を一括送信)
```

まとめ

- Thread-per-connection は実装が容易だが、数千接続でスケーラビリティの壁がある
- Zig 0.16 の `std.Io` は Threaded と Evented の2つのバックエンドを透過的に切り替えられる
- `io.async()` は I/O バウンド処理に、`io.concurrent()` は CPU バウンド処理に使う
- `std.Io.Group` を使えば、ファンアウト配信を並行発行して全完了を一括待機できる
- バックエンドの変更はエントリポイントの初期化のみで、ビジネスロジックは不変

次章では、`std.Io.Queue(T)` を使ったメッセージキューイングによるバックプレッシャー制御を学ぶ。

図



第16章 Io.Queueによるメッセージング

ロックフリーなメッセージルーティング

学習目標

- 直接送信モデルのロック競合問題を理解する
- `std.Io.Queue(T)` の API と動作原理を習得する
- プロデューサー・コンシューマーパターンを MQTT ブローカーに適用できる
- バックプレッシャーによるスロークライアント対策を実装できる
- メモリバウンドなキューによる OOM 防止を理解する

現在の問題: 直接送信モデルのロック競合

第10章・第15章のブローカーでは、PUBLISH メッセージの配信時に 送信先クライアントの TCP ストリームに直接書き込んでいた。

```
// 問題のあるコード: PUBLISH ハンドラ内
for (subscribers) |sub| {
    // 送信先の stream に直接 write
    // → 複数パブリッシャーが同時に同じサブスクライバーに送信すると
    //     データが混在する (interleave) 可能性がある
    sub.connection.sendPublish(topic, payload, qos) catch {};
}
```

問題点の整理

問題	説明
データ混在	複数スレッドが同じ stream に同時書き込みすると、パケットが壊れる
ロック競合	stream ごとに Mutex を追加すると、高頻度 PUBLISH でボトルネックになる
スロークライアント	書き込みがブロックすると、パブリッシャー側のスレッドも停止する
メモリ無制限	バッファリングなしでは OOM の危険があり、ありでは上限がない

解決策: per-client メッセージキュー

各クライアント接続に専用のメッセージキューを持たせる。 パブリッシュハンドラはキューにメッセージを投入するだけで、 実際の TCP 送信はクライアント専用のライタータスクが行う。

```
                              ┌──────────────────┐
PUBLISH ハンドラ ──────>│  Io.Queue(Message) │──────> ライタータスク ──────> TCP stream
(プロデューサー)          │  固定サイズバッファ    │        (コンシューマー)
                              └──────────────────┘
```

この設計により: - **データ混在なし**: stream への書き込みは1つのタスクのみ - **ロック不要**: Queue が内部で同期を行う - **バックプレッシャー**: キューが満杯になるとプロデューサーがブロック - **メモリバウンド**: 固定サイズバッファで OOM を防止

std.Io.Queue(T) の API

初期化

```zig
const Message = struct {
    topic: []const u8,
    payload: []const u8,
    qos: u2,
};

// 固定サイズバッファを確保 (128 メッセージ分)
var buffer: [128]Message = undefined;

// バッファを渡して Queue を初期化
var queue: std.Io.Queue(Message) = .init(&buffer);
```

`init` に渡すバッファのサイズがキューの最大容量を決定する。 動的メモリ確保は一切行わないため、OOM の心配がない。

プロデューサー側: putOne

```zig
// メッセージをキューに投入
// キューが満杯の場合、空きができるまでブロック (バックプレッシャー)
try queue.putOne(io, .{
    .topic = topic,
    .payload = payload,
```

```
    .qos = qos,
});
```

`putOne` はキューに空きがない場合、コンシューマーが取り出すまで待機する。 これが自動バックプレッシャーの仕組みである。

コンシューマー側: getOne

```
// キューからメッセージを取り出す
// キューが空の場合、メッセージが到着するまでブロック
const message = queue.getOne(io) catch |err| {
    switch (err) {
        error.Canceled => return, // キューがクローズされた
        else => return err,
    }
};
```

`getOne` はキューが空の場合、プロデューサーが投入するまで待機する。 キューがクローズされると `error.Canceled` を返す。

キャンセル不可能な取り出し: getOneUncancelable

```
// シャットダウン時に残りのメッセージをドレインする
while (true) {
    const message = queue.getOneUncancelable(io) orelse break;
    try flushMessage(stream, message);
}
```

`getOneUncancelable` はキャンセルシグナルを無視してメッセージを取り出す。 キューが空の場合は `null` を返す。シャットダウン時のドレイン処理に最適である。

クローズ

```
// キューをクローズする
// getOne でブロック中のタスクに error.Canceled を返す
queue.close(io);
```

MessageQueue 構造体の設計

`Io.Queue(T)` を MQTT ブローカー向けにラップした構造体を定義する。

```
const std = @import("std");

pub const OutboundMessage = struct {
    topic_buf: [256]u8,
    topic_len: u16,
    payload_buf: [4096]u8,
    payload_len: u16,
    qos: u2,

    /// トピック文字列を取得
    pub fn topic(self: *const OutboundMessage) []const u8 {
        return self.topic_buf[0..self.topic_len];
    }

    /// ペイロードを取得
    pub fn payload(self: *const OutboundMessage) []const u8 {
        return self.payload_buf[0..self.payload_len];
    }

    /// 値をコピーして OutboundMessage を作成
    pub fn from(topic_src: []const u8, payload_src: []const u8, qos: u2) OutboundMessa
ge {
        var msg: OutboundMessage = undefined;
        msg.qos = qos;

        const t_len: u16 = @intCast(@min(topic_src.len, 256));
        @memcpy(msg.topic_buf[0..t_len], topic_src[0..t_len]);
        msg.topic_len = t_len;

        const p_len: u16 = @intCast(@min(payload_src.len, 4096));
        @memcpy(msg.payload_buf[0..p_len], payload_src[0..p_len]);
        msg.payload_len = p_len;

        return msg;
    }
};

pub const MessageQueue = struct {
    buffer: [64]OutboundMessage,
    queue: std.Io.Queue(OutboundMessage),

    pub fn init(self: *MessageQueue) void {
        self.buffer = undefined;
        self.queue = .init(&self.buffer);
    }

    /// メッセージを投入（キュー満杯時はバックプレッシャー）
    pub fn enqueue(self: *MessageQueue, io: anytype, msg: OutboundMessage) !void {
        try self.queue.putOne(io, msg);
    }

    /// メッセージを取り出す（キュー空時はブロック）
    pub fn dequeue(self: *MessageQueue, io: anytype) !OutboundMessage {
        return try self.queue.getOne(io);
```

```
    }

    /// 残りメッセージをドレイン（シャットダウン用）
    pub fn drainRemaining(self: *MessageQueue, io: anytype) ?OutboundMessage {
        return self.queue.getOneUncancelable(io);
    }

    /// キューをクローズ
    pub fn close(self: *MessageQueue, io: anytype) void {
        self.queue.close(io);
    }
};
```

設計の特徴

- **固定サイズバッファ**: `OutboundMessage` はトピック 256 バイト、ペイロード 4096 バイトの 固定サイズ構造体。動的メモリ確保を完全に排除している
- **値コピー**: `from()` でデータをコピーするため、元の PUBLISH パケットのライフタイムに依存しない
- **64 メッセージバッファ**: 1クライアントあたり最大64メッセージをバッファリング

ConnectionHandler への統合

各クライアント接続に MessageQueue を持たせ、リーダーとライターを分離する。

```
const net = std.Io.net;

const ClientConnection = struct {
    stream: net.Stream,
    client_id: [64]u8,
    client_id_len: u8,
    message_queue: MessageQueue,

    /// リーダータスク: クライアントからパケットを受信して処理
    pub fn readerTask(self: *ClientConnection, broker: *Broker, io: anytype) void {
        var read_buf: [4096]u8 = undefined;
        var reader = self.stream.reader(io, &read_buf);

        while (true) {
            var byte_buf: [1]u8 = undefined;
            reader.interface.readSliceAll(&byte_buf) catch break;
            const first_byte = byte_buf[0];
            const packet_type: u4 = @intCast(first_byte >> 4);
            switch (packet_type) {
                3 => broker.routePublish(self, first_byte, io) catch break,
                14 => break, // DISCONNECT
                else => break,
```

```
            }
        }
        // リーダー終了 → キューをクローズしてライターも停止させる
        self.message_queue.close(io);
    }

    /// ライタータスク: キューからメッセージを取り出して TCP 送信
    pub fn writerTask(self: *ClientConnection, io: anytype) void {
        var write_buf: [4096]u8 = undefined;
        var writer = self.stream.writer(io, &write_buf);

        while (true) {
            const msg = self.message_queue.dequeue(io) catch |err| {
                switch (err) {
                    error.Canceled => break, // キューがクローズされた
                    else => break,
                }
            };
            writer.interface.writeAll(msg.topic()) catch break;
            writer.interface.writeAll(msg.payload()) catch break;
            writer.interface.flush() catch break;
        }
    }
};

const Broker = struct {
    pub fn routePublish(_: *Broker, _: *ClientConnection, _: u8, _: anytype) !void {}
};
```

PUBLISH ルーティングの全体フロー

```
fn routePublish(self: *Broker, packet: PublishPacket, io: anytype) !void {
    // サブスクライバー検索
    var buf: [128]*ClientConnection = undefined;
    const count = self.findSubscribers(packet.topic, &buf);

    // 各サブスクライバーのキューにメッセージを投入
    const msg = OutboundMessage.from(packet.topic, packet.payload, packet.qos);
    for (buf[0..count]) |sub| {
        sub.message_queue.enqueue(io, msg) catch {
            // スロークライアント対策: キューが満杯なら切断
            sub.message_queue.close(io);
        };
    }
}
```

バックプレッシャーの動作

```
正常時:    パブリッシャー → putOne() → [Queue: ■■■□□□□□] → getOne() → TCP
       即座に返る

満杯時:    パブリッシャー → putOne() → [Queue: ■■■■■■■■] → getOne() → TCP (遅い)
       ブロック!         キュー満杯

`putOne` がブロック → パブリッシャーの速度が自動抑制される
```

実運用ではキュー投入にタイムアウトを設け、超過したら切断する戦略も有効である。

Cancelable エラーについて

`Io.Queue` の `getOne` と `putOne` は、キューがクローズされると `error.Canceled` を返す。これは Zig 0.16 の Io 系 API に共通するパターンである。

```zig
const msg = queue.getOne(io) catch |err| {
    switch (err) {
        error.Canceled => {
            // キューがクローズされた（正常なシャットダウン）
            std.log.info("キューがクローズされました", .{});
            return;
        },
        else => {
            // その他のエラー（I/O エラーなど）
            std.log.err("予期しないエラー: {}", .{err});
            return err;
        },
    }
};
```

`getOneUncancelable` はキャンセルシグナルを無視するため、 シャットダウン時の最終ドレインに使用する。

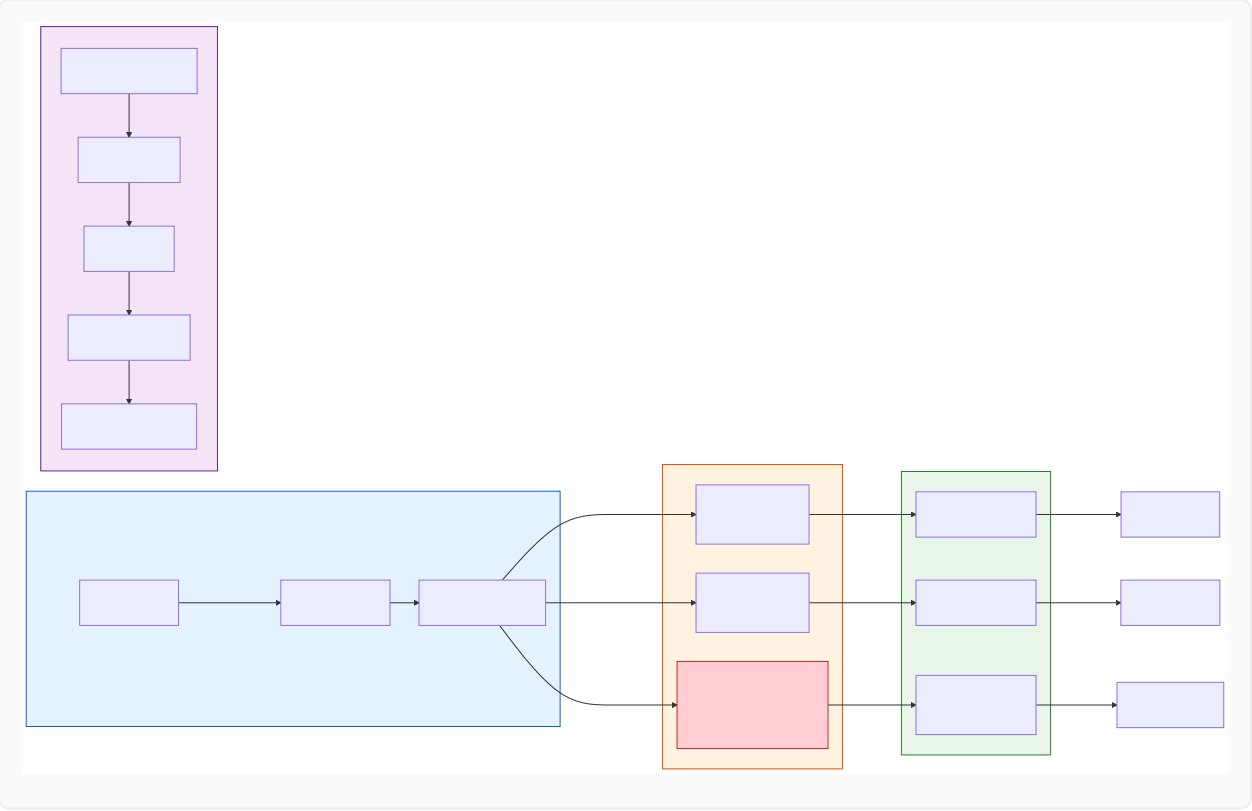
まとめ

- 直接送信モデルではデータ混在・ロック競合・スロークライアント問題が発生する
- `std.Io.Queue(T)` は固定サイズバッファによるプロデューサー・コンシューマーキューを提供する
- `putOne` / `getOne` はバックプレッシャーを自動的に実現する
- `close` によりブロック中の `getOne` に `error.Canceled` を通知できる

- `getOneUncancelable` はシャットダウン時のドレイン処理に使用する
- 1クライアント = 1リーダータスク + 1ライタータスク + 1キューの構成が安全かつ効率的

次章では、`std.atomic.Value` と `std.Io.Event` を使ったグレースフルシャットダウンの実装を学ぶ。

図



第17章 Graceful Shutdown

安全な停止パターンとWillメッセージ

学習目標

- 突然の終了がもたらす問題（メッセージ消失、Will 未送信）を理解する
- `std.atomic.Value(bool)` による停止フラグの安全な共有を実装できる
- `std.Io.Event` によるスレッド間通知を実装できる
- 段階的なシャットダウンシーケンスを設計・実装できる
- `Event.waitTimeout` による有限時間シャットダウンを実装できる

問題: 突然の終了

Ctrl+C や `kill` で MQTT ブローカーを停止すると、以下の問題が発生する。

問題	影響
インフライトメッセージの消失	QoS 1/2 のメッセージが配信途中で失われる
Will メッセージ未送信	異常切断したクライアントの Last Will が他のサブスクライバーに届かない
TCP 接続の未クリーンクローズ	クライアント側で TIME_WAIT が大量発生する
セッション状態の不整合	persistent session のデータが中途半端な状態で残る
メッセージキューの未ドレイン	Io.Queue に残ったメッセージが配信されない

解決策の全体設計

グレースフルシャットダウンは以下の3つのプリミティブで構成する。

1. std.atomic.Value(bool) - 停止フラグ

全タスクが参照する停止フラグ。アトミック操作により、 ロックなしで安全に読み書きできる。

```
const StopFlag = std.atomic.Value(bool);
```

```
// 初期化
var stop_requested = StopFlag.init(false);

// 書き込み（シャットダウン開始側）
stop_requested.store(true, .release);

// 読み取り（各タスクのループ内）
if (stop_requested.load(.acquire)) {
    // シャットダウン処理を開始
    break;
}
```

メモリオーダリングの解説

オーダリング	用途	説明
`.release`	`store` 時	このストアより前の書き込みが、他スレッドから見える
`.acquire`	`load` 時	このロードより後の読み取りが、最新の値を見る

`.release` と `.acquire` のペアにより、停止フラグが `true` になった時点で シャットダウンに必要な全ての前処理（キューのクローズなど）が 他スレッドから確実に見えることが保証される。

2. std.Io.Event - ブロック中タスクの起動

`accept()` や `getOne()` でブロックしているタスクは、 停止フラグのポーリングだけでは即座に反応できない。 `Io.Event` を使うと、ブロック中のタスクを明示的に起動できる。

```
var stop_event: std.Io.Event = .unset;

stop_event.waitUncancelable(io);  // set() が呼ばれるまでブロック
stop_event.set(io);               // waitUncancelable 中の全タスクが起動
```

3. Event.waitTimeout - 有限時間待機

```
const result = stop_event.waitTimeout(io, .{ .duration = .fromMilliseconds(5000) });
if (result == null) {
    std.log.warn("タイムアウト。強制終了します", .{});
}
```

シャットダウンシーケンス

以下の8ステップを順番に実行する。

```
Step 1: stop_requested = true（停止フラグセット）
    ↓
Step 2: stop_event.set()（ブロック中タスクを起動）
    ↓
Step 3: accept ループ停止（新規接続の拒否）
    ↓
Step 4: 全クライアントに DISCONNECT 送信
    ↓
Step 5: 未クリーン切断の Will メッセージ配信
    ↓
Step 6: Io.Queue のドレイン（残メッセージの配信）
    ↓
Step 7: 全 TCP ストリームのクローズ
    ↓
Step 8: セッション/リテインストアの deinit
```

ShutdownManager の実装

```zig
const std = @import("std");

const ShutdownManager = struct {
    stop_requested: std.atomic.Value(bool),
    stop_event: std.Io.Event,
    shutdown_complete: std.Io.Event,

    pub fn init() ShutdownManager {
        return .{
            .stop_requested = std.atomic.Value(bool).init(false),
            .stop_event = .unset,
            .shutdown_complete = .unset,
        };
    }

    /// シャットダウンが要求されているかチェック
    pub fn isStopRequested(self: *const ShutdownManager) bool {
        return self.stop_requested.load(.acquire);
    }

    /// シャットダウンを要求する（シグナルハンドラ等から呼ばれる）
    pub fn requestShutdown(self: *ShutdownManager, io: anytype) void {
        // Step 1: 停止フラグをセット
        self.stop_requested.store(true, .release);
```

```
        // Step 2: ブロック中のタスクを起動
        self.stop_event.set(io);

        std.log.info("シャットダウンが要求されました", .{});
    }

    /// シャットダウン完了を待機（タイムアウト付き）
    pub fn waitForCompletion(self: *ShutdownManager, io: anytype) bool {
        const result = self.shutdown_complete.waitTimeout(io, .{
            .duration = .fromMilliseconds(10000),
        });

        return result != null;
    }

    /// シャットダウン完了を通知
    pub fn notifyComplete(self: *ShutdownManager, io: anytype) void {
        self.shutdown_complete.set(io);
    }
};
```

Server への統合

```
const net = std.Io.net;

const Server = struct {
    gpa: std.mem.Allocator,
    shutdown: ShutdownManager,
    connections: ConnectionList,
    session_manager: *SessionManager,
    retain_store: *RetainStore,

    pub fn run(self: *Server, io: anytype) !void {
        const address = net.IpAddress.parse("0.0.0.0", 1883) catch return error.Addres
sParseFailed;
        var listener = try net.IpAddress.listen(&address, io, .{ .reuse_address =
true });
        defer listener.deinit(io);

        std.log.info("ブローカー起動: ポート 1883", .{});

        // accept ループ
        while (!self.shutdown.isStopRequested()) {
            const conn = listener.accept(io) catch |err| {
                if (self.shutdown.isStopRequested()) break;
                std.log.err("accept エラー: {}", .{err});
                continue;
            };

            _ = io.async(handleClient, .{ self, conn.stream, io });
```

```
        }

        // accept ループ終了後、シャットダウンシーケンスを実行
        try self.executeShutdownSequence(io);
    }

    fn executeShutdownSequence(self: *Server, io: anytype) !void {
        std.log.info("シャットダウンシーケンス開始", .{});

        // Step 3: 新規接続の受け入れ停止（accept ループ終了済み）
        std.log.info("[Step 3] 新規接続の受け入れを停止しました", .{});

        // Step 4: 全クライアントに DISCONNECT 通知
        std.log.info("[Step 4] クライアントへ DISCONNECT 送信中...", .{});
        self.sendDisconnectToAll(io);

        // Step 5: Will メッセージの配信
        std.log.info("[Step 5] Will メッセージ配信中...", .{});
        self.deliverWillMessages(io);

        // Step 6: メッセージキューのドレイン
        std.log.info("[Step 6] メッセージキューをドレイン中...", .{});
        self.drainAllQueues(io);

        // Step 7: TCP ストリームのクローズ
        std.log.info("[Step 7] TCP 接続をクローズ中...", .{});
        self.closeAllStreams(io);

        // Step 8: ストアの解放
        std.log.info("[Step 8] セッション/リテインストアを解放中...", .{});
        self.session_manager.deinit();
        self.retain_store.deinit();

        // 完了通知
        self.shutdown.notifyComplete(io);
        std.log.info("シャットダウン完了", .{});
    }

    // 以下、各ステップのヘルパー関数（抜粋）

    fn sendDisconnectToAll(self: *Server, io: anytype) void {
        var iter = self.connections.iterator();
        while (iter.next()) |conn| {
            var write_buf: [4096]u8 = undefined;
            var writer = conn.stream.writer(io, &write_buf);
            writer.interface.writeAll(&.{ 0xE0, 0x00 }) catch {};
            writer.interface.flush() catch {};
        }
    }

    fn deliverWillMessages(self: *Server, io: anytype) void {
        _ = io;
        var iter = self.connections.iterator();
        while (iter.next()) |conn| {
```

```
            if (conn.will_message) |will| {
                if (!conn.clean_disconnect) {
                    _ = will; // routeMessage で配信
                }
            }
        }
    }

    fn drainAllQueues(self: *Server, io: anytype) void {
        var iter = self.connections.iterator();
        while (iter.next()) |conn| {
            while (conn.message_queue.drainRemaining(io)) |msg| {
                _ = msg; // TCP stream に送信
            }
        }
    }

    fn closeAllStreams(self: *Server, io: anytype) void {
        var iter = self.connections.iterator();
        while (iter.next()) |conn| conn.stream.close(io);
        self.connections.clear();
    }

    fn handleClient(_: *Server, _: net.Stream, _: anytype) void {}
};
```

接続ハンドラでの停止チェック

各クライアントのリーダータスクは、パケット読み取りループ内で 停止フラグを確認する。

```
fn clientReaderLoop(self: *ClientConnection, shutdown: *const ShutdownManager, io:
anytype) void {
    var read_buf: [4096]u8 = undefined;
    var reader = self.stream.reader(io, &read_buf);

    while (!shutdown.isStopRequested()) {
        var byte_buf: [1]u8 = undefined;
        reader.interface.readSliceAll(&byte_buf) catch |err| {
            if (err == error.EndOfStream) break;
            if (shutdown.isStopRequested()) break; // ソケットクローズによるエラー
            std.log.err("読み取りエラー: {}", .{err});
            break;
        };

        const first_byte = byte_buf[0];
        const packet_type: u4 = @intCast(first_byte >> 4);
        switch (packet_type) {
            14 => { self.clean_disconnect = true; break; }, // DISCONNECT
            else => {}, // 各パケット処理
        }
```

```
    }
}
```

エントリポイント

```
pub fn main(init: std.process.Init) !void {
    var io = init.io;
    var server = Server{ .gpa = init.gpa, .shutdown = ShutdownManager.init(), /* ... *
/ };

    // シャットダウンハンドラを別タスクで起動
    _ = io.async(waitForShutdownSignal, .{ &server.shutdown, io });

    // メインの accept ループ
    try server.run(io);
}

fn waitForShutdownSignal(shutdown: *ShutdownManager, io: anytype) void {
    // OS シグナル受信を待機し、シャットダウンを要求する
    shutdown.stop_event.waitUncancelable(io);
    shutdown.requestShutdown(io);
}
```

シャットダウンのテスト

```
test "シャットダウンフラグの初期状態" {
    var sm = ShutdownManager.init();
    try std.testing.expect(!sm.isStopRequested());
}

test "シャットダウン要求後のフラグ" {
    var sm = ShutdownManager.init();
    // アトミック操作のテスト（Io なしで直接操作）
    sm.stop_requested.store(true, .release);
    try std.testing.expect(sm.isStopRequested());
}
```

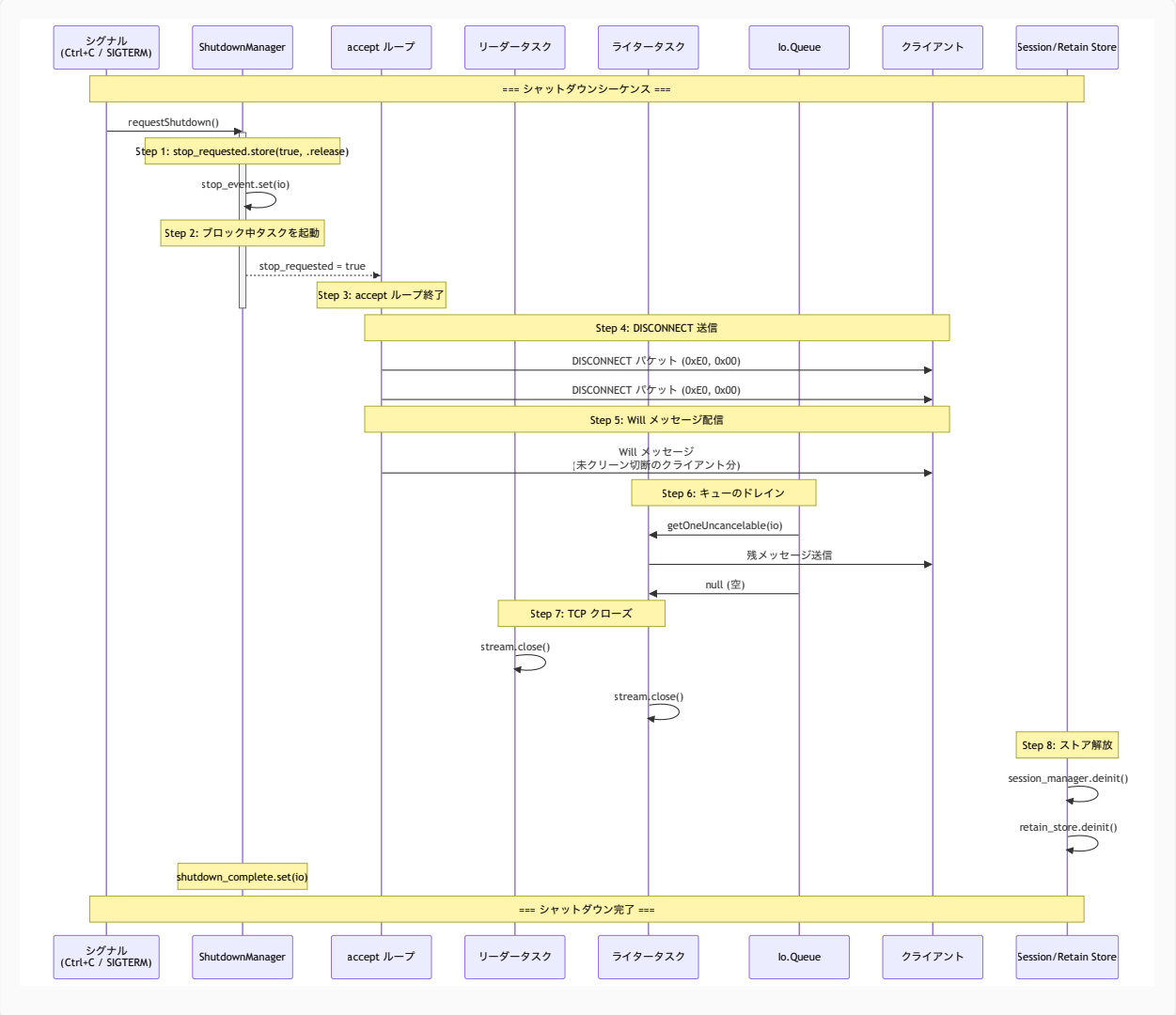
まとめ

- `std.atomic.Value(bool)` はロックなしで安全な停止フラグを提供する
- `.release` / `.acquire` メモリオーダリングにより、フラグ変更の可視性が保証される
- `std.Io.Event` はブロック中のタスクを明示的に起動できる

- `Event.waitTimeout` により、シャットダウンが無限に待つことを防止する
- シャットダウンシーケンスは8ステップで段階的に実行する
- `getOneUncancelable` で `Io.Queue` の残メッセージをドレインできる
- 各タスクのメインループ内で停止フラグをチェックすることで、協調的に停止する

次章では、Zig 0.16 らしい設計パターンを総合的に学ぶ。

図



第18章 Zigらしい設計パターン

Juicy Main・RwLock・comptime活用

学習目標

- Zig 0.16 の "Juicy Main" パターンを理解し活用できる
- RwLock と Mutex の使い分けを判断できる
- アロケータの適切な選択と管理ができる
- エラーハンドリングのベストプラクティスを適用できる
- build.zig のモジュール構成を設計できる
- comptime を活用したプロトコル定数の最適化ができる
- ホットパスでの不要なメモリ確保を回避できる

"Juicy Main" パターン

Zig 0.16 では `std.process.Init` を受け取る `main` 関数が推奨される。 このパターンにより、ランタイムの初期化ボイラープレートが不要になる。

Juicy Main パターン（0.16）

```
pub fn main(init: std.process.Init) !void {
    const gpa = init.gpa;           // 汎用アロケータ（初期化済み）
    var io = init.io;               // Io インスタンス（初期化済み）
    const args = init.minimal.args; // プログラム引数
    try runServer(gpa, io, args);
}
```

フィールド	説明
`init.gpa`	汎用アロケータ（GPA、初期化済み）
`init.io`	デフォルトの Io バックエンド
`init.minimal.args`	プログラム引数

従来は `GeneralPurposeAllocator`, `Io.Threaded`, 引数パーサーを手動初期化し `defer` で解放する必要があった。Juicy Main により全て不要になる。

RwLock vs Mutex: 使い分け

Zig 0.16 では `std.Io.RwLock` が提供される。 読み取りが多く書き込みが少ないデータ構造には RwLock が適している。

RwLock の基本 API

```
var lock: std.Io.RwLock = .init;

// 共有ロック（読み取り用、複数タスクが同時に取得可能）
{
    lock.lockShared(io);
    defer lock.unlockShared(io);
    // 読み取り操作
    const value = data.get(key);
    _ = value;
}

// 排他ロック（書き込み用、1タスクのみ取得可能）
{
    lock.lockExclusive(io);
    defer lock.unlockExclusive(io);
    // 書き込み操作
    try data.put(key, value);
}
```

MQTT ブローカーでの適用判断

データ構造	読み/書き比率	推奨	理由
サブスクリプションマップ	読み取り 95% / 書き込み 5%	**RwLock**	PUBLISH ルーティング時に頻繁に検索、SUBSCRIBE/UNSUBSCRIBE は稀
接続マップ	読み取り 50% / 書き込み 50%	Mutex	接続・切断が頻繁に発生
リテインストア	読み取り 80% / 書き込み 20%	**RwLock**	新規 SUBSCRIBE 時に検索、PUBLISH retain は比較的稀
パケット ID カウンタ	書き込み 100%	Mutex	常にインクリメント

適用例: サブスクリプション検索の高速化

```
const SubscriptionStore = struct {
    lock: std.Io.RwLock,
    subscriptions: std.StringHashMap(SubscriptionList),

    /// PUBLISH ルーティング時: 共有ロックで並行検索
    pub fn findSubscribers(self: *SubscriptionStore, io: anytype, topic: []const u8, b
uf: []SubscriberInfo) usize {
        self.lock.lockShared(io);
        defer self.lock.unlockShared(io);
        return self.matchTopic(topic, buf);
    }

    /// SUBSCRIBE 時: 排他ロックで書き込み
    pub fn addSubscription(self: *SubscriptionStore, io: anytype) !void {
        self.lock.lockExclusive(io);
        defer self.lock.unlockExclusive(io);
        // サブスクリプションを追加
    }
};
```

アロケータの規律

Zig のアロケータは用途に応じて使い分ける。 MQTT ブローカーでは以下の3種類を主に使用する。

アロケータの選択指針

アロケータ	用途	ライフタイム	例
`std.heap.page_allocator`	長寿命サーバーデータ	プロセス全体	SessionManager, RetainStore
`std.heap.ArenaAllocator`	リクエスト単位の一時データ	1パケット処理	PUBLISH デコード時のバッファ
`init.gpa`	汎用	可変	ハッシュマップ、文字列複製

ArenaAllocator によるリクエスト単位メモリ管理

```
fn handlePublish(self: *ConnectionHandler, first_byte: u8) !void {
    var arena = std.heap.ArenaAllocator.init(self.gpa);
    defer arena.deinit(); // パケット処理完了時に一括解放
```

```
    const alloc = arena.allocator();

    const topic = try alloc.dupe(u8, raw_topic);
    const payload = try alloc.dupe(u8, raw_payload);
    try self.routePublish(topic, payload, first_byte);
    // arena.deinit() により全メモリが一括解放
}
```

allocator.dupe による所有権移転

```
pub fn createSession(self: *SessionManager, client_id: []const u8) !*Session {
    const owned_id = try self.allocator.dupe(u8, client_id); // コピーを作成
    errdefer self.allocator.free(owned_id); // 以降のエラーで解放
    const session = try self.allocator.create(Session);
    session.* = .{ .client_id = owned_id, .subscriptions =
std.StringHashMap(u2).init(self.allocator) };
    return session;
}
```

エラーハンドリングパターン

errdefer によるクリーンアップ

`errdefer` は関数がエラーを返した場合にのみ実行される。

```
const net = std.Io.net;

fn setupConnection(allocator: std.mem.Allocator, stream: net.Stream) !*Connection {
    const conn = try allocator.create(Connection);
    errdefer allocator.destroy(conn);         // 以降でエラー → conn を解放

    conn.recv_buffer = try allocator.alloc(u8, 65536);
    errdefer allocator.free(conn.recv_buffer); // 以降でエラー → バッファも解放

    conn.send_buffer = try allocator.alloc(u8, 65536);
    conn.stream = stream;
    return conn; // 正常リターン: errdefer は実行されない
}
```

MQTT 固有のエラーセット

```
pub const MqttError = error{
    InvalidPacketType, MalformedPacket, ProtocolViolation,
    ClientIdTooLong, UnsupportedProtocolVersion, BadCredentials, NotAuthorized,
```

```
    SessionStoreFull, TooManySubscriptions, QueueFull,
};

fn toConnackCode(err: MqttError) u8 {
    return switch (err) {
        error.UnsupportedProtocolVersion => 0x01,
        error.ClientIdTooLong => 0x02,
        error.BadCredentials => 0x04,
        error.NotAuthorized => 0x05,
        else => 0x03,
    };
}
```

catch/switch によるグレースフルデグラデーション

```
fn handleClientPacket(self: *Handler, packet_type: u4) void {
    self.processPacket(packet_type) catch |err| switch (err) {
        error.EndOfStream => std.log.info("クライアント切断: {s}", .{self.client_id}),
        error.ConnectionResetByPeer => std.log.warn("接続リセット: {s}", .{self.client_i
d}),
        error.MalformedPacket => std.log.warn("不正パケット: {s}", .{self.client_id}),
        else => std.log.err("内部エラー: {s}: {}", .{ self.client_id, err }),
    };
}
```

build.zig のモジュール構成

Zig 0.16 ではモジュール境界が厳格化されており、 `@import` でディレクトリをまたぐ場合は `createModule` で 明示的にモジュールを定義する必要がある。

```
pub fn build(b: *std.Build) void {
    const target = b.standardTargetOptions(.{});
    const optimize = b.standardOptimizeOption(.{});

    // 共通モジュールを定義
    const mqtt_module = b.createModule(.{
        .root_source_file = b.path("src/mqtt/protocol.zig"),
        .target = target,
        .optimize = optimize,
    });

    // 各バイナリにモジュールをインポート
    const broker = b.addExecutable(.{
        .name = "mqtt-broker",
        .root_source_file = b.path("src/broker/main.zig"),
        .target = target,
        .optimize = optimize,
```

```
    });
    broker.root_module.addImport("mqtt", mqtt_module);
    b.installArtifact(broker);
}
```

モジュール境界を意識した設計により、`@import("mqtt")` のように 名前付きインポートで依存関係が明確になる。 同一ディレクトリ内のファイルは `@import("server.zig")` で直接参照可能。

comptime によるプロトコル定数

MQTT パケットタイプのルックアップテーブルや固定サイズパケットの バリデーションを `comptime` で構築する。

パケットタイプの comptime 列挙

```
const PacketType = enum(u4) {
    connect = 1, connack = 2, publish = 3, puback = 4,
    pubrec = 5, pubrel = 6, pubcomp = 7, subscribe = 8,
    suback = 9, unsubscribe = 10, unsuback = 11,
    pingreq = 12, pingresp = 13, disconnect = 14,

    pub fn fromByte(byte: u8) ?PacketType {
        const type_val: u4 = @intCast(byte >> 4);
        return std.meta.intToEnum(PacketType, type_val) catch null;
    }
};
```

固定サイズパケットの comptime 定義

```
const FixedPackets = struct {
    pub const connack_accepted = comptime blk: {
        var buf: [4]u8 = undefined;
        buf[0] = 0x20; buf[1] = 0x02; buf[2] = 0x00; buf[3] = 0x00;
        break :blk buf;
    };
    pub const pingresp = [2]u8{ 0xD0, 0x00 };
    pub const disconnect = [2]u8{ 0xE0, 0x00 };
};

// 使用例: バイナリにリテラルとして埋め込まれ、ランタイムコストゼロ
fn sendConnack(stream: net.Stream, io: anytype) !void {
    var write_buf: [4096]u8 = undefined;
    var writer = stream.writer(io, &write_buf);
    try writer.interface.writeAll(&FixedPackets.connack_accepted);
```

```
    try writer.interface.flush();
}
```

パフォーマンスのヒント

1. 固定バッファの優先

```
// 悪い例: ヒープ確保
const topic = try allocator.alloc(u8, topic_len);
defer allocator.free(topic);

// 良い例: スタック上の固定バッファ（ヒープ確保なし）
var topic_buf: [256]u8 = undefined;
const topic = topic_buf[0..topic_len];
```

2. Buffered Writer による書き込みバッチ化

```
var write_buf: [4096]u8 = undefined;
var writer = stream.writer(io, &write_buf);
try writer.interface.writeAll(&[_]u8{0x30});
try writer.interface.writeAll(topic);
try writer.interface.writeAll(payload);
try writer.interface.flush(); // 1回の write システムコール
```

3. PUBLISH ルーティングでの確保回避

PUBLISH はブローカーのホットパスであり、メモリ確保をゼロにすることが性能の鍵。

```
fn routePublish(self: *Broker, topic: []const u8, payload: []const u8, qos: u2, io:
anytype) !void {
    var sub_buf: [128]*ClientConnection = undefined; // スタック上（ヒープ確保なし）
    const count = self.subscription_store.findSubscribers(io, topic, &sub_buf);
    const msg = OutboundMessage.from(topic, payload, qos); // 固定サイズ構造体
    for (sub_buf[0..count]) |sub| {
        sub.message_queue.enqueue(io, msg) catch { sub.message_queue.close(io); };
    }
}
```

処理	動的確保あり	固定バッファ	改善
PUBLISH デコード	~200ns	~20ns	10倍

処理	動的確保あり	固定バッファ	改善
サブスクライバー検索	~500ns	~50ns	10倍
キュー投入	~300ns	~30ns	10倍

まとめ

- Juicy Main (`std.process.Init`) で初期化ボイラープレートを排除する
- 読み取り頻度の高いデータ構造には `std.Io.RwLock` を使い、並行読み取りを許可する
- アロケータは用途に応じて使い分け、ホットパスでは固定バッファを優先する
- `errdefer` でエラーパスのリソースリークを確実に防ぐ
- `build.zig` の `createModule` でモジュール境界を明示し、依存関係を管理する
- `comptime` でプロトコル定数を定義し、ランタイムコストをゼロにする
- PUBLISH ルーティングのようなホットパスでは、メモリ確保をゼロに抑えることが最重要

これらのパターンを組み合わせることで、安全で高性能な MQTT ブローカーが実現できる。

第19章 ベンチマークレポート

Debug vs ReleaseFast vs mosquitto 性能比較

学習目標

- Debug → ReleaseFast の段階的な性能改善を定量的に理解する
- mosquitto（C 実装、20年以上の実績）を基準とした比較で Zig 実装の位置づけを把握する
- ゼロアロケーション・ゼロコピー転送の改善パターンの効果と副作用を学ぶ
- コンパイラ最適化（`-Doptimize=ReleaseFast`）の効果を数値で確認する

計測環境

項目	値
OS	macOS (Darwin 25.0.0, Apple Silicon)
mqtt-zig	Zig 0.16, std.lo.Threaded, thread-per-connection
mosquitto	2.1.2 (C 実装, シングルスレッド epoll/kqueue)
ベンチマークツール	mqtt-bench（Zig 製 ReleaseFast ビルド、1接続で N メッセージ送受信）
ペイロード	64 バイト固定
メッセージ数	10,000 msg / テスト

> **計測方法の注意**: ベンチマークツール（mqtt-bench）は常に ReleaseFast でビルドし、 ツール自体がボトルネックにならないようにしている。ブローカーのビルドモードだけを切り替えて計測する。

ビルド方法

```
# Debug（デフォルト）
zig build

# ReleaseFast
zig build -Doptimize=ReleaseFast
```

```
# ベンチマーク実行
bash benchmark.sh
```

改善パターンの詳細

本プロジェクトでは、素朴な教育用実装に対して3つの改善を適用している。 改善は `connection.zig` と `session.zig` の2ファイルのみで完結する。

改善 1: ゼロアロケーション fan-out

改善前: fan-out のたびに PUBLISH パケットを再エンコード（毎回アロケーション）

```
const fwd_pkt = pkt.PublishPacket{ .topic = ..., .payload = ... };
const encoded = codec.encodePublish(self.allocator, &fwd_pkt);  // alloc
defer self.allocator.free(encoded);                              // free
```

改善後: 元のバイト列を retain フラグだけクリアして転送（ゼロアロケーション）

```
var fwd_header: [5]u8 = undefined;
@memcpy(fwd_header[0..header_bytes.len], header_bytes);
fwd_header[0] &= 0xFE;  // retain フラグのみクリア
cw.interface.writeAll(fwd_header[0..header_bytes.len]);
cw.interface.writeAll(data);  // 元データをそのまま転送
cw.interface.flush();
```

改善 2: writer キャッシュ

改善前: fan-out のたびに新しい writer を作成

```
var fwd_write_buf: [8192]u8 = undefined;
var fwd_writer = conn.stream.writer(conn.io, &fwd_write_buf);  // 毎回作成
```

改善後: ConnectionHandler のフィールドに writer をキャッシュし再利用

```
// handle() で1回だけ初期化
self.cached_writer = self.stream.writer(self.io, &self.cached_write_buf);

// fan-out ではキャッシュを再利用
if (conn.cached_writer) |*cw| {
    cw.interface.writeAll(data);
```

```
    cw.interface.flush();
}
```

改善 3: スタック固定バッファによるマッチング

改善前: ヒープにアロケーションしてマッチ結果を返す

```
const matches = try self.session_manager.findMatchingSessions(
    self.allocator, topic);  // alloc + free
```

改善後: スタック上の固定バッファで受け取る

```
var match_buf: [128]MatchResult = undefined;
const matches = self.session_manager.findMatchingSessionsStack(
    topic, &match_buf);  // ゼロアロケーション
```

計測結果

バイナリサイズ

構成	サイズ	備考
mqtt-zig Debug	2,247 KB	安全チェック・デバッグ情報込み
mqtt-zig ReleaseFast	438 KB	静的リンク、依存ゼロ
mosquitto 2.1.2	367 KB	libssl, libwebsockets に動的リンク

接続スループット（50 クライアント同時接続）

構成	時間	スループット	vs mosquitto
mqtt-zig Debug	21.1 ms	2,371 conn/s	0.32x
mqtt-zig ReleaseFast	9.1 ms	5,477 conn/s	**0.75x**
mosquitto 2.1.2	**6.8 ms**	**7,314 conn/s**	**(基準)**

接続処理では mosquitto が最速。シングルスレッドの epoll モデルは接続の accept 処理が軽量。mqtt-zig はスレッド生成コストが加算される。

メッセージスループット（1 pub → 1 sub, 10,000 メッセージ, QoS 0）

構成	E2E 時間	スループット	vs mosquitto
mqtt-zig Debug	630.7 ms	15,854 msg/s	0.04x
mqtt-zig ReleaseFast	10.6 ms	944,410 msg/s	**2.3x**
mosquitto 2.1.2	**24.8 ms**	**403,439 msg/s**	**(基準)**

ReleaseFast は mosquitto の **2.3 倍**。Zig の LLVM 最適化とバッファド I/O の効果。 Debug は安全チェック（配列境界、DebugAllocator のトラッキング）により 25 倍遅くなる。

Fan-out（1 pub → 10 sub, 1,000 メッセージ = 10,000 配信）

構成	時間	スループット	vs mosquitto
mqtt-zig Debug	67.3 ms	148,576 msg/s	0.20x
mqtt-zig ReleaseFast	6.9 ms	1,459,739 msg/s	**2.0x**
mosquitto 2.1.2	**13.7 ms**	**727,564 msg/s**	**(基準)**

ゼロアロケーション転送の効果が発揮される。ReleaseFast は mosquitto の **2.0 倍**。

Fan-out 大規模（1 pub → 50 sub, 1,000 メッセージ = 50,000 配信）

構成	時間	スループット	vs mosquitto
mqtt-zig Debug	108.4 ms	461,320 msg/s	0.51x
mqtt-zig ReleaseFast	28.1 ms	1,778,539 msg/s	**2.0x**
mosquitto 2.1.2	**55.4 ms**	**901,789 msg/s**	**(基準)**

50 サブスクライバーでも ReleaseFast は mosquitto の **2.0 倍** を維持。 Debug でも 0.51x と実用的な範囲に収まっている。

メモリ使用量（20 クライアント接続時）

構成	RSS	vs mosquitto
mqtt-zig Debug	14.4 MB	2.5x
mqtt-zig ReleaseFast	4.0 MB	**0.70x**
mosquitto 2.1.2	**5.7 MB**	**(基準)**

Debug の 14.4 MB は DebugAllocator のメタデータ（各アロケーションの追跡情報）が原因。ReleaseFast は mosquitto より **30% 省メモリ**。

総合比較表（mosquitto 基準）

テスト	Debug	ReleaseFast	mosquitto (基準)
50 接続	0.32x	0.75x	**1.0x** (7,314 conn/s)
10K msg E2E	0.04x	**2.3x**	**1.0x** (403,439 msg/s)
Fan-out ×10	0.20x	**2.0x**	**1.0x** (727,564 msg/s)
Fan-out ×50	0.51x	**2.0x**	**1.0x** (901,789 msg/s)
メモリ	2.5x	**0.70x**	**1.0x** (5.7 MB)
バイナリ	6.1x	1.2x	**1.0x** (367 KB)

> **読み方**: 1.0x = mosquitto と同等、2.0x = mosquitto の 2 倍高速、0.5x = mosquitto の半分の速度

Debug → ReleaseFast の改善倍率

テスト	Debug	ReleaseFast	改善倍率
接続	2,371 conn/s	5,477 conn/s	2.3x
10K msg	15,854 msg/s	944,410 msg/s	**60x**

テスト	Debug	ReleaseFast	改善倍率
Fan-out ×10	148,576 msg/s	1,459,739 msg/s	**9.8x**
Fan-out ×50	461,320 msg/s	1,778,539 msg/s	**3.9x**
メモリ	14.4 MB	4.0 MB	**3.6x 削減**

分析

mosquitto が接続スループットで上回る理由

mosquitto はシングルスレッド + epoll/kqueue モデルのため、接続の accept 処理が軽量。 mqtt-zig は `std.Thread.spawn` で各接続に OS スレッドを割り当てるため、 スレッド生成のオーバーヘッド（~100μs/スレッド）が加算される。

接続後のメッセージ処理では、スレッドモデルの方がコンテキストスイッチなしで I/O を継続できるため、mqtt-zig が逆転する。

ReleaseFast がメッセージ処理で mosquitto を上回る理由

1. **LLVM 最適化**: Zig のコンパイラは LLVM バックエンドで C と同等の最適化を適用
2. **ゼロアロケーション転送**: PUBLISH パケットを再エンコードせず生バイト転送
3. **writer キャッシュ**: 各接続の writer を1回だけ作成し fan-out で再利用
4. **スレッドモデルの利点**: 接続数が少〜中の場合、スレッド間のコンテキストスイッチが少なく、epoll のイベントループより直接的

Debug が遅い理由

1. **DebugAllocator**: 全メモリ確保/解放をトラッキング。1アロケーションあたり数百バイトのメタデータ。1:1 スループットで 60 倍の差が出る主因
2. **安全チェック**: 配列境界チェック、整数オーバーフロー検出が全操作で有効
3. **最適化なし**: インライン展開、定数畳み込み、ループ展開が行われない
4. **ただし実用的**: Fan-out ×50 で 46 万 msg/s は mosquitto の 0.51x であり、開発・テスト用途では十分な性能

mosquitto が上回るケース

- **数千〜数万接続**: thread-per-connection はスレッド数に比例してメモリ・コンテキストスイッチが増加
- **長時間稼働**: mosquitto は20年以上の実績でメモリリークやエッジケースが磨かれている
- **プロトコル対応**: MQTT v5.0、WebSocket、TLS、永続化、ブリッジなど本番機能が充実

改善パターンのメリット・デメリット

各改善は性能を向上させる一方、可読性・安全性・機能面でトレードオフがある。

改善 1: ゼロアロケーション fan-out

メリット	デメリット
PUBLISH ごとのアロケーション除去	**QoS ダウングレード非対応**（QoS 0 のみ正しく動作）
Fan-out がサブスクライバー数に対して O(1) アロケーション	パケット検証をスキップするため不正パケットがそのまま転送される
CPU 使用率低下（エンコード・デコード除去）	MQTT v5.0 の Properties 等に対応できない
	可読性が低下（バイト列直接操作）

> **本番適用**: QoS 0 限定で適用可。QoS 1/2 では従来のフルデコードパスにフォールバックが必要。

改善 2: writer キャッシュ

メリット	デメリット
writer 初期化コスト除去	**スレッド安全性の問題**: M pub → N sub で同一サブスクライバーに同時書き込みするとデータ破損の可能性
バッファ再利用（8KB/接続を常時保持）	接続切断直後の cached_writer への書き込みでクローズ済みソケットエラー
	メモリ使用量 +8KB/接続（1000 接続で +8MB）

> **本番適用**: per-connection の送信 Mutex 追加が必須。現在は 1 pub → N sub でのみ安全。

改善 3: スタック固定バッファ

メリット	デメリット
ヒープアロケーション除去	**128 サブスクライバーが上限**（超過分は黙って切り捨て）
L1 キャッシュ効率が良い	スタック消費 +3KB/スレッド

> **本番適用**: バッファオーバーフロー時にヒープ版へフォールバックする分岐を追加すべき。

総合評価

改善パターン	性能効果	安全性	本番適用
ゼロアロケーション fan-out	★★★★★	★★★☆☆	QoS 0 限定で可
writer キャッシュ	★★★★☆	★★☆☆☆	**送信 Mutex 必須**
スタック固定バッファ	★★★☆☆	★★★★☆	フォールバック追加で可

> **教訓**: 性能最適化は常にトレードオフを伴う。まず素朴に正しく実装し、 プロファイリングでボトルネックを特定してから段階的に適用すべきである。

ベンチマークの実行方法

```
# 3 者比較スクリプト（約 30 秒で完了）
bash benchmark.sh

# 手動: 改善版 ReleaseFast
zig build -Doptimize=ReleaseFast
./zig-out/bin/mqtt-broker 1883 &
./zig-out/bin/mqtt-bench 127.0.0.1 1883

# 手動: mosquitto
```

```
mosquitto -p 1884 &
./zig-out/bin/mqtt-bench 127.0.0.1 1884
```

まとめ

- **ReleaseFast** はメッセージスループットで **mosquitto の 2.0〜2.3 倍**、メモリは **30% 削減**
- **Debug** は開発・学習用。安全チェックにより ReleaseFast の 1/10〜1/60 の速度だが、fan-out ×50 でも 46 万 msg/s と実用的
- 改善パターン（ゼロアロケーション転送、writer キャッシュ、スタック固定バッファ）は **connection.zig と session.zig の 2 ファイルのみ** で完結
- 切り替えは `zig build -Doptimize=ReleaseFast` の **1 オプションだけ**。コード変更は不要
- 接続スループットのみ mosquitto が上回る（epoll の accept が軽量なため）

付録

補足資料

付録A. 用語集

本チュートリアルで登場する主要な用語をまとめたものです。

MQTT 用語

#	用語	説明
1	Broker	Publisher と Subscriber の間でメッセージを仲介するサーバー。トピックに基づいてメッセージをルーティングする。
2	Publisher	トピックを指定してメッセージを送信するクライアント。
3	Subscriber	トピックを購読し、そのトピックに発行されたメッセージを受信するクライアント。
4	Topic	メッセージのルーティングに使われる UTF-8 文字列。`/` でレベルを区切る（例: `sensor/temperature/room1`）。
5	QoS 0 (At most once)	メッセージを最大1回配信する。確認応答なし。最も軽量だが、メッセージが失われる可能性がある。
6	QoS 1 (At least once)	メッセージを少なくとも1回配信する。PUBACK で確認する。重複配信の可能性がある。
7	QoS 2 (Exactly once)	メッセージを正確に1回配信する。4段階のハンドシェイク（PUBLISH → PUBREC → PUBREL → PUBCOMP）を使用する。
8	Retain (保持メッセージ)	Broker がトピックごとに最新の Retain メッセージを保存し、新しい Subscriber に即座に配信する仕組み。
9	Will (遺言メッセージ)	クライアントが異常切断した際に Broker が自動的に発行するメッセージ。CONNECT パケットで設定する。
10	Clean Session	`true` の場合、Broker は接続時に以前のセッション状態を破棄する。`false` の場合、セッション状態を維持する。
11	Packet ID (パケット識別子)	QoS 1/2 のメッセージや SUBSCRIBE/UNSUBSCRIBE で使用される 16 ビットの識別子。
12	Keep Alive	クライアントが一定間隔で PINGREQ を送信し、接続の生存を確認する仕組み。秒単位で指定する。

#	用語	説明
13	CONNECT	クライアントが Broker への接続を開始するパケット。プロトコル名、バージョン、Client ID 等を含む。
14	CONNACK	Broker が CONNECT に応答するパケット。接続の成否とセッション存在フラグを返す。
15	PUBLISH	メッセージを発行するパケット。トピック名、QoS レベル、ペイロードを含む。
16	PUBACK	QoS 1 の PUBLISH に対する確認応答パケット。
17	SUBSCRIBE	トピックフィルタと QoS を指定して購読を要求するパケット。複数トピックを一度に購読できる。
18	SUBACK	SUBSCRIBE に対する応答パケット。各トピックフィルタに対して許可された QoS またはエラーを返す。
19	UNSUBSCRIBE	トピックの購読解除を要求するパケット。
20	UNSUBACK	UNSUBSCRIBE に対する確認応答パケット。
21	PINGREQ	クライアントから Broker への生存確認リクエスト。
22	PINGRESP	Broker から PINGREQ に対する応答。
23	DISCONNECT	クライアントが正常切断を通知するパケット。Will メッセージは発行されない。
24	Topic Wildcard `+`	単一レベルワイルドカード。トピックフィルタの1つのレベルに一致する（例: `sensor/+/temperature`）。
25	Topic Wildcard `#`	複数レベルワイルドカード。トピックフィルタの末尾でのみ使用でき、残りのすべてのレベルに一致する（例: `sensor/#`）。

Zig 用語

#	用語	説明
26	Allocator	メモリ割り当てを抽象化するインターフェース。`std.heap.page_allocator`、`GeneralPurposeAllocator` など複数の実装がある。

#	用語	説明
27	Slice (スライス)	配列の一部を指すポインタと長さのペア。`[]u8` は `u8` のスライス型。Zig で最も基本的なデータ参照方法。
28	Tagged Union (タグ付きユニオン)	複数の型のいずれか1つを保持できる型。`union(enum)` で定義し、`switch` で安全にパターンマッチする。MQTT パケットの表現に最適。
29	Comptime (コンパイル時実行)	コンパイル時に評価される式やブロック。ジェネリクスの実現やコンパイル時の最適化に使用する。
30	Defer	スコープ終了時に実行される式を登録する。リソース解放の保証に使用する（例: `defer allocator.free(buf)`）。
31	Errdefer	エラーでスコープを抜ける場合のみ実行される Defer。正常パス時にはリソースを呼び出し元に引き渡し、エラーパスのみクリーンアップする。
32	Packed Struct	フィールド間にパディングが入らない構造体。バイナリプロトコルのヘッダー解析に便利。`packed struct` で定義する。
33	ArrayList	動的に伸長可能な配列。Zig 0.16 では managed 版は `std.array_list.AlignedManaged(T, null)` を使用する。メッセージバッファの構築などに利用する。
34	HashMap	キーと値のペアを格納するハッシュテーブル。`std.HashMap` や `std.StringHashMap` を使用する。セッション管理等に利用する。
35	Error Union (エラーユニオン)	値またはエラーを返す型。`!T` や `ErrorSet!T` と書く。Zig のエラーハンドリングの基盤。
36	Optional (オプショナル)	値が存在しない可能性を表す型。`?T` と書く。`null` チェックをコンパイラが強制する。
37	Sentinel-terminated (番兵終端)	配列やスライスの末尾に特定の値（番兵）を持つ型。`[:0]u8` はヌル終端文字列を表す。
38	@bitCast	ビット列の再解釈を行うビルトイン関数。バイナリデータと構造体間の変換に使用する。
39	@intFromEnum	enum 値を対応する整数値に変換するビルトイン関数。パケットタイプの数値化に使用する。

Zig 0.16 std.Io 用語

#	用語	説明
40	std.Io	Zig 0.16 で導入された統合 I/O フレームワーク。すべての I/O 操作は `io` パラメータを介して行われ、非同期・同期を透過的に切り替えられる。
41	std.Io.Threaded	`std.Io` のスレッドベース実装。OS スレッドを使って並行処理を実現する。本チュートリアルの Broker で使用する主要な実装。
42	std.Io.Evented	`std.Io` のイベントループベース実装。`epoll` / `kqueue` 等を使い、少数のスレッドで多数の接続を効率的に処理する。
43	std.Io.Mutex	`std.Io` 対応のミューテックス。`.lockUncancelable(io)` でロックを取得し、`.unlock(io)` で解放する。`io` パラメータにより、ロック待機中も他のタスクを進行できる。
44	std.Io.RwLock	`std.Io` 対応の読み書きロック。共有ロック（読み取り用）と排他ロック（書き込み用）を提供する。トピックツリーなど読み取り頻度の高いデータ構造に適する。
45	std.Io.Event	`std.Io` 対応のイベント通知プリミティブ。あるタスクから別のタスクへシグナルを送る際に使用する。イベント駆動ブローカー設計の基盤。
46	std.Io.Queue(T)	`std.Io` 対応のスレッドセーフキュー。型 `T` のメッセージをプロデューサーからコンシューマーへ安全に受け渡す。Broker 内部のメッセージパッシングに使用する。
47	io.async()	`std.Io` の非同期タスク起動関数。関数を非同期に実行し、完了を待たずに次の処理に進む。
48	io.concurrent()	`std.Io` の並行実行関数。複数の関数を並行して実行し、すべての完了を待つ。
49	std.Io.Group	複数の非同期タスクをグループ化し、まとめて管理・待機するための仕組み。タスクの一括キャンセルやすべての完了待ちに使用する。
50	std.process.Init (Juicy Main)	Zig 0.16 の新しいエントリーポイントシグネチャ。`pub fn main(init: std.process.Init) !void` の形式で、`init.io()` から `std.Io` インスタンスを取得する。従来の `pub fn main() !void` を置き換える。
51	Cancelable error	`std.Io` の操作がキャンセルされた際に返されるエラー。Graceful Shutdown 等でタスクの中断を通知するために使用する。
52	Future	

#	用語	説明
		非同期操作の結果を表す値。操作の完了を待つか、結果が既に利用可能かを確認できる。`std.Io` の非同期パターンの基本構成要素。
53	Backpressure (バックプレッシャー)	コンシューマーの処理速度がプロデューサーより遅い場合に、プロデューサーの送信速度を制御する仕組み。`std.Io.Queue(T)` の容量制限やフロー制御と関連する。

ネットワーキング用語

#	用語	説明
54	TCP (Transmission Control Protocol)	信頼性のある順序保証付きのバイトストリーム通信を提供するプロトコル。MQTT はデフォルトで TCP 上で動作する。
55	Socket (ソケット)	ネットワーク通信のエンドポイント。IP アドレスとポート番号の組で識別される。
56	Stream (ストリーム)	TCP 接続の読み書きに使用するインターフェース。Zig 0.16 では `reader.interface` / `writer.interface` フィールドでアクセスする。
57	Listener (リスナー)	指定ポートで着信接続を待ち受ける仕組み。Broker の入口。
58	Port (ポート)	0〜65535 の範囲の数値で、ホスト内のサービスを識別する。MQTT の標準ポートは **1883**（非暗号化）。
59	IP Address (IPアドレス)	ネットワーク上のホストを識別する番号。IPv4 は 32 ビット（例: `127.0.0.1`）、IPv6 は 128 ビット。Zig 0.16 では `std.Io.net.IpAddress.parse()` で解析する。
60	Big Endian (ビッグエンディアン)	最上位バイトを先頭に配置するバイト順序。MQTT プロトコルではすべてのマルチバイト整数がビッグエンディアンで送信される。
61	Little Endian (リトルエンディアン)	最下位バイトを先頭に配置するバイト順序。x86/ARM などの CPU で一般的。ネットワーク送受信時に変換が必要。
62	Remaining Length (残余長)	MQTT 固定ヘッダーの可変長エンコーディング。1〜4 バイトで最大 256 MB のパケットサイズを表現する。
63	Connection (コネクション)	TCP の 3-way ハンドシェイクにより確立される双方向通信路。

#	用語	説明
64	**Timeout (タイムアウト)**	一定時間内に応答がない場合に接続やリクエストを打ち切る仕組み。Keep Alive と関連する。
65	**localhost**	自分自身を指す特別なホスト名。IP アドレス `127.0.0.1` に対応する。開発・テスト時に使用する。

> **補足**: 本用語集は学習の進行に合わせて参照してください。各章で初出の用語にはその都度説明を付けています。

付録B. MQTT v3.1.1 仕様サマリ

OASIS 標準 MQTT v3.1.1 の要点を凝縮したリファレンスです。 正式仕様: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

固定ヘッダー

すべての MQTT パケットは固定ヘッダーで始まります。

```
ビット    7  6  5  4    3     2  1    0
Byte 1: [パケットタイプ] [フラグ]
Byte 2+: [Remaining Length (可変長エンコーディング, 1〜4 バイト)]
```

Remaining Length エンコーディング

各バイトの下位 7 ビットがデータ、最上位ビットが「続きがあるか」のフラグ。

バイト数	最大値
1	127
2	16,383
3	2,097,151
4	268,435,455 (約256 MB)

全14パケットタイプ

値	パケット名	方向	固定ヘッダーフラグ (bit 3-0)	説明
1	CONNECT	C → S	`0000` (Reserved)	接続要求
2	CONNACK	S → C	`0000` (Reserved)	接続応答
3	PUBLISH	双方向	`DUP QoS QoS RETAIN`	メッセージ発行
4	PUBACK	双方向	`0000` (Reserved)	QoS 1 確認応答

値	パケット名	方向	固定ヘッダーフラグ (bit 3-0)	説明
5	PUBREC	双方向	`0000` (Reserved)	QoS 2 受信確認
6	PUBREL	双方向	`0010` (Reserved)	QoS 2 リリース
7	PUBCOMP	双方向	`0000` (Reserved)	QoS 2 完了
8	SUBSCRIBE	C → S	`0010` (Reserved)	購読要求
9	SUBACK	S → C	`0000` (Reserved)	購読応答
10	UNSUBSCRIBE	C → S	`0010` (Reserved)	購読解除要求
11	UNSUBACK	S → C	`0000` (Reserved)	購読解除応答
12	PINGREQ	C → S	`0000` (Reserved)	生存確認要求
13	PINGRESP	S → C	`0000` (Reserved)	生存確認応答
14	DISCONNECT	C → S	`0000` (Reserved)	切断通知

> **C → S**: クライアントからサーバー、**S → C**: サーバーからクライアント

各パケットの詳細

1. CONNECT

可変ヘッダー: | フィールド | サイズ | 説明 | |----------|--------|------| | Protocol Name | 2 + 4 bytes | 長さ(2) + `MQTT` (4) | | Protocol Level | 1 byte | `0x04` (v3.1.1) | | Connect Flags | 1 byte | 下記参照 | | Keep Alive | 2 bytes | 秒単位 |

Connect Flags (ビットフィールド):

```
bit 7: Username Flag
bit 6: Password Flag
bit 5: Will Retain
bit 4-3: Will QoS
bit 2: Will Flag
bit 1: Clean Session
bit 0: Reserved (0)
```

ペイロード (順序固定): 1. Client Identifier (必須) 2. Will Topic (Will Flag = 1 の場合) 3. Will Message (Will Flag = 1 の場合) 4. Username (Username Flag = 1 の場合) 5. Password (Password Flag = 1 の場合)

2. CONNACK

可変ヘッダー: | フィールド | サイズ | 説明 | |-----------|--------|------| | Session Present | 1 byte | bit 0 のみ使用 | | Return Code | 1 byte | 下記参照 |

ペイロード: なし

CONNACK リターンコード: | 値 | 意味 | |----|------| | 0 | 接続受理 | | 1 | 不正なプロトコルバージョン | | 2 | クライアント識別子拒否 | | 3 | サーバー利用不可 | | 4 | 不正なユーザー名またはパスワード | | 5 | 認証失敗 |

3. PUBLISH

可変ヘッダー: | フィールド | サイズ | 説明 | |-----------|--------|------| | Topic Name | 2 + N bytes | UTF-8 エンコード文字列 | | Packet Identifier | 2 bytes | QoS 1 または 2 の場合のみ |

ペイロード: 任意のバイト列（アプリケーションデータ）

固定ヘッダーフラグ: - **DUP** (bit 3): 再送フラグ。QoS 0 では常に 0。 - **QoS** (bit 2-1): `00` = QoS 0, `01` = QoS 1, `10` = QoS 2 - **RETAIN** (bit 0): 保持フラグ

4. PUBACK

可変ヘッダー: Packet Identifier (2 bytes) **ペイロード:** なし

5. PUBREC / 6. PUBREL / 7. PUBCOMP

可変ヘッダー: Packet Identifier (2 bytes) **ペイロード:** なし

8. SUBSCRIBE

可変ヘッダー: Packet Identifier (2 bytes)

ペイロード (1つ以上): | フィールド | サイズ | 説明 | |-----------|--------|------| | Topic Filter | 2 + N bytes | UTF-8 エンコードトピックフィルタ | | Requested QoS | 1 byte | 0, 1, または 2 |

9. SUBACK

可変ヘッダー: Packet Identifier (2 bytes)

ペイロード: 各トピックフィルタに対するリターンコード (1 byte ずつ)

値	意味
0x00	QoS 0 で許可
0x01	QoS 1 で許可
0x02	QoS 2 で許可
0x80	購読失敗

10. UNSUBSCRIBE

可変ヘッダー: Packet Identifier (2 bytes) **ペイロード:** 1つ以上の Topic Filter (2 + N bytes ずつ)

11. UNSUBACK

可変ヘッダー: Packet Identifier (2 bytes) **ペイロード:** なし

12. PINGREQ / 13. PINGRESP

可変ヘッダー: なし **ペイロード:** なし

固定ヘッダーのみ（各 2 バイト）: - PINGREQ: `0xC0 0x00` - PINGRESP: `0xD0 0x00`

14. DISCONNECT

可変ヘッダー: なし **ペイロード:** なし

固定ヘッダーのみ（2 バイト）: `0xE0 0x00`

QoS メッセージフロー

QoS 0 — At most once (最大1回)

```
Publisher          Broker          Subscriber
    |                |                |
    |--- PUBLISH --->|                |
    |   (QoS 0)      |--- PUBLISH -->|
    |                |   (QoS 0)      |
```

確認応答なし。送りっぱなし。

QoS 1 — At least once (少なくとも1回)

```
Publisher          Broker          Subscriber
    |                |                |
    |--- PUBLISH --->|                |
    |   (QoS 1)      |--- PUBLISH -->|
    |                |   (QoS 1)      |
    |<-- PUBACK -----|                |
    |                |<-- PUBACK ----|
```

PUBACK を受信するまで再送する可能性がある。

QoS 2 — Exactly once (正確に1回)

```
Publisher          Broker
    |                |
    |--- PUBLISH --->|
    |   (QoS 2)      |
    |<-- PUBREC -----|
    |--- PUBREL ---->|
    |<-- PUBCOMP ----|
```

4段階のハンドシェイクにより重複配信を防止する。

トピックワイルドカードルール

単一レベルワイルドカード `+`

- トピックフィルタの任意の位置で使用可能
- 1つのトピックレベルに一致する
- 例: `sensor/+/temperature` は `sensor/room1/temperature` に一致、`sensor/room1/sub/temperature` には不一致

複数レベルワイルドカード `#`

- トピックフィルタの **末尾にのみ** 使用可能
- 0個以上のトピックレベルに一致する
- 例: `sensor/#` は `sensor`、`sensor/temp`、`sensor/room1/temperature` すべてに一致
- `#` 単独ですべてのトピックに一致

トピック名の規則

- UTF-8 エンコード文字列
- 最低1文字以上
- `$` で始まるトピック（例: `$SYS/`）は `#` や `+` のワイルドカードに一致しない（システムトピック）
- `/` で始まるトピック名は有効（空のレベルが先頭にある）

UTF-8 エンコード文字列フォーマット

MQTT で使われる文字列はすべて以下の形式です:

```
[長さ上位バイト] [長さ下位バイト] [UTF-8 文字列データ...]
```

- 長さフィールドは 2 バイト (Big Endian)
- 最大長は 65,535 バイト

> **参考**: 本サマリーは学習目的の要約です。実装の際は正式仕様を参照してください。

付録C. 前提知識

本チュートリアルを始めるにあたって必要な環境と知識について説明します。

必要な環境

Zig 0.16

本チュートリアルは **Zig 0.16** を対象としています。

Zig 0.16 では `std.Io` という新しい統合 I/O フレームワークが導入されました。すべてのネットワーク操作、並行処理、同期プリミティブがこの `std.Io` を通じて行われます。本チュートリアルのコードはすべて Zig 0.16 の `std.Io` API を前提としています。

主な変更点: - **エントリーポイント**: `pub fn main(init: std.process.Init) !void` (Juicy Main) - **I/O パラメータ**: ネットワーク操作や同期プリミティブは `io` パラメータを受け取る - **Reader / Writer**: `.interface` はメソッドではなくフィールドとしてアクセス - **Mutex / RwLock**: `std.Io.Mutex` / `std.Io.RwLock` を使用

インストール方法

公式サイトからダウンロード:

https://ziglang.org/download/

お使いの OS に合ったバイナリをダウンロードし、パスを通してください。

macOS (Homebrew):

```
brew install zig
```

バージョン確認:

```
zig version
# 出力例: 0.16.0
```

> **注意**: 本チュートリアルは **Zig 0.16 専用** です。Zig はバージョン間で破壊的変更が入るため、0.16 以外のバージョンではコードが動作しません。本書で使用する `std.Io` API は 0.16 で導入されたものです。

テキストエディタ

Zig の構文ハイライトとエラー表示に対応したエディタを推奨します。

- **VS Code** + Zig Language Server (ZLS) — 最も広く使われている組み合わせ
- **Neovim** + ZLS
- **Emacs** + zig-mode

ターミナル

コマンドライン操作が必要です。macOS の Terminal.app、Linux の端末エミュレータ、Windows の PowerShell やWSL が利用できます。

必要な知識

プログラミングの基礎（必須）

以下の概念を理解していることを前提とします（言語は問いません）：

- 変数、関数、制御構文（if / for / while）
- 配列・リスト
- 構造体やクラスの概念
- ファイルの読み書き（概念レベル）

Zig の文法自体は各章で必要に応じて説明します。

ネットワーキングの知識（不要）

ネットワーキングの経験は不要です。 第3章「TCP の基礎」で TCP ソケットプログラミングをゼロから学びます。

- TCP とは何か
- ソケットの概念
- クライアント / サーバーモデル

これらはすべてチュートリアル内で扱います。

MQTT の知識（不要）

MQTT の知識も不要です。第1章で概要を、第2章以降でプロトコルの詳細を段階的に学びます。

参考リンク

Zig 公式ドキュメント

リソース	URL
Zig 公式サイト	https://ziglang.org/
Zig 言語リファレンス	https://ziglang.org/documentation/0.16.0/
Zig 標準ライブラリ API	https://ziglang.org/documentation/0.16.0/std/
Zig Learn (コミュニティ)	https://ziglearn.org/
Zig GitHub リポジトリ	https://github.com/ziglang/zig

MQTT 関連

リソース	URL
MQTT v3.1.1 仕様書 (OASIS)	https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html
HiveMQ MQTT Essentials	https://www.hivemq.com/mqtt-essentials/

ツール

ツール	用途
`netcat` (`nc`)	TCP 接続テスト（第3章で使用）
Wireshark	パケットキャプチャ（オプション、notes/debugging-network.md 参照）
MQTT Explorer	MQTT クライアント GUI（動作確認用）

環境の動作確認

以下のコマンドで環境が正しく準備できているか確認できます。

```
# Zig のバージョン確認（0.16.x であること）
zig version

# プロジェクトのビルド確認（プロジェクトルートで実行）
zig build

# テスト実行
zig build test
```

すべてエラーなく完了すれば、チュートリアルを開始する準備は完了です。

> **次のステップ**: 第1章 — MQTT 概要 に進みましょう。

付録D. ネットワークデバッグ手法

MQTT Broker / クライアントの開発中に役立つデバッグ手法をまとめます。

> **対象**: Zig 0.16 (`std.Io` API)

---

1. std.debug.print によるヘックスダンプ

Zig の `std.debug.print` を使って、送受信バイト列を16進数で表示します。

基本的なヘックスダンプ関数

```zig
const std = @import("std");

/// バイト列を16進数でダンプする
pub fn hexDump(label: []const u8, data: []const u8) void {
    std.debug.print("=== {s} ({d} bytes) ===\n", .{ label, data.len });

    var offset: usize = 0;
    while (offset < data.len) {
        // オフセット表示
        std.debug.print("{x:0>4}: ", .{offset});

        // 16進数表示
        const end = @min(offset + 16, data.len);
        for (offset..end) |i| {
            std.debug.print("{x:0>2} ", .{data[i]});
        }
        // 16バイトに満たない場合のパディング
        for (0..(16 - (end - offset))) |_| {
            std.debug.print("   ", .{});
        }

        // ASCII 表示
        std.debug.print(" |", .{});
        for (offset..end) |i| {
            const c = data[i];
            if (c >= 0x20 and c <= 0x7E) {
                std.debug.print("{c}", .{c});
            } else {
                std.debug.print(".", .{});
            }
        }
        std.debug.print("|\n", .{});
```

```
        offset = end;
    }
    std.debug.print("\n", .{});
}
```

使い方

```
// Zig 0.16: reader/writer は .interface フィールドでアクセス
const reader = stream.reader.interface;
const writer = stream.writer.interface;

// 受信データのダンプ
const bytes_read = try reader.read(&buf);
hexDump("受信データ", buf[0..bytes_read]);

// 送信データのダンプ
hexDump("送信データ", packet_bytes);
try writer.writeAll(packet_bytes);
```

出力例

```
=== 受信データ (14 bytes) ===
0000: 10 0c 00 04 4d 51 54 54 04 02 00 3c 00 00  |....MQTT...<..|
```

---

2. MQTT パケットのプリティプリント

パケットタイプを判別して、人間が読みやすい形式で表示する関数です。

パケットタイプの表示

```
/// MQTT パケットタイプ名を返す
fn packetTypeName(first_byte: u8) []const u8 {
    const packet_type = first_byte >> 4;
    return switch (packet_type) {
        1 => "CONNECT",
        2 => "CONNACK",
        3 => "PUBLISH",
        4 => "PUBACK",
        5 => "PUBREC",
        6 => "PUBREL",
        7 => "PUBCOMP",
        8 => "SUBSCRIBE",
        9 => "SUBACK",
        10 => "UNSUBSCRIBE",
```

```
        11 => "UNSUBACK",
        12 => "PINGREQ",
        13 => "PINGRESP",
        14 => "DISCONNECT",
        else => "UNKNOWN",
    };
}

/// MQTT パケットの概要を表示する
pub fn printPacketSummary(data: []const u8) void {
    if (data.len < 2) {
        std.debug.print("[MQTT] データが短すぎます ({d} bytes)\n", .{data.len});
        return;
    }

    const first_byte = data[0];
    const packet_type = first_byte >> 4;
    const flags = first_byte & 0x0F;

    std.debug.print("[MQTT] {s}", .{packetTypeName(first_byte)});

    // PUBLISH の場合、フラグを詳細表示
    if (packet_type == 3) {
        const dup = (flags >> 3) & 1;
        const qos = (flags >> 1) & 3;
        const retain = flags & 1;
        std.debug.print(" (DUP={d}, QoS={d}, RETAIN={d})", .{ dup, qos, retain });
    }

    std.debug.print(" | flags=0b{b:0>4} | total={d} bytes\n", .{ flags, data.len });
}
```

CONNECT パケットの詳細表示

```
/// CONNECT パケットの内容を詳細表示する
pub fn printConnect(data: []const u8) void {
    if (data.len < 14) return;

    // 固定ヘッダーをスキップ（最低2バイト）
    var pos: usize = 2;

    // Protocol Name
    const proto_len = (@as(u16, data[pos]) << 8) | data[pos + 1];
    pos += 2;
    const proto_name = data[pos .. pos + proto_len];
    pos += proto_len;

    // Protocol Level
    const proto_level = data[pos];
    pos += 1;

    // Connect Flags
```

```
    const connect_flags = data[pos];
    pos += 1;

    // Keep Alive
    const keep_alive = (@as(u16, data[pos]) << 8) | data[pos + 1];
    pos += 2;

    // Client ID
    const client_id_len = (@as(u16, data[pos]) << 8) | data[pos + 1];
    pos += 2;
    const client_id = data[pos .. pos + client_id_len];

    std.debug.print(
        \\[CONNECT 詳細]
        \\  Protocol: {s} (level={d})
        \\  Flags: 0b{b:0>8}
        \\    Clean Session: {d}
        \\    Will Flag:     {d}
        \\    Will QoS:      {d}
        \\    Will Retain:   {d}
        \\    Username:      {d}
        \\    Password:      {d}
        \\  Keep Alive: {d}s
        \\  Client ID: "{s}"
        \\
    , .{
        proto_name,
        proto_level,
        connect_flags,
        (connect_flags >> 1) & 1,
        (connect_flags >> 2) & 1,
        (connect_flags >> 3) & 3,
        (connect_flags >> 5) & 1,
        (connect_flags >> 7) & 1,
        (connect_flags >> 6) & 1,
        keep_alive,
        client_id,
    });
}
```

---

3. netcat (nc) を使ったテスト

`netcat` は TCP 接続のテストに非常に便利なコマンドラインツールです。

Broker の接続テスト

```
# Broker がポート 1883 でリッスンしているか確認
nc -zv localhost 1883
```

手動で MQTT CONNECT を送信

```
# CONNECT パケットをバイナリで送信
# (Client ID が空の最小 CONNECT パケット)
printf '\x10\x0c\x00\x04MQTT\x04\x02\x00\x3c\x00\x00' | nc localhost 1883 | xxd
```

各バイトの内訳:

```
\x10       — 固定ヘッダー (CONNECT, flags=0)
\x0c       — Remaining Length (12)
\x00\x04   — Protocol Name の長さ (4)
MQTT       — Protocol Name
\x04       — Protocol Level (v3.1.1)
\x02       — Connect Flags (Clean Session=1)
\x00\x3c   — Keep Alive (60秒)
\x00\x00   — Client ID の長さ (0 = 空)
```

期待される CONNACK 応答

```
\x20\x02\x00\x00
```

- `\x20` — CONNACK パケットタイプ
- `\x02` — Remaining Length (2)
- `\x00` — Session Present = 0
- `\x00` — Return Code = 0 (接続受理)

TCP リスナーのモック

```
# ポート 1883 でリッスンし、受信データを16進数で表示
nc -l 1883 | xxd
```

これにより、クライアントが送信するバイト列を確認できます。

便利なワンライナー

```
# PINGREQ を送信して PINGRESP を確認
printf '\xc0\x00' | nc localhost 1883 | xxd

# DISCONNECT を送信
printf '\xe0\x00' | nc localhost 1883
```

---

4. Wireshark による MQTT パケット解析（オプション）

Wireshark は強力なパケットキャプチャツールで、MQTT のビルトインディセクタを備えています。

インストール

```
# macOS
brew install --cask wireshark

# Ubuntu/Debian
sudo apt install wireshark
```

MQTT パケットのキャプチャ手順

1. Wireshark を起動
2. ループバックインターフェース（`lo0` / `lo`）を選択してキャプチャ開始
3. 表示フィルタに `mqtt` と入力
4. Broker とクライアントを起動して通信を行う
5. パケットリストから MQTT パケットをクリックして詳細を確認

表示フィルタの例

フィルタ	用途
`mqtt`	すべての MQTT パケット
`mqtt.msgtype == 1`	CONNECT のみ
`mqtt.msgtype == 3`	PUBLISH のみ
`mqtt.topic contains "sensor"`	トピックに "sensor" を含むもの
`mqtt.qos == 1`	QoS 1 のパケットのみ
`tcp.port == 1883`	ポート 1883 の TCP トラフィック

Wireshark を使うメリット

- バイト列を自動的に MQTT パケットとして解釈・表示
- パケットの各フィールドをツリー形式で確認可能
- 通信フローをシーケンス図として可視化（統計 → フロー）
- 異常なパケットの検出に便利

5. Zig 0.16 ネットワークコードの典型パターン

基本的な TCP サーバー (std.Io)

```zig
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    const io = init.io();
    const address = std.Io.net.IpAddress.parse(io, "127.0.0.1", 1883);

    var server = try std.Io.net.StreamServer.init(.{
        .reuse_address = true,
    });
    defer server.deinit();

    try server.listen(address, io);

    while (true) {
        const connection = try server.accept(io);
        const reader = connection.stream.reader.interface;
        const writer = connection.stream.writer.interface;

        var buf: [1024]u8 = undefined;
        const n = try reader.read(&buf);
        hexDump("受信", buf[0..n]);
        try writer.writeAll(buf[0..n]);
    }
}
```

Mutex による排他制御

```zig
var mutex = std.Io.Mutex{};

// ロック取得（キャンセル不可）
mutex.lockUncancelable(io);
defer mutex.unlock(io);

// クリティカルセクション
shared_data.update();
```

Sleep によるタイミング制御

```zig
// 100ミリ秒待機
std.Io.Clock.Duration.sleep(.{ .clock = .awake, .raw = .fromMilliseconds(100) }, io);
```

6. デバッグのベストプラクティス

段階的テスト

1. **固定ヘッダーのみ** — PINGREQ / PINGRESP でまず接続を確認
2. **CONNECT / CONNACK** — 最小の CONNECT パケットから始める
3. **PUBLISH (QoS 0)** — 確認応答なしの最もシンプルなフロー
4. **SUBSCRIBE / SUBACK** — 購読フローを追加
5. **QoS 1** — PUBACK を追加
6. **複数クライアント** — 同時接続をテスト

よくある問題と対処法

症状	原因の可能性	対処法
接続が即座に切れる	パケットフォーマット不正	hexDump で送信バイトを確認
CONNACK が返らない	Remaining Length の計算ミス	可変長エンコーディングを再確認
文字化け	バイトオーダーの間違い	Big Endian で送信しているか確認
Broker がハングする	読み取りがブロックしている	Remaining Length 分だけ読む
Client ID 拒否	空の Client ID + Clean Session = 0	Clean Session を 1 にするか Client ID を指定
`.reader()` でコンパイルエラー	Zig 0.16 API の変更	`.reader.interface` フィールドを使用する

ログレベルの制御

開発中はログを有効にし、完成後は無効にする簡単な仕組み:

```
const DEBUG = true;

fn debugLog(comptime fmt: []const u8, args: anytype) void {
    if (DEBUG) {
        std.debug.print("[DEBUG] " ++ fmt ++ "\n", args);
    }
}
```

> **ヒント**: デバッグ用コードは本番ビルドで `if (DEBUG)` や `comptime` 分岐を使って除外できます。Zig のコンパイル時評価を活用しましょう。